



Departamento de
Computação - **UFSCar**



Departamento de Computação
Centro de Ciências Exatas e Tecnologia
Universidade Federal de São Carlos

Algoritmos e Estruturas de Dados II

Apostila sobre algoritmos de ordenação eficientes, estruturas de dados
avançadas e algoritmos em grafos

Prof. Alexandre Luis Magalhães Levada
Email: alexandre.levada@ufscar.br

Sumário

Prefácio.....	3
Revisão: Complexidade de algoritmos.....	4
Algoritmos de ordenação.....	12
Heapsort.....	12
Shellsort.....	19
Mergesort.....	24
Quicksort.....	28
Limitante Inferior para Ordenação Baseada em Comparações.....	34
Ordenação Não Baseada em Comparações.....	36
Countingsort.....	36
Radixsort.....	38
Bucketsort.....	39
Árvores AVL (Balanceadas).....	44
Estruturas de Dados: Árvores de Busca Digitais (Tries).....	56
Estruturas de Dados: Árvores Rubro-Negras.....	61
Estruturas de Dados: Skip Lists.....	74
Estruturas de Dados: Tabelas de Espalhamento (Hash Tables).....	82
Grafos: Fundamentos Básicos.....	103
Grafos: Busca em Grafos (BFS e DFS).....	108
Busca em Largura (Breadth-First Search – BFS).....	110
Busca em Profundidade (Depth-First Search – DFS).....	113
Grafos: Caminhos Mínimos e o Algoritmo de Dijkstra.....	119
Bibliografia.....	133
Sobre o autor.....	134

Prefácio

O estudo de algoritmos e estruturas de dados é fundamental para programadores, cientistas e engenheiros de computação, pois os capacitam a desenvolver aplicações e métodos computacionais de maneira mais eficiente. Para ajudar a alcançar esse objetivo, essa apostila foi desenvolvida com o intuito de compilar alguns tópicos selecionados sobre análise de algoritmos, estruturas de dados eficientes e algoritmos em grafos.

O conteúdo da apostila está organizado como segue: o capítulo 1 apresenta uma revisão sobre a complexidade de algoritmos e a notação Big-O. O capítulo 2 descreve algoritmos de ordenação eficientes e análises de suas complexidades para o pior caso, caso médio e melhor caso, além de discutir limitantes inferiores para ordenação com comparações e os algoritmos com complexidade linear. O capítulo 3 trata das árvores balanceadas (AVL) e algoritmos para rotação de nós. Em seguida, no capítulo 4 apresentamos árvores rubro-negras, que são árvores balanceadas alternativas as árvores AVL com diversas propriedades interessantes. O capítulo 5 introduz as árvores de busca digitais (Tries) que definem a posição dos elementos na estrutura a partir dos bits de suas chaves. O capítulo 6 apresenta as skip lists, que são estruturas de dados probabilísticas eficientes baseadas em uma hierarquia de listas encadeadas. O capítulo 7 trata das tabelas de espalhamento e do filtro de Bloom, que são estruturas de dados eficientes por permitirem a busca por uma chave em tempo constante. O capítulo 8 apresenta algoritmos para busca em grafos: a busca em largura e a busca em profundidade. O capítulo 9 discute como encontrar caminhos mínimos em grafos ponderados a partir dos algoritmos de Dijkstra e Bellman-Ford.

Por fim, gostaria de realizar uma menção especial as aulas do Prof. Mário César San Felice do Departamento de Computação da Universidade Federal de São Carlos que serviram como um guia de referência para a elaboração do material.

Bons estudos!

“Experiência não é o que acontece com você; é o que você faz com o que lhe acontece”
(Aldous Huxley)

Revisão: Complexidade de algoritmos

Na ciência da computação, mais especificamente no estudo de algoritmos, uma das primeiras perguntas que surgem é: como medir a eficiência de algoritmos, ou seja, dados dois algoritmos A_1 e A_2 , como saber qual deles é mais eficiente?

Na realidade, estamos interessados em 2 quantidades:

a) tempo de execução (complexidade de tempo)

b) quantidade de memória utilizada (complexidade de espaço)

A quantidade de memória exigida pelo algoritmo depende basicamente das variáveis alocadas, o que é razoavelmente simples de se determinar. Já o tempo de execução requer uma análise mais formal e aprofundada do algoritmo. Uma dúvida natural de diversos programadores é: porque não podemos apenas cronometrar o tempo gasto pela execução de uma implementação do algoritmo?

→ **Porque essencialmente, esse tempo medido iria depender de diversos fatores externos ao algoritmo, como a linguagem de programação utilizada (C vs Python vs Java) e o hardware da máquina (processador).**

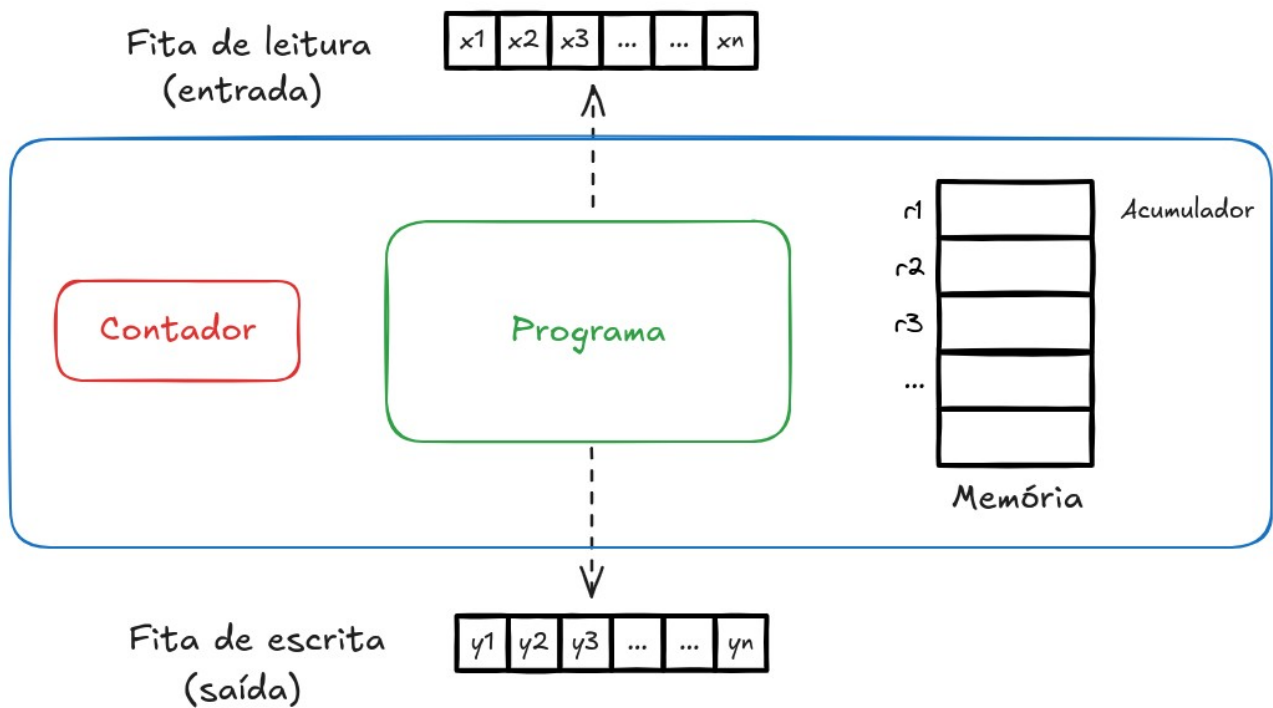
Desejamos realizar uma análise que seja universal, ou seja, dependa de fatores externos ao algoritmo em questão.

Complexidade computacional é um conceito antigo. Foi formulada sistematicamente no início da década de 1960 e, desde então, tem sido universalmente usada como função de custo para projetar algoritmos.

As operações elementares de uma CPU são chamadas de instruções e seus “custos” são chamados de latências. As instruções são armazenadas na memória e executadas uma a uma pelo processador, que possui algum estado interno armazenado em vários registradores. Um desses registradores é o ponteiro de instrução, que indica o endereço da próxima instrução a ser lida e executada. Cada instrução altera o estado do processador de uma certa maneira (incluindo mover o ponteiro da instrução), possivelmente modifica a memória principal e leva um número diferente de ciclos de CPU para ser concluída antes que a próxima possa ser iniciada. Para estimar o **tempo real** de execução de um programa, é necessário somar todas as latências de suas instruções executadas e dividir pela frequência do clock, ou seja, o número de ciclos que uma determinada CPU faz por segundo. Esse processo além de muito complicado, é praticamente inviável.

O modelo RAM (Random Access Machine)

O modelo **RAM** (*Random Access Machine*) é uma abstração de uma CPU, sendo o modelo teórico de computação mais básico já proposto. É um banco potencialmente não vinculado de células de memória, cada uma das quais pode conter um número ou caractere arbitrário. As células da memória são numeradas e leva tempo para acessar qualquer célula da memória, ou seja, todas as operações (leitura/gravação da memória, aritméticas padrão e operações booleanas) levam uma unidade de tempo. **RAM** é um modelo teórico padrão de computação com memória infinita e custo de acesso uniforme (todo acesso a memória tem o mesmo custo). O modelo **RAM** é fundamental para a análise da complexidade de algoritmos. O diagrama a seguir ilustra o modelo **RAM**.



O modelo de computação **RAM** (*Random Access Machine*) mede o tempo de execução de um algoritmo somando o número de etapas necessárias para executar o algoritmo em um conjunto de dados. O modelo **RAM** opera segundo os seguintes princípios:

- Operações lógicas ou aritméticas básicas (+, *, =, if, call) são consideradas operações simples que ocorrem em um intervalo de tempo.
- Loops e sub-rotinas são operações complexas compostas por múltiplos intervalos de tempo.
- Todo acesso à memória ocorre exatamente em um intervalo de tempo.

Na prática, esses princípios são equivalentes a dizer que a CPU possui as instruções LOAD, STORE, READ, WRITE, ADD, SUBTRACT, MULTIPLY, DIVIDE, TEST, JUMP, e HALT e que todas elas são realizadas em tempo constante (mesmo intervalo de tempo fixo e pré-determinado). Este modelo encapsula a funcionalidade central dos computadores modernos, mas não os imita completamente. Por exemplo, uma operação de adição e uma operação de multiplicação possuem um único passo de tempo no modelo RAM; no entanto, na realidade, serão necessárias mais operações para uma máquina calcular um produto versus uma soma. Apesar disso, na prática, o modelo RAM funciona muito bem para o cálculo da complexidade de algoritmos, pois ele é focado na análise assintótica. Como forma de simplificar ainda mais os cálculos matemáticos, **o objetivo consiste em contar o número de operações de atribuição** existentes em um algoritmo A (número de acessos a memória).

Para isso, assume-se algumas hipóteses:

1. Cada comando de atribuição executa em tempo constante, ou seja, possui complexidade $O(1)$.
2. Uma função $T(n)$ deve retornar o número de atribuições a serem executadas quando a entrada do problema resolvido pelo algoritmo possui tamanho n .

Vejamos um simples exemplo a seguir, com uma função que soma os n primeiros inteiros.

```

sum_of_n(n) {
    soma = 0
    for i = 1 to n
        soma = soma + i
    return soma
}

```

É fácil notar que a instrução de inicialização é executada apenas uma vez e que o loop FOR executa n vezes, o que nos leva a:

$$T(n) = n + 1$$

Note que quando n cresce muito, apenas uma parte dominante da função é importante (n).

Notação Big-O

Ao invés de nos preocuparmos em contar exatamente o número de instruções de um programa, é mais tratável matematicamente analisar a ordem de magnitude da função $T(n)$, ou seja, o que acontece com a função $T(n)$ quando n cresce arbitrariamente.

Suponha de exista uma função $f(n)$, definida para todos os inteiros maiores ou iguais a zero, tal que para constantes $c, m > 0$, temos:

$$T(n) \leq c f(n)$$

para $\forall n \geq m$ (n suficientemente grande). Então, dizemos que $T(n)$ é $O(f(n))$ ou ainda:

$$T(n) = O(f(n))$$

Considere o código a seguir, de uma função que soma os elementos de uma matriz quadrada.

```

sum_of_matrix(M, n) {
    soma = 0
    for i = 1 to n {
        soma_linha = 0
        for j = 1 to n
            soma_linha += M[i,j]
        soma += soma_linha
    }
    return soma
}

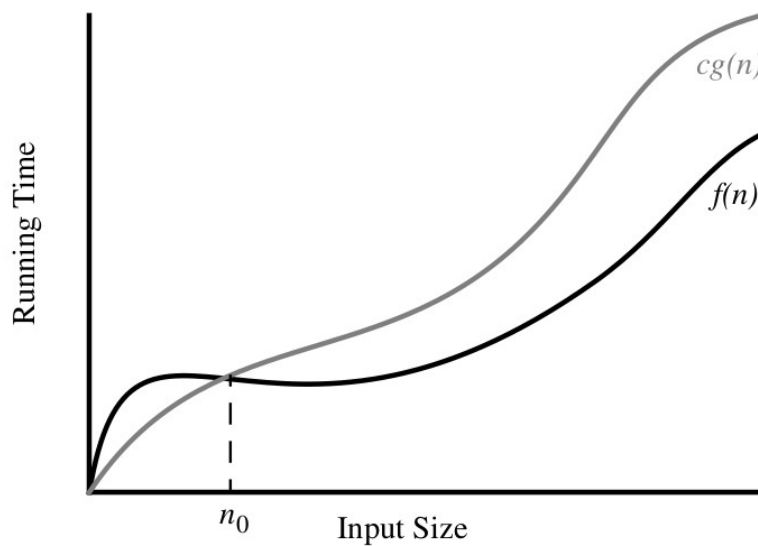
```

Vamos calcular o número de instruções a serem executadas por esse algoritmo. Note que no loop mais interno (j) temos n iterações. Assim para cada valor de i no loop mais externo, temos $n + 2$ atribuições. Como temos n possíveis valores para i , chegamos em:

$$T(n) = n(n+2) = n^2 + 2n + 1$$

Note que se tomarmos $c = 2$ e $f(n) = n^2$, temos $n^2 + 2n + 1 \leq 2n^2$ para todo $n > 2$

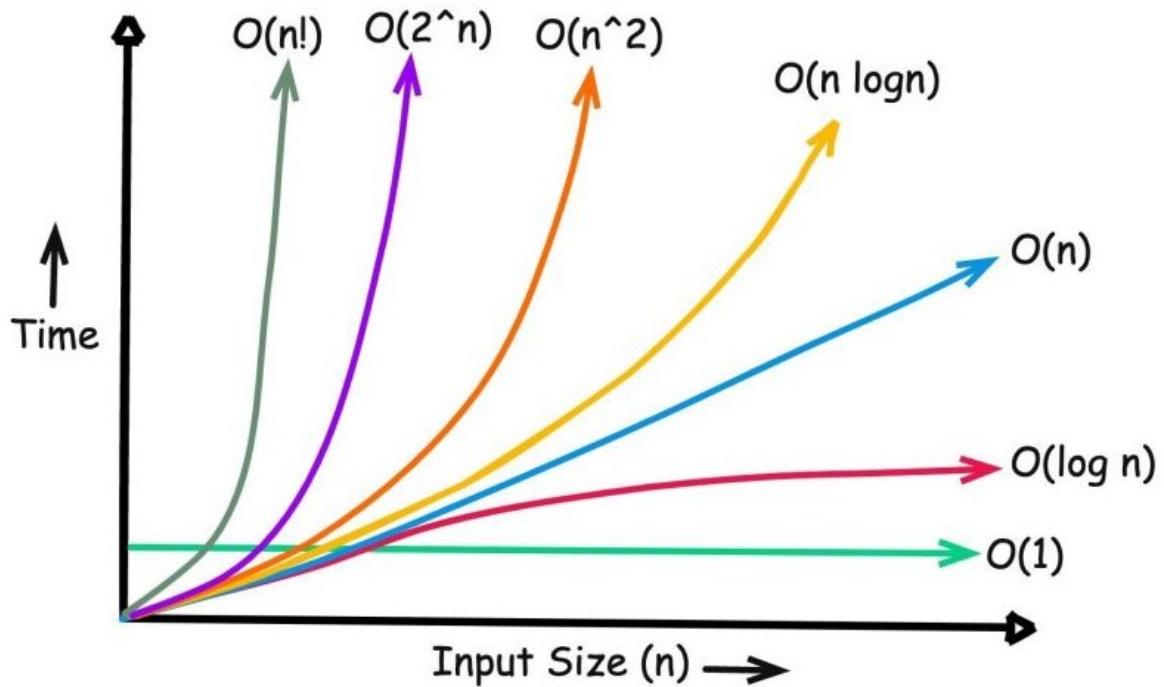
o que implica em dizer que $T(n)$ é $O(n^2)$. Poderíamos ter escolhido a função $f(n) = n^2$, mas o objetivo da notação Big-O é fornecer o limite superior mais justo possível!



Costuma-se dividir os algoritmos nas seguintes classes de complexidade:

$f(n)$	Classe
1	Constante
$\log n$	Logarítmica
n	Linear
$n \log n$	Log-linear
n^2	Quadrática
n^3	Cúbica
n^k	Polinomial
2^n	Exponencial
$n!$	Fatorial

A figura a seguir mostra a taxa de crescimento de algumas dessas funções.



Convém ressaltar que a análise exata de algoritmos é uma tarefa difícil. É da natureza da análise de algoritmo ser independente da máquina e da linguagem. Por exemplo, se o seu computador ficar duas vezes mais rápido após uma atualização recente, a complexidade do seu algoritmo ainda permanecerá a mesma. Além disso, a análise de algoritmos paralelos requer outros modelos de computação mais avançados que o modelo **RAM**.

Analizando algoritmos

Considere o exemplo a seguir, com duas estruturas de repetição em série.

```
Func_A(n) {  
    c = 0  
    for i = 1 to n  
        c += 1  
    for j = 1 to n:  
        c += 2  
    return c  
}
```

Note que temos uma atribuição inicial (1) e logo dois loops com n iterações. Cada um deles, contribui com n para o total, de modo que no total temos $T(n) = 2n + 1$, o que resulta em uma complexidade $O(n)$. Considere o algoritmo a seguir, que utiliza duas estruturas de repetição aninhadas.

```
Func_B(n) {  
    c = 0  
    for i = 1 to n {  
        for j = 1 to n  
            c = c + 1  
        }  
    return c  
}
```

Nesse caso, o loop interno tem n operações. Como o loop externo é executado n vezes, e temos uma inicialização, o total de operações é $T(n) = n^2 + 1$, o que resulta em $O(n^2)$.

E se no loop interno ao invés de n fosse 10?

Teríamos $T(n) = 10n + 1$, o que é $O(n)$.

Vejamos a seguir mais um exemplo com estruturas de repetição aninhadas, onde o loop mais interno depende a variável contadora do loop mais externo.

```
Func_C(n) {  
    count = 0  
    for i = 1 to n {  
        for j = 1 to i  
            c = c + 1  
        }  
    return c  
}
```

Note que quando $i = 0$, o loop interno executa uma vez, quando $n = 1$, o loop interno executa duas vezes, quando $n = 2$, o loop interno executa 3 vezes, e assim sucessivamente. Assim, o número de

vezes que a variável count é incrementada é igual a: $1 + 2 + 3 + 4 + \dots + n$. Devemos resolver esse somatório para calcular a complexidade dessa função.

$$\sum_{k=1}^n k$$

Primeiramente, note que

$$(k+1)^2 = k^2 + 2k + 1$$

o que implica em

$$(k+1)^2 - k^2 = 2k + 1$$

Assim, $\sum_{k=1}^n [(k+1)^2 - k^2] = \sum_{k=1}^n [2k + 1]$. Porém, o lado esquerdo é uma soma telescópica e temos:

$$\sum_{k=1}^n [(k+1)^2 - k^2] = (n+1)^2 - 1$$

Dessa forma, podemos escrever:

$$(n+1)^2 - 1 = \sum_{k=1}^n [2k + 1]$$

Aplicando a propriedade associativa, temos:

$$(n+1)^2 - 1 = 2 \sum_{k=1}^n k + \sum_{k=1}^n 1$$

o que nos leva a:

$$2 \sum_{k=1}^n k = n^2 + 2n + 1 - 1 - n = n^2 + n$$

Finalmente, colocando n em evidência e dividindo por 2, finalmente temos:

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}$$

Portanto, $T(n)$ é igual a:

$$T(n) = \frac{1}{2}(n^2 + n) + 1$$

o que resulta em $O(n^2)$.

O próximo exemplo mostra uma função em que a variável contadora é dividida por 2 a cada iteração.

```

Func_D(n) {
    c = 0
    i = n
    while i > 1 {
        c = c + 1
        i = i // 2      # divisão inteira
    }
    return c
}

```

Essa função calcula quantas vezes o número pode ser dividido por 2. Por exemplo, considere a entrada $n = 16$. Em cada iteração esse valor será dividido por 2, até que atinja o zero.

$16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

A variável c termina a função valendo 4, pois $2^4 = 16$.

Se $n = 25$, temos:

$25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 1$

A variável c termina a função valendo 5, pois $2^4 < 25 < 2^5$

Se $n = 40$, temos:

$40 \rightarrow 20 \rightarrow 10 \rightarrow 5 \rightarrow 2 \rightarrow 1$

A variável c termina a função valendo 5, pois $2^5 < 40 < 2^6$

Portanto, o número de iterações do loop é $\log_2 n$. Dentro do loop existem duas instruções, portanto neste caso teremos:

$T(n) = 1 + 2\lfloor \log_2 n \rfloor$, onde a função piso(x) retorna o maior inteiro menor que x .
o que resulta em $O(\log_2 n)$.

O exemplo a seguir mostra como é possível ter algoritmos com complexidade log-linear.

```

Func_E(n) {
    c = 0
    for i = 1 to n
        c = c + Func_D(n)
    return c
}

```

Note que, como a função $\text{Func_D}(n)$ tem complexidade logarítmica, e o loop tem n iterações, temos que a complexidade da função em questão é $O(n \log_2 n)$.

A seguir apresentamos duas funções para verificar se um número inteiro é primo. Analise a complexidade de cada uma delas e justifique qual delas é mais eficiente.

<pre> prime_A(n) { if n == 1 return False for i = 2 to n-1 { </pre>	<pre> prime_B(n) { if n == 1 return False if n == 2 </pre>
---	--

```

    resto = n % i
    if resto == 0
        return False
}
return True
}

    return True
    if n % 2 == 0
        return False
    i = 3
    while i <= sqrt(n) {
        resto = n % i
        if resto == 0
            return False
        else
            i += 2
    }
    return True
}

```

Primeiramente, vamos analisar o algoritmo prime_A: note que se $n = 0, 1, 2$ ou 3 , o algoritmo tem complexidade $O(1)$, ou seja, temos o melhor caso. Caso contrário, devemos calcular o resto da divisão de n por i ($n - 1 - 2 + 1 = (n - 3 = 2)$ vezes, o que significa que o algoritmo é $O(n)$).

Agora, vamos analisar o algoritmo prime_B: note que se $n = 0, 1, 2$ ou 3 , o algoritmo tem complexidade $O(1)$, ou seja, temos o melhor caso. Caso contrário, o número de atribuições realizadas será $2 \frac{\sqrt{n}-3}{2}$, uma vez que a cada iteração do WHILE, calcula-se o resto e incrementa-se a variável contadora i , o que significa que o algoritmo é $O(n^{1/2})$, e portanto, mais rápido que o algoritmo prime_A ($\sqrt{n}$ cresce um pouco mais rápido que $\log n$, veja a derivada!).

Portanto, o algoritmo prime_B é mais eficiente que o algoritmo prime_A. Em termos práticos, isso significa que em nenhum computador do mundo, menor_A será mais rápido que menor_B para valores de n suficientemente grandes (para n pequeno pode até ser).

"You will never speak to anyone more than you speak to yourself in your head. Be kind to yourself."
 -- Author Unknown

Algoritmos de ordenação

A ordenação de dados é um dos problemas mais importantes da computação.

Problema: ordenação de dados

Entrada: uma sequência arbitrária de chaves $a_1, a_2, a_3, \dots, a_n$ (elementos em uma lista/array)

Saída: uma permutação da sequência de entrada tal que $a_1' \leq a_2' \leq a_3' \leq \dots \leq a_n'$ (ordem crescente)

Uma primitiva básica utilizada nos algoritmos que iremos estudar é a função $\text{swap}(a, b)$, que realiza a troca das posições da chave a com a chave b .

```
swap(a, b) {  
    temp = a  
    a = b  
    b = temp  
}
```

Como podemos perceber, a complexidade desta operação é $O(1)$, o que é algo fundamental no cálculo das complexidades dos algoritmos.

Heapsort

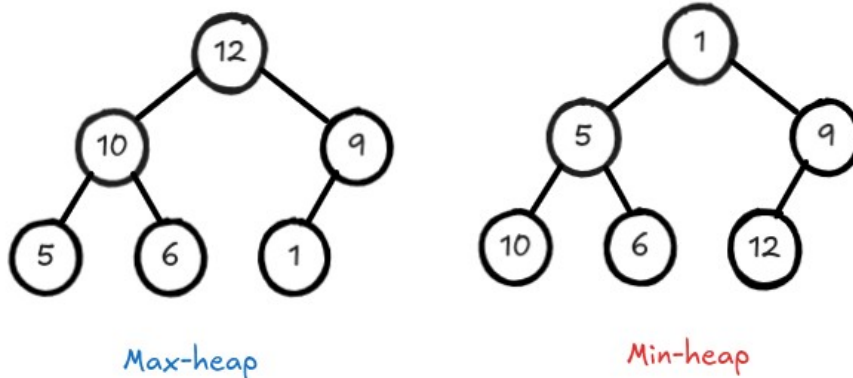
Veremos a seguir como empregar estruturas de dados do tipo heaps para projetar um algoritmo de ordenação eficiente, chamado de Heapsort.

A estrutura de dados Heap

Um Heap é uma estrutura de dados especial baseada em árvores binárias completas. Uma árvore binária completa é uma estrutura de dados do tipo Heap se:

- i. Todo nó i é menor que seus dois filhos (min-heap), ou
- ii. Todo nó i é maior que seus dois filhos (max-heap)

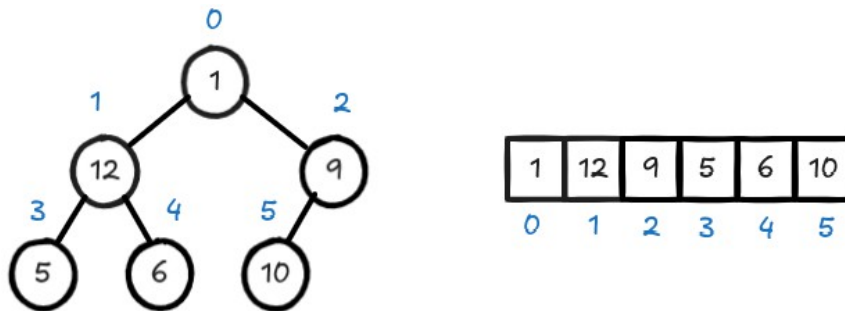
A figura a seguir ilustra um max-heap e um min-heap.



Em um heap de n elementos, se $T[i]$ denota o i -ésimo elemento (nó), então:

- a) Filho a esquerda: $T[2i+1]$, se $2i+1 \leq n$
- b) Filho a direita: $T[2i+2]$, se $2i+2 \leq n$
- c) Pai: $T[\lfloor (i-1)/2 \rfloor]$ (onde $\lfloor . \rfloor$ denota a função piso)

Um exemplo ilustrativo pode ser encontrado na figura a seguir.



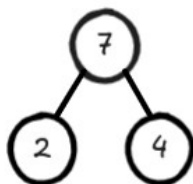
Uma pergunta natural que surge é: como transformar um array arbitrário em um heap? Veremos que existe um processo de heapificação, definido pela função `heapfy`.

Como heapificar um array/árvore?

A partir de uma árvore binária completa, podemos modificá-la para se tornar um max-heap executando uma função chamada `heapify` em todos os elementos não-folha do heap. Para isso assume-se que temos a representação da árvore na forma vetorial. Como a função `heapify` usa recursão, pode ser um pouco complicada de entender a primeira vista.

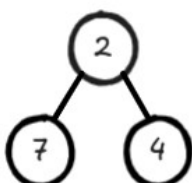
Então, vamos primeiro pensar em um exemplo didático de como você empilharia uma árvore com apenas três elementos. O exemplo a seguir mostra dois cenários - um em que a raiz é o maior elemento e não precisamos fazer nada. Este é o caso base da recursão, onde atingimos a condição de parada. E outro em que a raiz tem um elemento maior que o filho e precisamos trocá-los para manter o max-heap.

Cenário 1

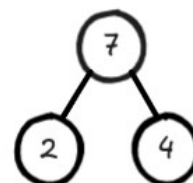


pai já é maior que os 2 filhos

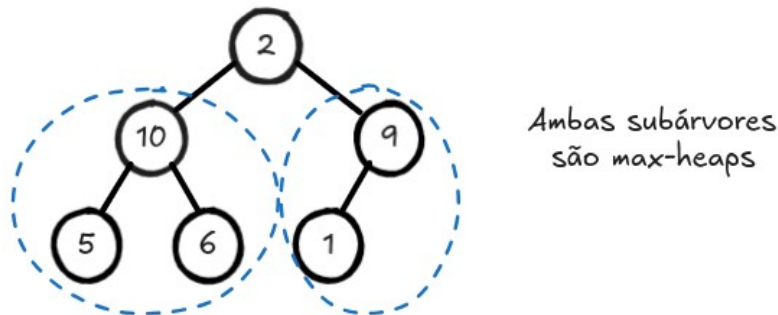
Cenário 2



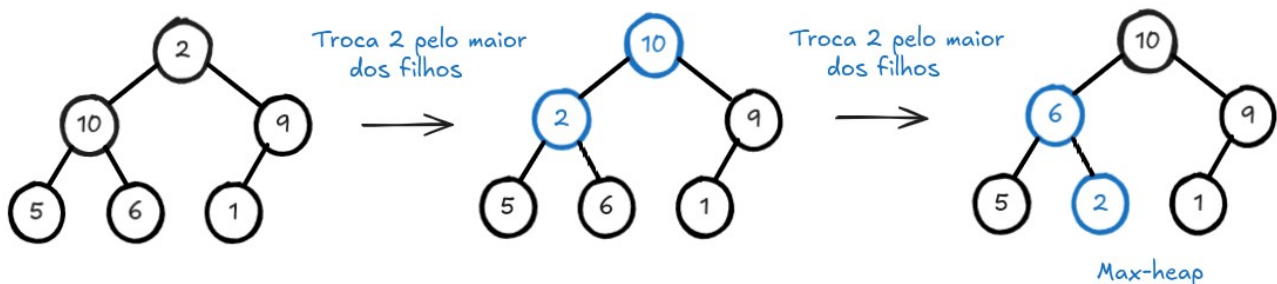
substituir o pai pelo maior dos 2 filhos



Agora vamos pensar em outro cenário em que há mais de um nível no heap. Note que a árvore a seguir não é um max-heap por causa da raiz, mas todas as subárvores (a esquerda e a direita) são max-heaps.



Para manter a propriedade do max-heap para toda a árvore, temos que continuar empurrando o elemento 2 (raiz) para baixo até atingir sua posição correta na estrutura.



Assim, para manter a propriedade max-heap em uma árvore onde ambas as subárvores são max-heaps, precisamos executar heapify no elemento raiz repetidamente até que ele seja maior que seus filhos ou se torne um nó folha. A função a seguir ilustra o algoritmo.

Função recursiva para heapificar um elemento da árvore

```

heapify(L, n, i):
    # Encontra o maior entre o nó raiz i e os filhos
    largest = i
    l = 2 * i + 1
    r = 2 * i + 2
    if l < n and L[i] < L[l]
        largest = l
    if r < n and L[largest] < L[r]
        largest = r
    # Se nó raiz i não é maior, troca e continua heapify
    if largest != i {
        swap(L[i], L[largest])
        heapify(L, n, largest)
    }
  
```

Esta função funciona tanto para o caso base quanto para uma árvore de qualquer tamanho, pois o parâmetro i, define qual é o elemento a ser processado. Podemos, assim, mover o elemento raiz para a posição correta para manter o status do max-heap para qualquer tamanho de árvore, desde que as subárvores sejam max-heaps.

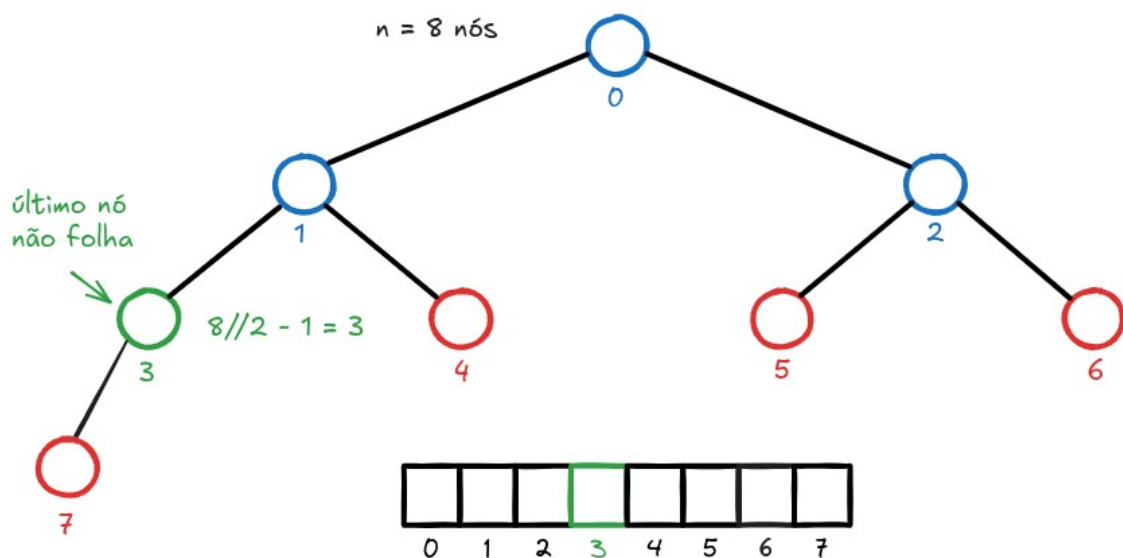
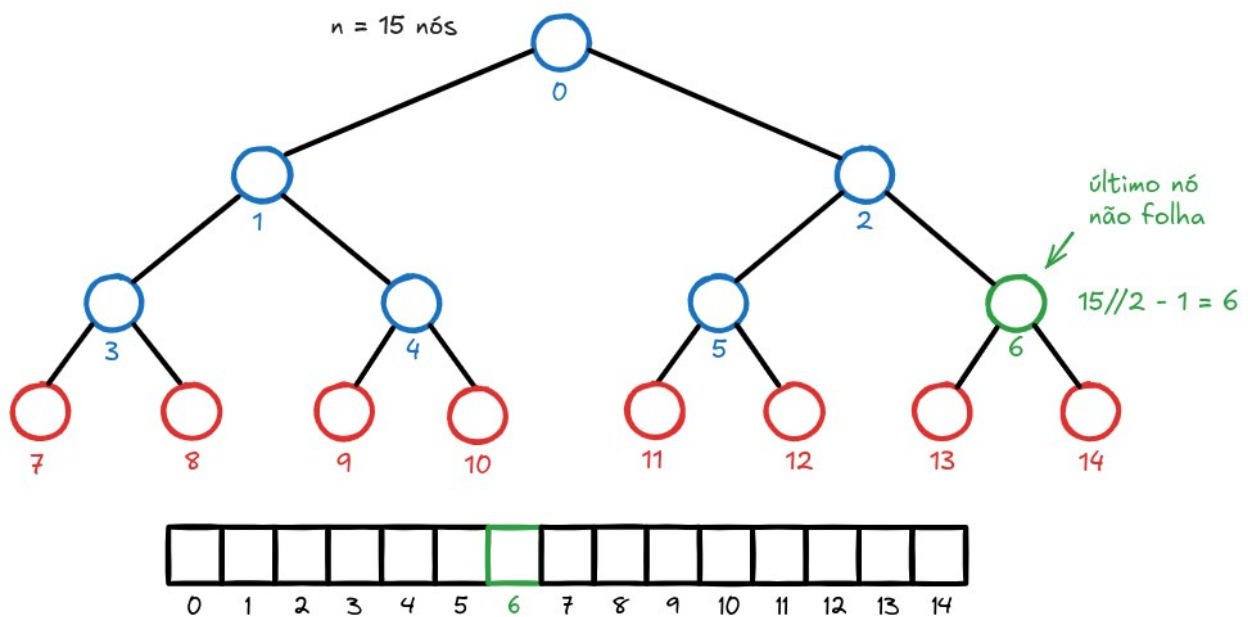
Porém, se as subárvores a esquerda ou a direita da raiz não forem max-heaps, uma única chamada da função heapify não é suficiente para criar um max-heap a partir de um array arbitrário.

Mas então como proceder para criar um max-heap a partir de um array arbitrário?

O índice do último elemento não folha em uma árvore binária completa é dado por $n//2 - 1$, pois a cada nível o número de nós em uma árvore binária dobra.

Por exemplo, em uma árvore binária completa com $n = 15$ nós, o último nó que não é folha é o nó com índice $15//2 - 1 = 7 - 1 = 6$.

Já em uma árvore binária completa de 8 nós, o índice do último nó que não é folha é $n = 8//2 - 1 = 4 - 1 = 3$. As figuras a seguir ilustram esse fato.



Dessa forma, a função para criar um max-heap a partir de um array arbitrário pode ser dada por:

```
# Constrói max heap
build_max_heap(L, n) {
    for i = n//2-1 downto 0
        heapify(L, n, i)
    return L
}
```

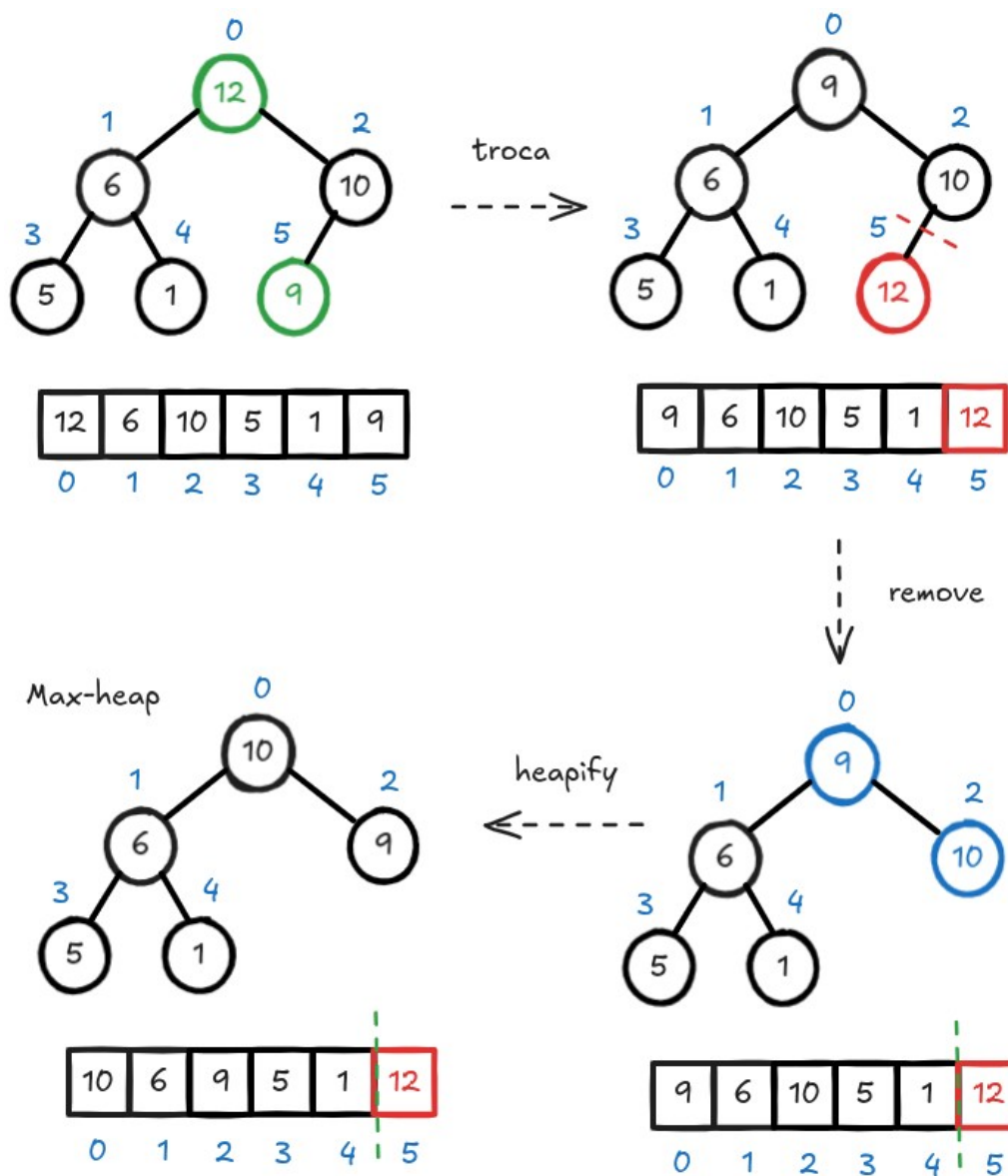
O algoritmo Heapsort pode ser entendido como uma otimização do algoritmo Insertionsort. Em resumo, ele é baseado nos seguintes passos:

1. Inicie heapificando o array inicial para ter um max-heap. Como a árvore em questão, na forma vetorial, satisfaz a propriedade max-heap, o maior elemento está localizado na raiz (índice 0).
2. **Swap:** Coloque o elemento raiz e na última posição do vetor, trocando sua posição com o último elemento.
3. **Remove:** Reduza o tamanho do max-heap em uma unidade.
4. **Heapify:** Aplique a função heapify para trazer o maior elemento para a raiz da árvore.
5. Repita os passos de 2 a 4 até que o vetor esteja completamente ordenado.

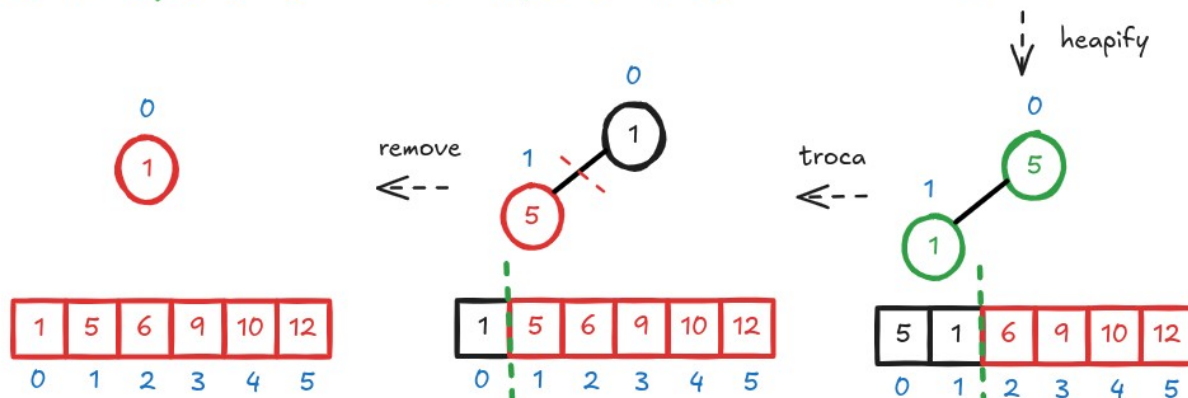
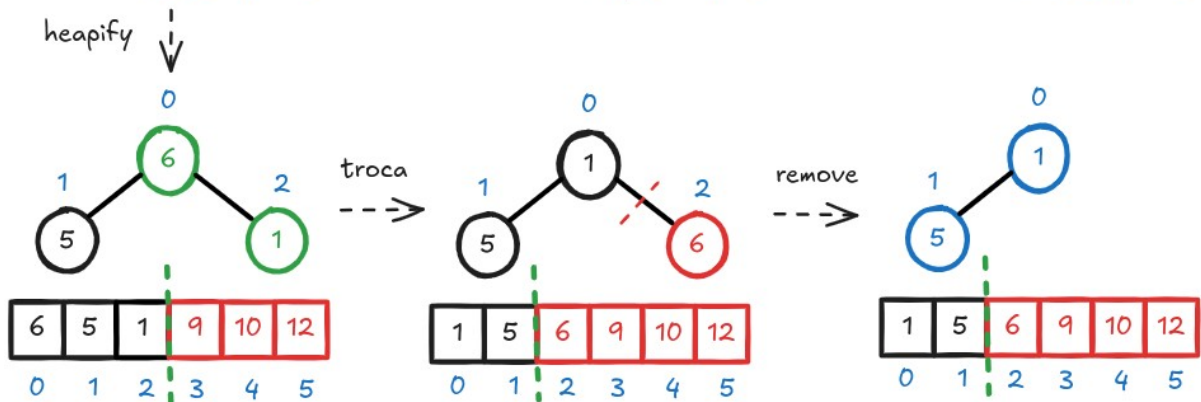
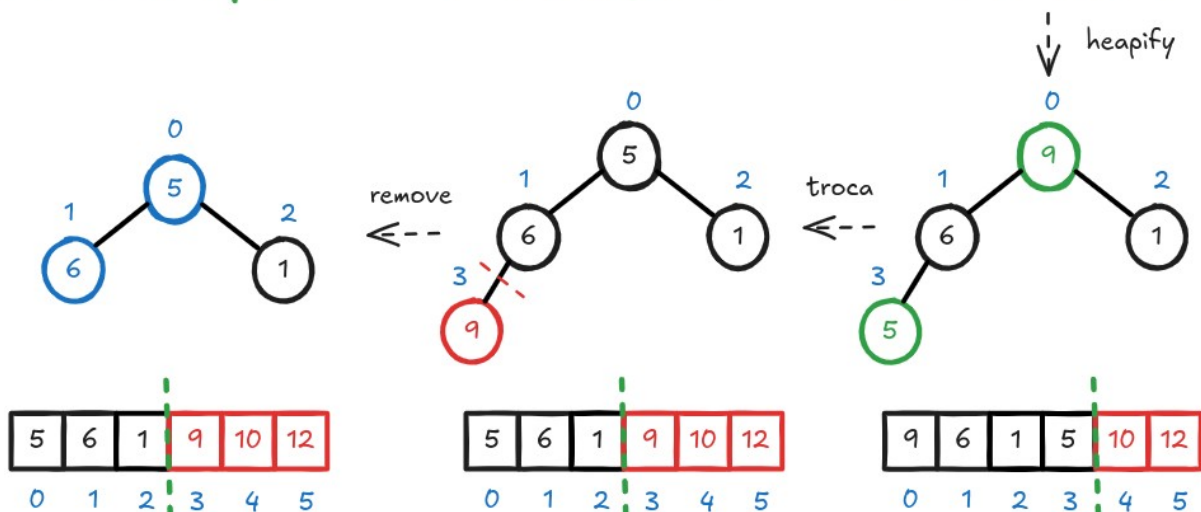
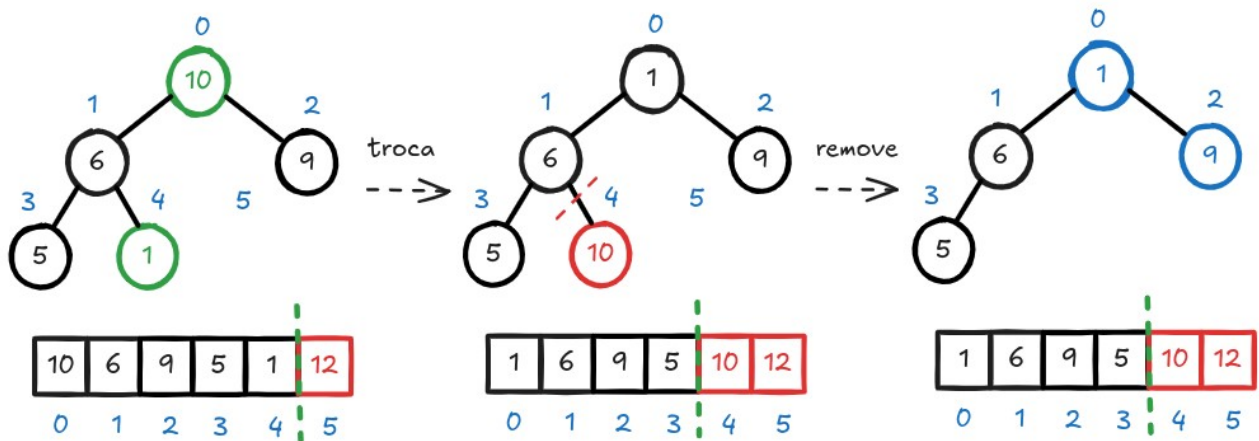
A função a seguir ilustra o pseudocódigo do algoritmo Heapsort, que faz uso das funções auxiliares descritas anteriormente.

```
heapsort(L, n) {
    # Constrói max heap
    L = build_max_heap(L, n)
    for i = n-1 downto 1 {
        # Troca
        swap(L[i], L[0])
        # Heapifica o elemento raiz
        heapify(L, i, 0)
    }
}
```

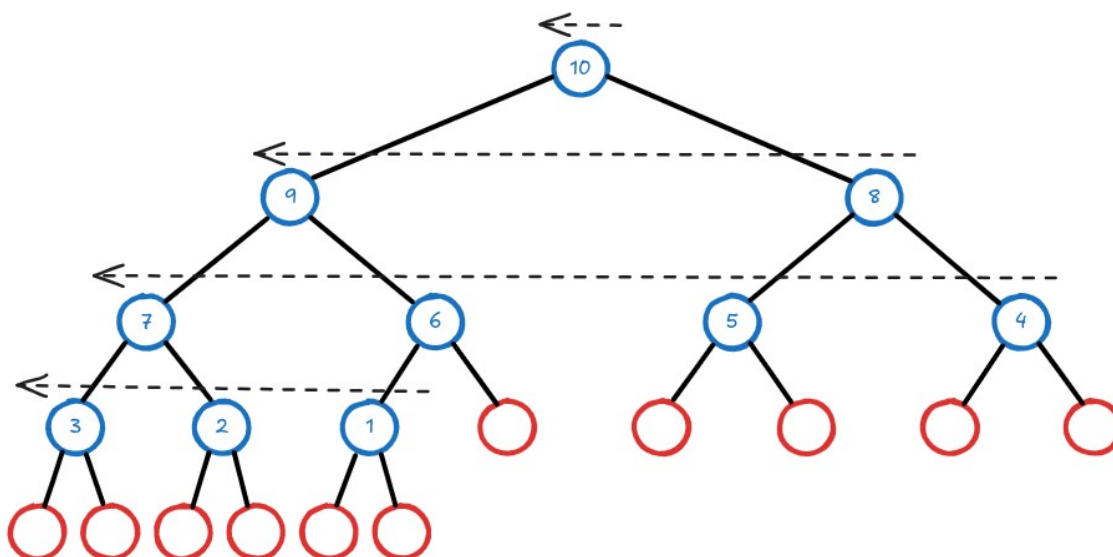
As figuras a seguir ilustram o funcionamento do algoritmo em um exemplo didático.



Considerando o max-heap criado anteriormente, iremos aplicar o algoritmo Heapsort no vetor inicial. As figuras a seguir ilustram o passo a passo do método.



A complexidade da função `heapify()` é $O(\log n)$. Com base nisso, agora podemos computar a complexidade necessária para criar o max-heap. Lembre que para isso devemos percorrer os nós que não são folha, como ilustra a figura a seguir.



Vimos que uma árvore de n nós possui no máximo $n/2$ nós intermediários (não folhas), de modo que a complexidade da construção do max-heap é $O(n/2 \log_2 n) = O(n \log_2 n)$. Portanto, a complexidade de pior caso do Heapsort é $O(n \log_2 n)$, de modo que ele pode ser considerado um algoritmo de ordenação eficiente.

Pode-se mostrar que no melhor caso, a complexidade do algoritmo Heapsort também é $O(n \log_2 n)$, porém os cálculos matemáticos não são de fácil compreensão. Para os leitores interessados, recomenda-se a seguinte referência:

Bollobás, B., Fenner, T. I., Frieze, A. M. On the Best Case of Heapsort, Journal of Algorithms, v. 20, pp. 205-217, 1996.

Disponível em: <https://www.math.cmu.edu/~af1p/Textfiles/Best.pdf>

Dessa forma, se tanto o pior caso como o melhor caso possuem complexidade log-linear, então é evidente que o mesmo deve valer para o caso médio. Em comparação com o Quicksort, sua grande vantagem é ser melhor no pior caso e ocupar menos memória. Por fim, pode-se mostrar que o algoritmo Heapsort não é estável.

Shellsort

O algoritmo Shellsort foi proposto originalmente por Donald Shell como uma otimização do método Insertionsort. A grande limitação do Insertionsort é a quantidade de comparações que devemos fazer até encontrar a posição correta do elemento. Por exemplo, quando o menor elemento está localizado na posição mais a direita do vetor (pior caso), são necessárias $n - 1$ comparações e trocas, o que o torna ineficiente para valores de n grande. A ideia por trás do algoritmo Shellsort é permitir trocas de elementos distantes um do outro no vetor, introduzindo o conceito de gap.

Os itens separados por um gap de h posições são rearranjados, gerando sequências ordenadas. Tais sequências são ditas estarem h -ordenadas. A cada iteração, o valor do gap h é reduzido progressivamente até atingir o valor 1, que resultará no vetor completamente ordenado. Quando a sequência está 1-ordenada, temos o final do algoritmo.

Pode-se mostrar que o Insertionsort é o Shellsort com $h = 1$. Sendo assim, no Shellsort, a ideia é que quando chegamos no $h = 1$, apenas algumas poucas trocas serão necessárias, pois algumas delas já foram feitas com valores maiores de h , tornando o método mais rápido e eficiente do que o Insertionsort.

Portanto, a lógica utilizada é exatamente a mesma do Insertionsort: inserir um elemento na sua posição correta dentro do conjunto. Vejamos um exemplo prático para ilustrar o funcionamento do método. Suponha que temos como entrada o seguinte vetor de inteiros:

[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

Utilizaremos para este exemplo a sequência de gaps original proposta por Shell que consiste de na 1ª iteração usar $n/2$, na 2ª iteração usar $n/4$, na 3ª iteração usar $n/8$, e assim sucessivamente. Como no caso em questão $n = 10$, temos:

=> 1ª iteração (vermelho já está na posição correta, não mexe)

$h = 5$

[5, 2, 13, 7, -3, 4, 15, 10, 1, 6] → [4, 2, 13, 7, -3, 5, 15, 10, 1, 6] (troca)
[5, 2, 13, 7, -3, 4, 15, 10, 1, 6] → [4, 2, 13, 7, -3, 5, 15, 10, 1, 6] (não troca)
[4, 2, 13, 7, -3, 5, 15, 10, 1, 6] → [4, 2, 10, 7, -3, 5, 15, 13, 1, 6] (troca)
[4, 2, 10, 7, -3, 5, 15, 13, 1, 6] → [4, 2, 10, 1, -3, 5, 15, 13, 7, 6] (troca)
[4, 2, 10, 7, -3, 5, 15, 13, 1, 6] → [4, 2, 10, 1, -3, 5, 15, 13, 7, 6] (não troca)

=> 2ª iteração

$h = 2$ (divisão inteira de 10 por 4)

[4, 2, 10, 1, -3, 5, 15, 13, 7, 6] → [-3, 2, 4, 1, 7, 5, 10, 13, 15, 6] (leva para posições corretas)
[-3, 2, 4, 1, 7, 5, 10, 13, 15, 6] → [-3, 1, 4, 2, 7, 5, 10, 6, 15, 13] (leva para posições corretas)

=> 3ª iteração (Insertionsort, mas em um vetor praticamente ordenado)

$h = 1$

[-3, 1, 4, 2, 7, 5, 10, 6, 15, 13] → [-3, 1, 2, 4, 5, 6, 7, 10, 13, 15] (leva para posições corretas)

A seguir é apresentado o algoritmo Shellsort na sua versão padrão.

```
ShellSort(L, n) {  
    # Iniciamos com um gap grande, depois reduzimos  
    gap = n//2  
    # Realiza um Insertionsort utilizando o gap atual  
    # Os primeiros elementos de cada sequência já estão ordenados  
    # Adiciona demais em suas posições corretas  
    while gap > 0 {  
        # não precisa iniciar em 0, pois já está na posição correta  
        for i = gap to n {  
            # iteração do Insertionsort com gap no lugar de 1  
            # encontra posição correta para pivô  
            j = i  
            while j >= gap and L[j-gap] > L[j] {  
                swap(L[j], L[j-gap])  
                j = j - gap  
            }  
        }  
    }
```

```

        # diminui o valor de gap
        gap = gap//2
    }
}

```

Análise da complexidade

Como o algoritmo Shellsort depende essencialmente da sequência de gaps empregada durante a ordenação dos dados, não existe apenas uma complexidade para o algoritmo, mas sim várias. Inclusive, a análise do algoritmo Shellsort no caso médio ainda é um problema em aberto e sem solução até hoje para diversas sequências de gap!

Para efeitos de simplificação dos cálculos, nesta explicação iremos considerar a sequência de gaps de Shell (padrão).

Melhor caso: lista já ordenada. Note que neste caso o loop WHILE interno não é executado nenhuma vez. O que nos resta são os seguintes comandos:

```

gap = n//2
while gap > 0 {
    for i = gap to n
        j = i
        gap = gap//2
}

```

Note que temos aqui o padrão de sucessivas divisões por 2. Logo, temos que o número de iterações do WHILE é $\log_2 n$, o que nos leva a:

$$T(n) = n \log n + \log n$$

o que é $O(n \log n)$.

Pior caso: lista em ordem decrescente. Neste caso, o número máximo de trocas depende do valor do gap. Inicialmente, temos $n/2$ sequências de tamanho 2, depois $n/4$ sequências de tamanho 4, depois $n/8$ sequências de tamanho 8 e assim sucessivamente. Portanto, podemos aproximar $T(n)$ (contando apenas o número de trocas) como:

$$T(n) = \left(n - \frac{n}{2}\right)1 + \left(n - \frac{n}{4}\right)3 + \left(n - \frac{n}{8}\right)7 + \dots + \left(n - \frac{n}{2^k}\right)(2^k - 1)$$

$$T(n) = \frac{1}{2}1n + \frac{3}{4}3n + \frac{7}{8}7n + \dots + \left(\frac{2^k - 1}{2^k}\right)(2^k - 1)n = n \left[\frac{1^2}{2^1} + \frac{3^2}{2^2} + \frac{7^2}{2^3} + \frac{15^2}{2^4} + \dots + \frac{(2^k - 1)^2}{2^k} \right]$$

Iremos denotar por S o somatório entre colchetes:

$$S = \sum_{i=1}^k \frac{(2^i - 1)^2}{2^i} = \sum_{i=1}^k \left[2^i - 2 + \frac{1}{2^i} \right]$$

o que nos leva a:

$$S = \sum_{i=1}^k 2^i - 2 \sum_{i=1}^k 1 + \sum_{i=1}^k \frac{1}{2^i}$$

Denominando o primeiro somatório de A, sabe-se que $2^{i+1} = 2 \cdot 2^i = 2^i + 2^i$, o que nos leva a seguinte identidade:

$$2^i = 2^{i+1} - 2^i$$

Assim, temos a seguinte soma telescópica:

$$A = \sum_{i=1}^k [2^{i+1} - 2^i] = 2^{k+1} - 2 = 2(2^k - 1)$$

É fácil perceber que o segundo somatório em S, denominado de B, é igual a:

$$B = 2 \sum_{i=1}^k 1 = 2k$$

Por fim, o terceiro somatório em S, denominado de C, é uma PG de razão igual a 1/2, cujo primeiro termo é igual a 1/2:

$$C = \frac{a_1(1-q^k)}{(1-q)} = \frac{1/2[1-(1/2)^k]}{(1-1/2)} = 1 - (1/2)^k$$

Logo, podemos expressar S como:

$$S = 2(2^k - 1) - 2k + (1 - (1/2)^k)$$

Note que a condição de parada do Shellsort é o gap ser igual a 1, ou seja, quando $\frac{n}{2^k} = 1$, o que nos leva a $n = 2^k$, o que implica em $k = \log_2 n$. Sendo assim, voltando para S, temos:

$$S = 2(n - 1) - 2 \log_2 n + 1 - \frac{1}{n}$$

Mas como $T(n) = nS$, chegamos em:

$$T(n) = 2n^2 - 2n - 2n \log_2 n + n - 1 \quad \text{o que é } O(n^2).$$

Caso médio: utilizando uma abordagem similar ao pior caso, ao invés de realizar todas as trocas, realiza metade delas (aproximadamente metade do tamanho da lista).

$$T(n) = \left(n - \frac{n}{2}\right)1 + \left(n - \frac{n}{4}\right)2 + \left(n - \frac{n}{8}\right)4 + \dots + \left(n - \frac{n}{2^k}\right)2^{k-1}$$

Aplicando a distributiva, temos:

$$T(n) = \frac{n}{2}(1 + 3 + 7 + 15 + \dots + (2^k - 1))$$

Sabemos que o somatório entre parêntesis vale $A = 2(n - 1) - \log_2 n$, o que nos leva a:

$$T(n) = \frac{n}{2} [2(n-1) - \log_2 n] = n^2 - n - \frac{1}{2} \log_2 n$$

o que resulta em $O(n^2)$ (como no pior caso).

Sequências de gap

Após a proposta original do algoritmo Shellsort, diversos estudos propuseram novas formas de definir os gaps utilizados nas iterações do algoritmo. O consenso geral é de que a cada passo o valor do gap h deve ser diminuído, até que na última iteração, atinja o menor valor possível. Sendo assim, diversas estratégias alternativas para a sequência de gaps mostraram que é possível melhorar o desempenho do algoritmo em termos de complexidade computacional. A seguir mostramos algumas das sequências de gaps propostas posteriormente à descoberta do algoritmo.

1. Sequência de Shell: $N/2, N/4, \dots, 1$ (sucessivas divisões inteiras por 2)

2. Sequência de Hibbard: $1, 3, 7, \dots, 2^k - 1$ (iniciando com $k = 1$)

A ideia aqui consiste em gerar a sequência até o k -ésimo elemento que seja menor ou igual a $N/2$. Os valores dos gaps h são tomados sempre do maior para o menor - $O(n^{3/2})$ no pior caso

3. Sequência de Knuth: $1, 4, 13, \dots, (3^k - 1) / 2$ (iniciando com $k = 1$ e não maior que $n/3$)

A ideia aqui consiste em gerar a sequência até o k -ésimo elemento que seja menor ou igual a $N/2$. Os valores dos gaps h são tomados sempre do maior para o menor - $O(n^{3/2})$ no pior caso

4. Sequência de Pratt: $1, 2, 3, 4, 6, 8, 9, 12, \dots$

A ideia aqui consiste em gerar a sequência até o k -ésimo elemento que seja menor ou igual a $N/2$. Os valores dos gaps h são tomados sempre do maior para o menor.

Sucessivos números da forma $2^p 3^q$. São definidos como os inteiros cujos fatores primos são todos menores ou iguais que 3 - $O(n(\log n)^2)$ no pior caso.

5. Sedgewick's increments: $1, 5, 19, 41, 109, \dots$

A ideia aqui consiste em gerar a sequência até o k -ésimo elemento que seja menor ou igual a $N/2$. Os valores dos gaps h são tomados sempre do maior para o menor - $O(n^{4/3})$ no pior caso.

$$\begin{cases} 9(2^k - 2^{k/2}) + 1, & \text{se } k \text{ par} \\ 8 \cdot 2^k - 6 \cdot 2^{(k+1)/2} + 1, & \text{se } k \text{ ímpar} \end{cases}$$

Existem diversas outras sequências de gaps na literatura, apresentamos aqui as mais conhecidas.

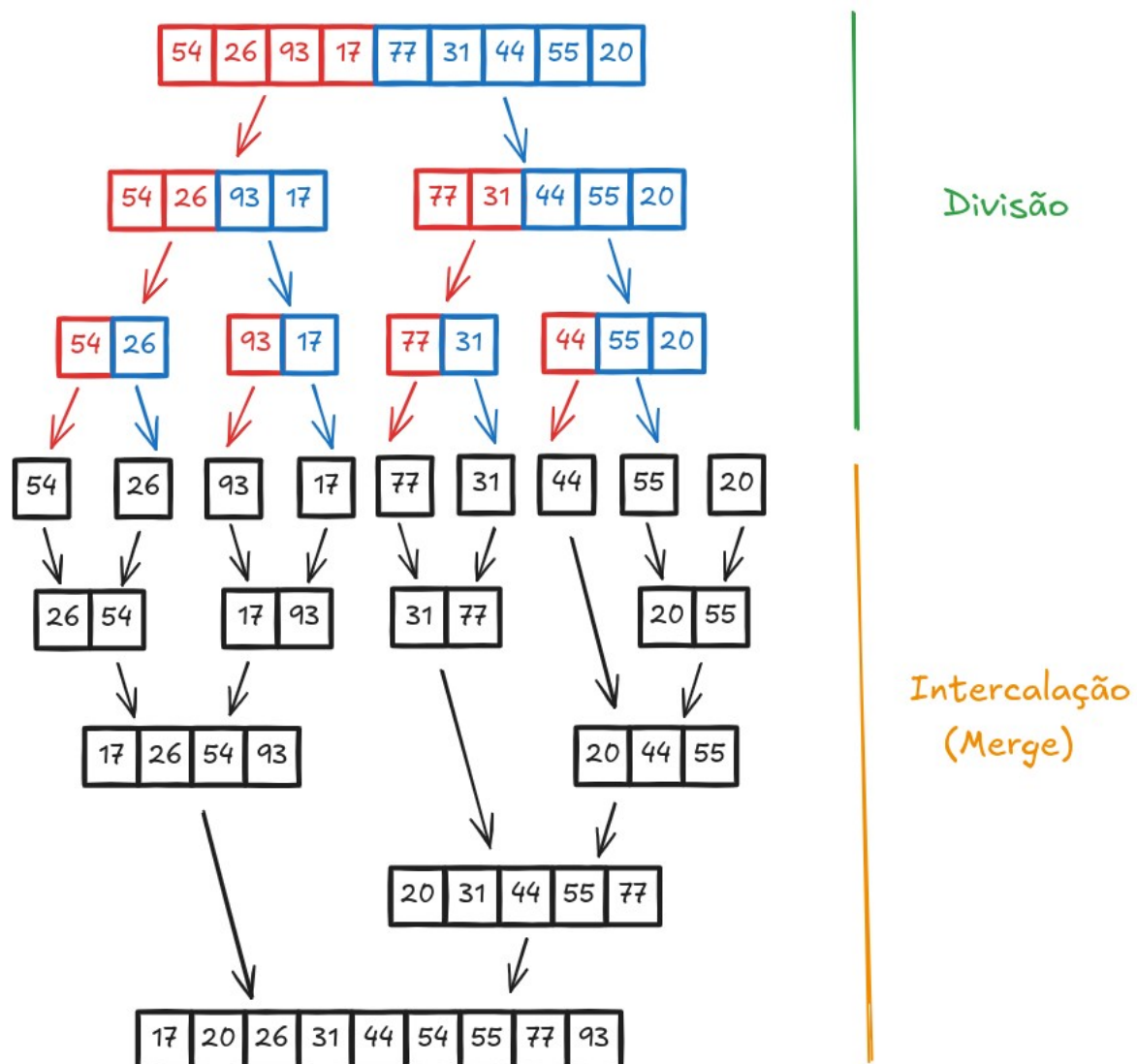
É por isso que diversos autores propuseram utilizar outras sequências de gap. Porém, com essas sequências mais complexas, em geral, não é possível obter expressões analíticas para $T(n)$.

É interessante notar que no pior caso, diferentes sequências de gap são capazes de produzir complexidades muito melhores que $O(n^2)$. Por exemplo, se considerarmos a sequência de Hibbard, pode-se mostrar que a complexidade de pior caso é $O(n^{3/2})$. No caso da sequência de Pratt, pode-se mostrar que a complexidade de pior caso é $O(n \log^2 n)$. Já para a sequência de Sedgwick, pode-se mostrar que a complexidade de pior caso é dada por $O(n^{4/3})$. Conforme mencionado anteriormente, calcular a complexidade do algoritmo Shellsort no caso médio para outras sequências que não a original de Shell, é um problema em aberto cuja solução ainda é desconhecida. Porém, há resultados que comprovam que ela varia entre $O(n^{5/4})$ e $O(n \log^2 n)$. Os dois próximos algoritmos de ordenação que iremos estudar são exemplos de abordagens

recursivas, onde a cada passo, temos um subproblema menor para ser resolvido pela mesma função. Por isso, a função é definida em termos de si própria.

Mergesort

O algoritmo Mergesort utiliza a abordagem Dividir para Conquistar. A ideia básica consiste em dividir o problema em vários subproblemas e resolver esses subproblemas através da recursividade e depois conquistar, o que é feito após todos os subproblemas terem sido resolvidos através da união das resoluções dos subproblemas menores. Trata-se de um algoritmo recursivo que divide uma lista continuamente pela metade. Se a lista estiver vazia ou tiver um único elemento, ela está ordenada por definição (o caso base). Se a lista tiver mais de um elemento, dividimos a lista e invocamos recursivamente um Mergesort em ambas as metades. Assim que as metades estiverem ordenadas, a operação fundamental, chamada de **intercalação**, é realizada. Intercalar é o processo de pegar duas listas menores ordenadas e combiná-las de modo a formar uma lista nova, única e ordenada. A figura a seguir ilustra as duas fases principais do algoritmo Mergesort: a divisão e a intercalação.



O exemplo a seguir mostra o passo a passo para a ordenação da seguinte lista:

[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

Passo 1: Dividir em subproblemas

1ª divisão: [5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

meio = $10 // 2 = 5$

2ª divisão: [5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

meio = $5 // 2 = 2$

3ª divisão: [5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

meio = $2 // 2 = 1$ ou meio = $3 // 2 = 1$

4ª divisão: [5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

meio = $2 // 2 = 1$

Passo 2: Intercalar listas (Merge) – as últimas a serem divididas serão as primeiras a fazer o merge

[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

[5, 2, 13, -3, 7, 4, 15, 10, 1, 6]

[2, 5, -3, 7, 13, 4, 15, 1, 6, 10]

[-3, 2, 5, 7, 13, 1, 4, 6, 10, 15]

[-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]

A seguir apresentamos o algoritmo Merge, que utiliza é responsável por realizar a intercalação de duas sublistas com custo computacional $O(n)$.

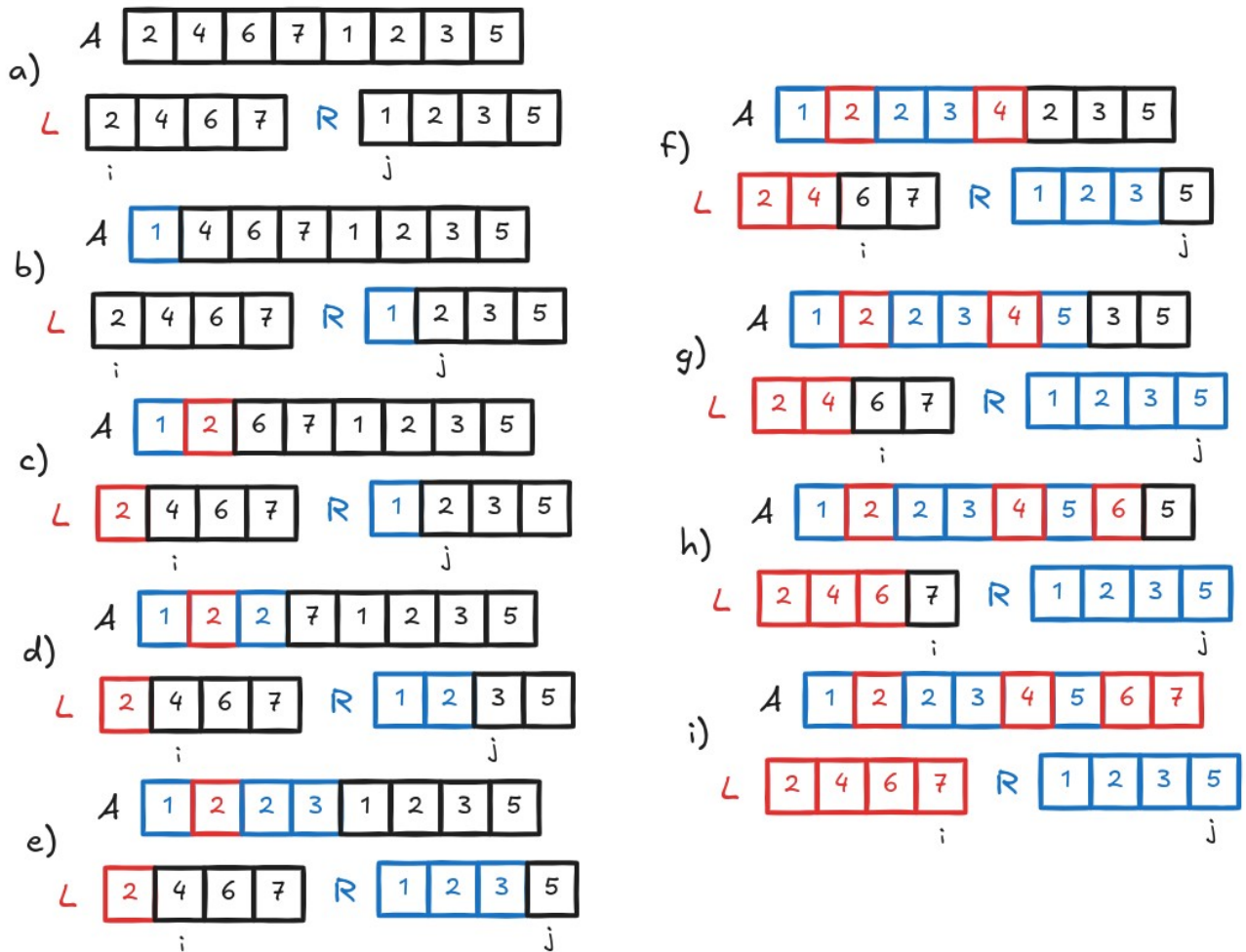
```
merge(A, low, q, high) {  
    nL = q - low + 1          # q é o ponto médio  
    nR = high - q             # número de elementos da sublista esq  
                               # número de elementos da sublista dir  
    Let L[0..nL - 1] be an array  
    Let R[0..nR - 1] be an array  
    for i = 0 to nL - 1  
        L[i] = A[low + i]  
    for j = 0 to nR - 1  
        R[j] = A[(q+1)+j]  
    i = 0  
    j = 0  
    k = 0  
    while i < nL and j < nR {  
        if L[i] ≤ R[j] {  
            A[k] = L[i]  
            i = i + 1  
        }  
        else {  
            A[k] = R[j]  
            j = j + 1  
        }  
        k = k + 1  
    }  
    while i < nL {  
        # se sobrar elementos em L
```

```

    A[k] = L[i]
    i = i + 1
    k = k + 1
}
while j < nR {                                # se sobrar elementos em R
    A[k] = R[j]
    j = j + 1
    k = k + 1
}
}

```

A figura a seguir ilustra o funcionamento da função merge.



A seguir é apresentada a função mergesort, que realiza a ordenação de uma lista.

```

mergesort(A, low, high) {
    if low ≥ high                                # lista tem zero ou um elemento
        return
    q = [(p+q)/2]
    mergesort(A, low, q)
    mergesort(A, q+1, high)
    merge(A, low, q, high)
}

```

A função Mergesort mostrada acima começa perguntando pelo caso base. Se o tamanho da lista for menor ou igual a um, então já temos uma lista ordenada e nenhum processamento adicional é

necessário. Se, por outro lado, o tamanho da lista for maior do que um, então devemos recursivamente aplicar o mergesort nas metades da esquerda e direita. É importante observar que a lista pode não ter um número par de elementos. Isso, contudo, não importa, já que a diferença de tamanho entre as listas será de apenas um elemento.

Quando a função Mergesort retorna da recursão (após a chamada nas metades esquerda, LE, e direita, LD), elas já estão ordenadas. O resto da função é responsável por intercalar as duas listas ordenadas menores em uma lista ordenada maior. Note que a operação de intercalação coloca um item por vez de volta na lista original (L) ao tomar repetidamente o menor item das listas ordenadas.

Análise da complexidade

É interessante notar que o algoritmo Mergesort se comporta da mesma forma para o caso médio, melhor caso e pior caso.

Os três passos úteis dos algoritmos de dividir para conquistar que se aplicam ao MergeSort são:

1. Dividir: Calcula o ponto médio do sub-arranjo, o que demora um tempo constante $O(1)$;
2. Conquistar: Recursivamente resolve dois subproblemas, cada um de tamanho $n/2$, o que contribui com $T(n/2) + T(n/2)$ para o tempo de execução;
3. Combinar: Unir os sub-arranjos em um único conjunto ordenado, que leva o tempo $O(n)$;

Assim, podemos escrever a relação de recorrência como:

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

Trata-se da mesma recorrência resolvida no algoritmo Quicksort. Note que expandindo a recorrência, temos:

$$T(n) = 2T\left(\frac{n}{2}\right) + n$$

$$T\left(\frac{n}{2}\right) = 2T\left(\frac{n}{4}\right) + n$$

$$T\left(\frac{n}{4}\right) = 2T\left(\frac{n}{8}\right) + n$$

$$T\left(\frac{n}{8}\right) = 2T\left(\frac{n}{16}\right) + n$$

...

Voltando com as substituições, podemos escrever:

$$T(n) = 2 \left[2 \left[2 \left[2T\left(\frac{n}{16}\right) + \frac{n}{8} \right] + \frac{n}{4} \right] + \frac{n}{2} \right] + n = 2^4 T\left(\frac{n}{2^4}\right) + 4n$$

Generalizando para um valor k arbitrário, podemos escrever:

$$T(n) = 2^k T\left(\frac{n}{2^k}\right) + kn$$

Para que tenhamos $T(1)$, é preciso que $n = 2^k$, ou seja, $k = \log_2 n$. Quando $k = \log_2 n$, temos:

$$T(n) = 2^{\log_2 n} T(1) + n \log_2 n = n T(1) + n \log_2 n$$

Como $T(1)$ é $O(1)$, temos que a complexidade do Mergesort em qualquer caso é $O(n \log_2 n)$.

Uma das maiores limitações do algoritmo MergeSort é que esse método passa por todo o longo processo mesmo se a lista L já estiver ordenada. Por essa razão, a complexidade de melhor caso é idêntica a complexidade de pior caso, ou seja, $O(n \log n)$. Para o caso de n muito grande, e listas compostas por números gerados aleatoriamente, pode-se mostrar que o número médio de comparações realizadas pelo algoritmo MergeSort é aproximadamente αn menor que o número de comparações no pior caso, onde:

$$\alpha = -1 + \sum_{k=0}^{\infty} \frac{1}{2^k + 1} \approx 0.2645$$

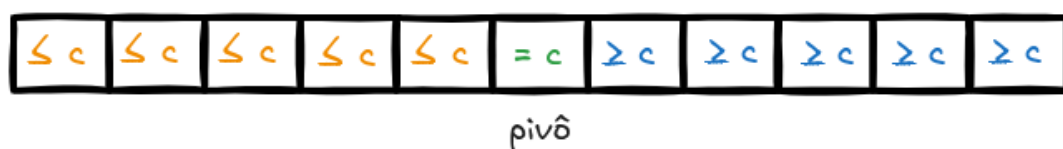
Quicksort

O algoritmo Quicksort é baseado na estratégia “Dividir para Conquistar”, pois ele quebra o problema de ordenar um vetor em subproblemas menores, mais fáceis e rápidos de serem resolvidos. Primeiramente, o método divide o vetor original em duas partes: os elementos menores que o pivô (tipicamente escolhido como o primeiro ou último elemento do conjunto). O método então ordena essas partes de maneira **recursiva**. O algoritmo pode ser dividido em 3 passos principais:

1. Escolha do pivô:

- a) Primeiro elemento da lista
- b) Último elemento da lista
- c) Mediana entre primeiro, central e último elementos (melhor)

2. Particionamento: reorganizar o vetor de modo que todos os elementos menores que o pivô apareçam antes dele (a esquerda) e os elementos maiores apareçam após ele (a direita). Ao término dessa etapa o pivô estará em sua posição final (existem várias formas de se fazer essa etapa)



3. Ordenação: recursivamente aplicar os passos acima aos sub-vetores produzidos durante o particionamento. O caso limite da recursão é o sub-vetor de tamanho 1, que já está ordenado.

O exemplo a seguir mostra os passos necessários para a ordenação do seguinte vetor:

[5, 2, 13, 7, -3, 4, 15, 10, 1, 6]

1º passo: Definir pivô = 6 (último elemento)

2º passo: Particionar vetor (menores a esquerda e maiores a direita)

[5, 2, -3, 4, 1, 6, 13, 7, 15, 10]

3º passo: Aplicar 1 e 2 recursivamente para as metades

a) 2 metades

Metade 1: [5, 2, -3, 4, 1] $\rightarrow$ pivô = 1

`[-3, 1, 5, 2, 4, 6, 13, 7, 15, 10]`

Metade 2: [13, 7, 15, 10] $\rightarrow$ pivô = 10

`[-3, 1, 5, 2, 4, 6, 7, 10, 15, 13]`

b) 4 metades

Note que a metade 1 possui um único elemento: $[-3] \rightarrow$ já está ordenada

Metade 2: $[5, 2, 4] \rightarrow \text{pivô} = 4$

`[-3, 1, 2, 4, 5, 6, 7, 10, 15, 13]`

Note que a metade 3 possui apenas um único elemento: $[7] \rightarrow$ já está ordenadas

Metade 4: $[15, 13] \rightarrow \text{pivô} = 13$

`[-3, 1, 2, 4, 5, 6, 7, 10, 13, 15]`

c) 4 metades: Note que cada uma das 4 metades restantes contém um único elemento e portanto já estão ordenadas. Fim.

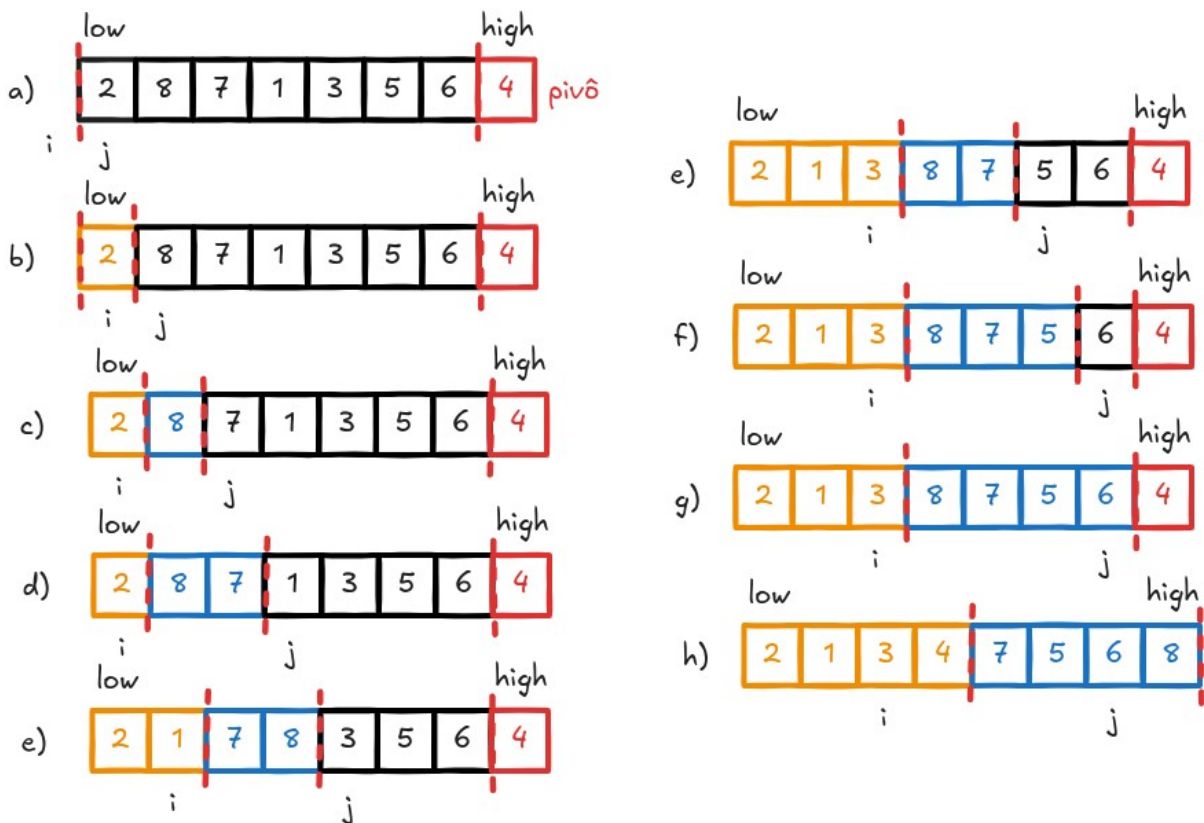
A seguir apresentamos o algoritmo Quicksort juntamente com uma função auxiliar para o particionamento da lista em duas sublistas menores.

```
QuickSort(L, low, high) {
    if low < high {
        q = partition(L, low, high)
        QuickSort(L, low, q-1)
        QuickSort(L, q+1, high)
    }
}
```

Para a eficiência do algoritmo Quicksort é fundamental termos como particionar a lista com custo computacional $O(n)$. A seguir veremos uma função eficiente para particionar a lista usando como pivô o último elemento de L .

```
partition(L, low, high) {
    x = L[high]
    i = low - 1
    for j = low to high - 1 {
        if L[j] <= x {
            i = i + 1
            swap(L[i], L[j])
        }
    }
    swap(L[i+1], L[high])
    return i+1
}
```

A figura a seguir ilustra o funcionamento do algoritmo para particionar uma lista em duas sublistas.



Análise da complexidade

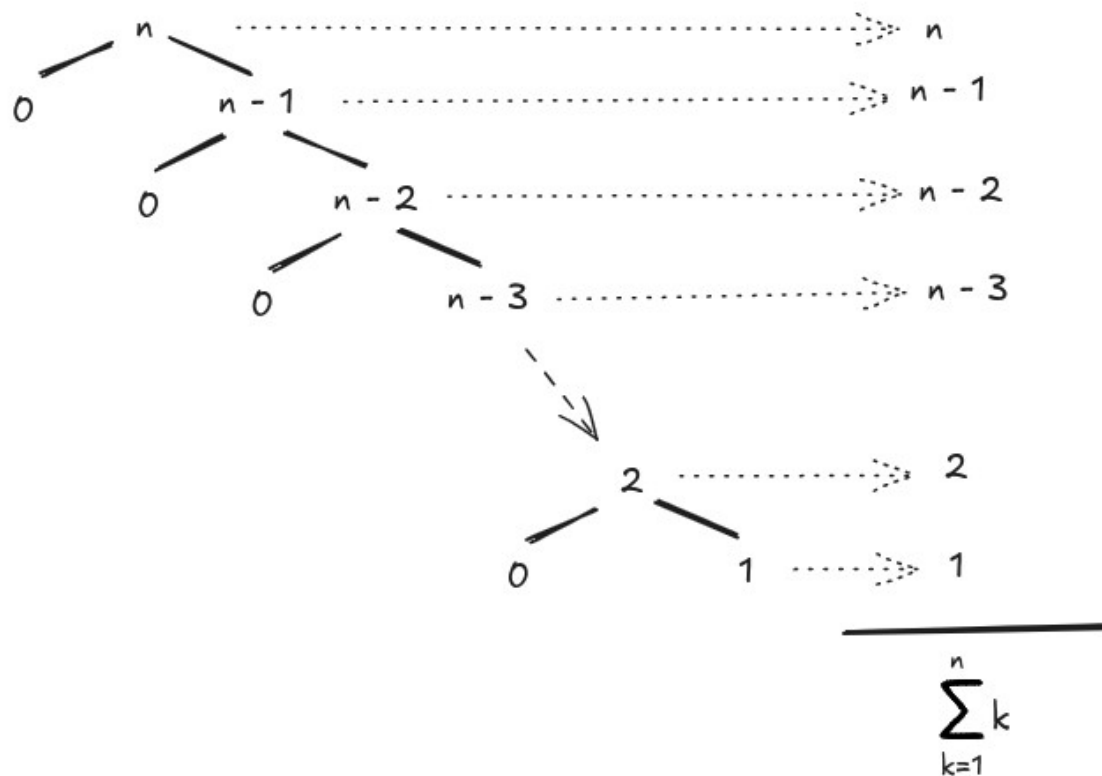
Iremos considerar 3 cenários distintos: pior caso, caso médio e melhor caso.

Pior caso: quando o pivô é sempre o maior ou menor elemento, o que gera partições totalmente desbalanceadas:

Na primeira chamada recursiva, temos uma lista de tamanho n , então para criar as novas listas L , teremos $n - 1$ elementos na primeira lista, o pivô e uma lista vazia. Isso nos permite escrever a seguinte relação de recorrência:

$$T(n) = T(n-1) + T(0) + n = T(n-1) + n$$

pois estamos decompondo um problema de tamanho n em um de tamanho zero e outro de tamanho $n - 1$, mas para realizar a divisão da lista em sublistas, utilizamos n operações (uma vez que a lista L tem n elementos)



Somando todas os custos em cada nível, temos o somatório:

$$T(n) = 1 + 2 + 3 + \dots + (n-1) + n = \sum_{k=1}^n k$$

cujas resposta é:

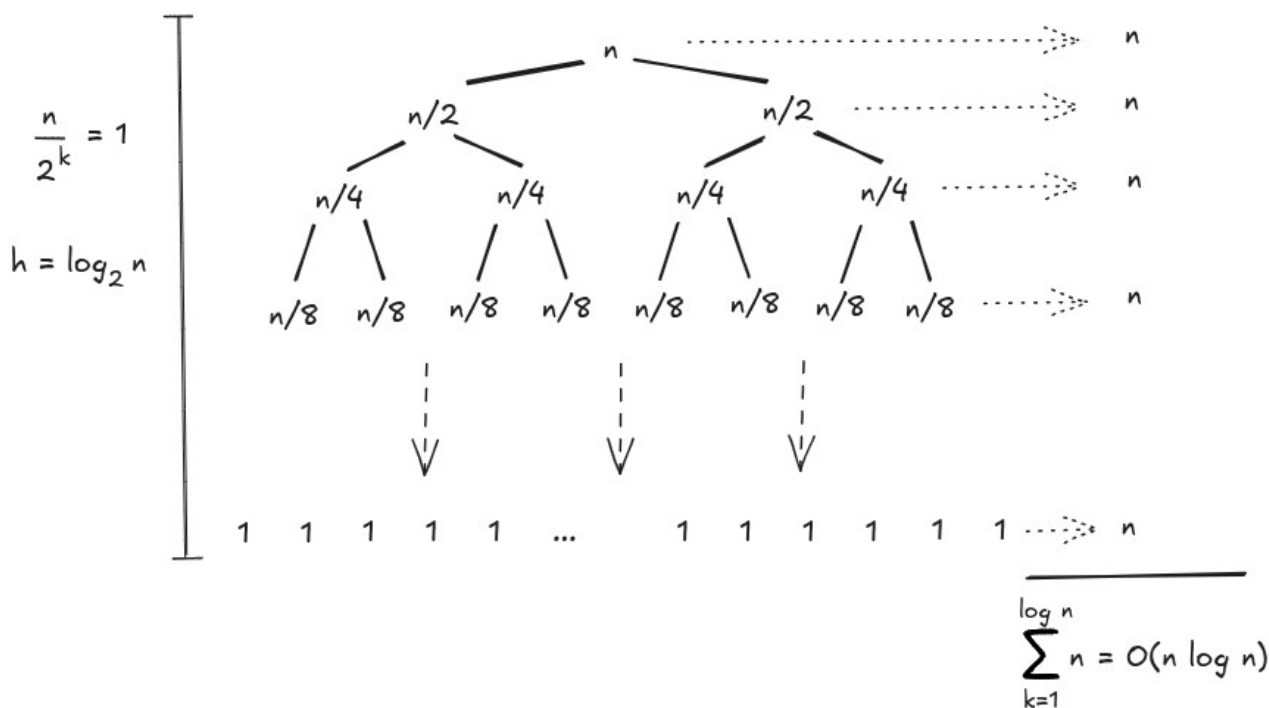
$$T(n) = \frac{n(n+1)}{2} = \frac{n^2 + n}{2}$$

o que resulta em uma complexidade $O(n^2)$.

Melhor caso: ocorre quando as duas partições tem exatamente o mesmo tamanho, ou seja, são $n/2$ elementos, o pivô e mais $n/2$ elementos. Podemos escrever a seguinte relação de recorrência:

$$T(n) = T\left(\frac{n}{2}\right) + T\left(\frac{n}{2}\right) + n = 2T\left(\frac{n}{2}\right) + n$$

pois estamos decompondo um problema de tamanho n em dois problemas de tamanho $n/2$, mas para realizar a divisão da lista L em sublistas, utilizamos n operações na função partition.



Como divide-se 1 problema de tamanho n em dois de tamanho $n/2$, a altura da árvore é $h = \log_2 n$ e como em cada nível da árvore de recursão o custo é n , temos que o custo total do algoritmo Quicksort no melhor caso é $O(n \log_2 n)$.

Caso médio: iremos utilizar uma estratégia de calcular a média para todas as possíveis partições.

Os tamanhos possíveis de partição são: $(1, n-1)$; $(2, n-2)$; $(3, n-3)$; ... $(n-1, 1)$. Assim, temos:

$$T(n) = \frac{1}{n} \left\{ \sum_{i=1}^n [T(i-1) + T(n-i)] \right\} + O(n)$$

Separando os somatórios, temos:

$$T(n) = \frac{1}{n} \sum_{i=1}^n T(i-1) + \frac{1}{n} \sum_{i=1}^n T(n-i) + cn$$

Note que os dois somatórios são de fato idênticos (os termos ocorrem na ordem inversa um do outro). Dessa forma, podemos escrever:

$$T(n) = \frac{2}{n} \sum_{i=1}^n T(i-1) + cn = \frac{2}{n} \sum_{i=0}^{n-1} T(i) + cn$$

Multiplicando ambos os lados por n , temos:

$$nT(n) = 2 \sum_{i=0}^{n-1} T(i) + cn^2 \quad (I)$$

Reescrevendo a identidade (I) fazendo $n = n-1$:

$$(n-1)T(n-1) = 2 \sum_{i=0}^{n-2} T(i) + c(n-1)^2 \quad (\text{II})$$

Calculando a diferença (I) – (II) e eliminando constantes:

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2cn$$

Isolando o termo T(n), chega-se em:

$$nT(n) = 2T(n-1) + nT(n-1) - T(n-1) + 2cn$$

Agrupando os termos:

$$nT(n) = (n+1)T(n-1) + 2cn$$

Dividindo ambos os lados por n(n+1):

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2c}{n+1}$$

Utilizando a recursão, podemos expandir T(n – 1):

$$\frac{T(n)}{n+1} = \frac{T(n-2)}{n-1} + \frac{2c}{n} + \frac{2c}{n+1}$$

Repetindo o processo para T(n – 2):

$$\frac{T(n)}{n+1} = \frac{T(n-3)}{n-2} + \frac{2c}{n-1} + \frac{2c}{n} + \frac{2c}{n+1}$$

Continuando o processo até T(1), temos a seguinte sequência:

$$\frac{T(n)}{n+1} = \frac{T(1)}{2} + \frac{2c}{3} + \frac{2c}{4} + \frac{2c}{5} + \dots + \frac{2c}{n+1}$$

Como T(1) = O(1), podemos escrever o somatório a seguir:

$$\frac{T(n)}{n+1} = O(1) + 2c \sum_{i=3}^{n+1} \frac{1}{i}$$

Para resolver o somatório em questão, utilizaremos uma aproximação do cálculo. Sabemos que:

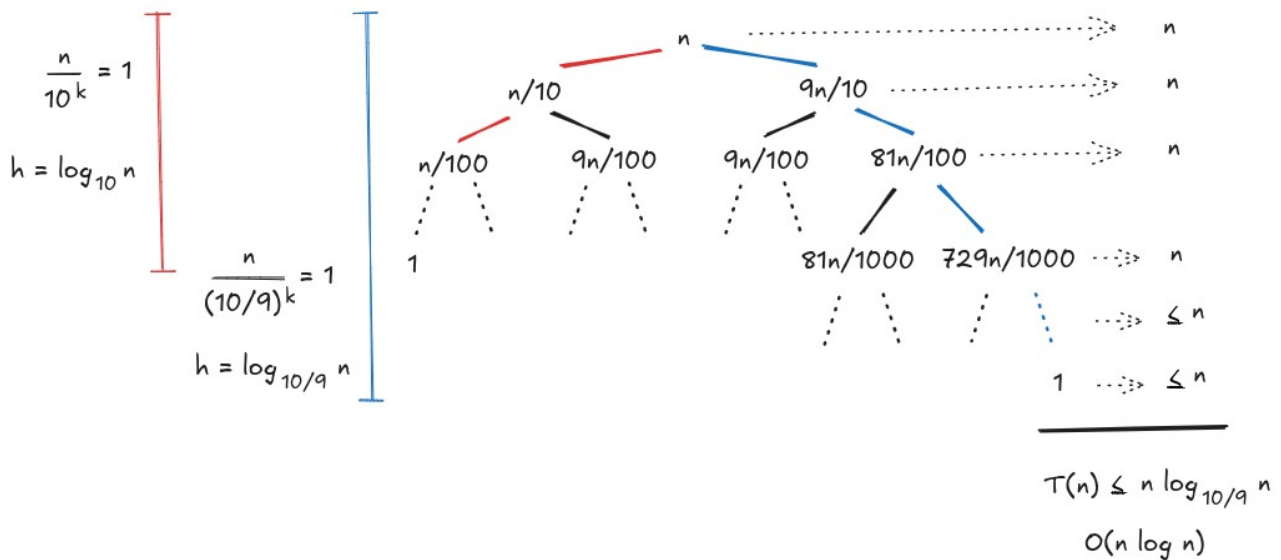
$$\log_e a = \ln a = \int_1^a \frac{1}{x} dx$$

Assim, podemos escrever:

$$T(n) = (n+1)(O(1) + 2cO(\log_e n)) = (n+1) + 2cnO(\log_e n) + 2cO(\log_e n)$$

o que resulta em complexidade $O(n \log_e n)$. Uma estratégia empírica que, em geral, melhora o desempenho do algoritmo Quicksort consiste em escolher como pivô a mediana entre o primeiro elemento, o elemento do meio e o último elemento (regra mediana de 3).

A grande vantagem do algoritmo Quicksort é que ele opera no modo caso médio, praticamente sempre. Isso porque mesmo com partições do tipo 90%-10%, pode-se verificar que a complexidade é $O(n \log n)$.



Neste caso, note que a altura do ramo mais a esquerda é a menor possível, sendo igual a $h_L = \log_{10} n$. Já altura do ramo mais a direita é a maior possível, sendo igual a $h_R = \log_{10/9} n$. Sendo assim, o custo dentro de um nível até h_L será igual a n , e o custo dentro de um nível após h_L será menor que n . Somando todos os custos, chegamos em $T(n) \leq cn \log_{10/9} n$. Sabendo que

$$\log_{10/9} n = \frac{\log_2 n}{\log_2 10/9} \approx \frac{\log_2 n}{0.152} \approx 6.578 \log_2 n, \text{ temos que } T(n) \text{ é } O(n \log_2 n).$$

Limitante Inferior para Ordenação Baseada em Comparações

Os algoritmos de ordenação vistos até o momento são todos baseados em comparações, ou seja, eles obtêm informação sobre a sequência através de uma função que recebe dois elementos e retorna qual deles é o maior, para saber se deve trocá-los ou não de posição.

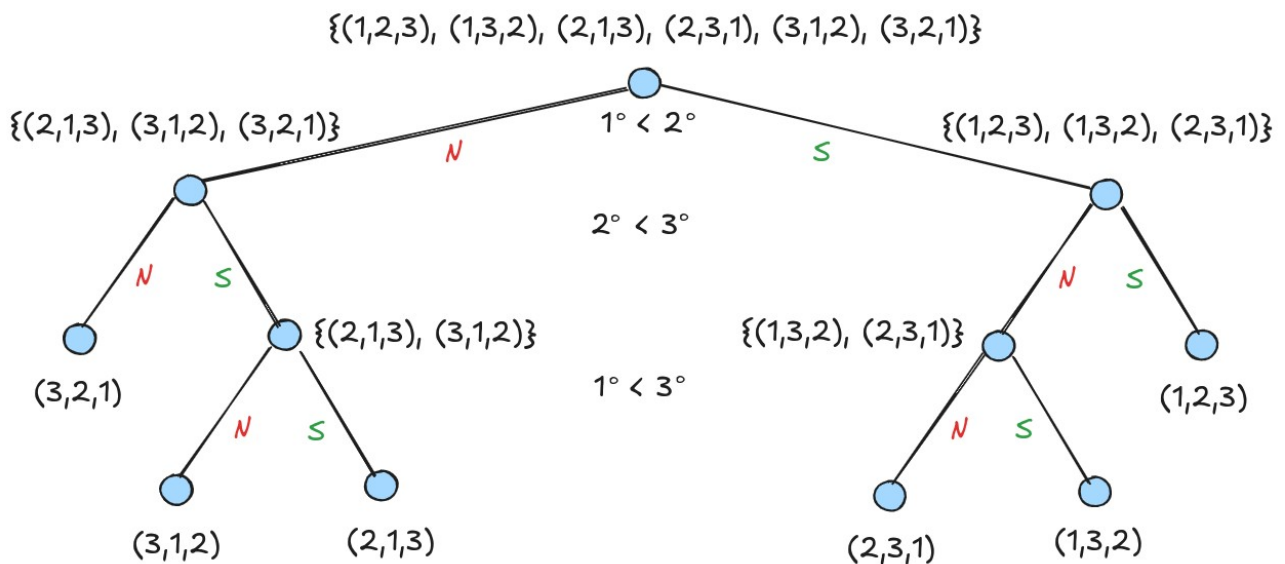
Para obter um limitante inferior para o número de trocas, vamos comparar duas grandezas:

- número de permutações que sequência de tamanho n pode apresentar;
- número de sequências que um algoritmo consegue distinguir após k comparações;

Com relação ao primeiro ponto, é fácil notar que esse número é $n!$

$$\overline{n} \quad \overline{n-1} \quad \overline{n-2} \quad \overline{n-3} \quad \dots \quad \overline{1}$$

Com relação ao segundo ponto, note que inicialmente todas as sequências são iguais para o algoritmo.



Note que no melhor das hipóteses, cada comparação divide o espaço de busca em duas metades, ou seja, dobra o número de sequências identificadas.

Sabemos que em uma árvore binária de altura k (número de decisões), o número máximo de nós armazenados (sequências diferentes) é igual a 2^k . Sendo assim, a pergunta é: quantas sequências distintas “cabem” na árvore binária? No máximo 2^k .

Pelo princípio da casa dos pombos, se temos $n+1$ pombos para serem colocados em n casas, uma casa deverá conter 2 ou mais pombos. Nesse cenário, temos:

* Pombos: $n!$ permutações.

* Casas: 2^k slots na árvore.

Assim, se $n! > 2^k$ não é possível distinguir todas as possíveis sequências. Então, para que seja possível é preciso ter:

$$2^k \geq n!$$

Primeiramente, note que pela definição de fatorial, temos:

$$n! = n \times (n-1) \times (n-2) \times \dots \times \left(\frac{n}{2}+1\right) \times \left(\frac{n}{2}\right) \times \left(\frac{n}{2}-1\right) \times \dots \times 3 \times 2 \times 1$$

Sendo assim, podemos escrever:

$$n! \geq \underbrace{\left[\frac{n}{2} \times \frac{n}{2} \times \frac{n}{2} \times \dots \times \frac{n}{2}\right]}_{n/2 \text{ termos}} \times \underbrace{[1 \times 1 \times 1 \dots \times 1]}_{n/2 \text{ termos}}$$

o que nos leva a:

$$n! \geq \left(\frac{n}{2}\right)^{\frac{n}{2}}$$

Assim, temos a seguinte desigualdade:

$$2^k \geq n! \geq \left(\frac{n}{2}\right)^{\frac{n}{2}}$$

Aplicando o logaritmo, finalmente chega-se a:

$$k \geq \frac{n}{2} \log_2 \frac{n}{2}$$

o que nos permite escrever que $k = \Omega(n \log n)$. Portanto, precisamos ter pelo menos $n \log n$ comparações (e trocas).

Ordenação Não Baseada em Comparações

Nem todos os algoritmos de ordenação são baseados em comparações. Existem diversos métodos que ordenam listas sem a necessidade de comparar e trocar elementos. Alguns exemplos são:

- Countingsort, Bucketsort, Radixsort e Pigeonholesort.
Veremos a seguir os algoritmos Countingsort e Bucketsort.

Countingsort

O algoritmo Countingsort ordena os elementos de uma sequência pela contagem do número de ocorrências de cada elemento. Para mostrar a intuição por trás do algoritmo, iremos descrever seus passos ilustrando seu funcionamento em um exemplo ilustrativo. O algoritmo Countingsort é composto pelos seguintes passos:

Passo 1. Encontrar o maior elemento da lista.

$L = [4, 2, 2, 8, 3, 3, 1]$ $\max = 8$

Passo 2. Criar um vetor de tamanho $(\max + 1)$ composto por zeros.

$H = [0, 0, 0, 0, 0, 0, 0, 0]$

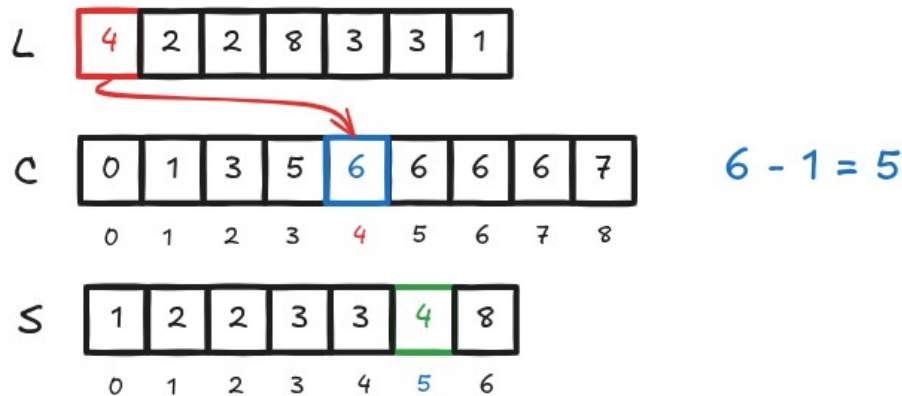
Passo 3. Armazene o número de ocorrências de cada elemento em seu respectivo índice em H. Note que poderão sobrar vários elementos nulos em H.

$H = [0, 1, 2, 2, 1, 0, 0, 1]$

Passo 4. Calcule a soma cumulativa dos elementos de H.

$C = [0, 1, 3, 5, 6, 6, 6, 7]$

Passo 5. Encontre o índice de cada elemento da lista L no vetor C, e coloque o respectivo elemento no índice $C[L[i]] - 1$ de uma lista S com o mesmo número de elementos de L.



Passo 6. Após colocar cada elemento na sua posição correta, diminua seu valor associado em C de uma unidade (ou seja, o 6 em C referente ao 4 inserido na posição 5, vira 5)

A seguir apresentamos o algoritmo Countingsort em pseudo-código para que seja possível analisarmos sua complexidade.

```
Countingsort(L, n) {
    m = max(L)                                # maior elemento de L
    C = zeros(m+1)                             # vetor de 0 a m
    S = zeros(n)                               # vetor com mesmo tamanho de L
    for i = 0 to n - 1
        C[L[i]] = C[L[i]] + 1                 # conta número de ocorrências

    for i = 1 to m
        C[i] = C[i] + C[i-1]                  # soma acumulada
    for i = 0 to n - 1 {
        C[L[i]] = C[L[i]] - 1
        S[C[L[i]]] = L[i]
    }
    return S
}
```

Análise da complexidade

Primeiramente, note que encontrar o maior elemento de L possui complexidade $O(n)$. Analisando cada uma das 3 estruturas de repetição (FOR), podemos escrever a função $T(n)$ como:

$$T(n) = O(n) + \sum_{i=0}^{n-1} 1 + \sum_{i=1}^m 1 + 2 \sum_{i=0}^{n-1} 1 = O(n) + O(n) + O(m) + O(n) = O(n+m)$$

Em geral, o algoritmo funciona muito bem quando o maior elemento da lista não é tão grande (pois isso reduz m). Por exemplo, quando $m < n$, como n domina m, a complexidade se torna $O(n)$. Porém, esse algoritmo tem limitações. A principal delas é quando o maior

elemento é muito grande, o que torna a complexidade $O(m)$: isso faz com que o vetor count seja muito maior que L ($m \gg n$). Mesmo que n seja pequeno, ou seja, uma lista com apenas 100 elementos, o algoritmo pode levar um tempo elevado!

Radixsort

Pode ser considerado uma generalização do algoritmo Countingsort.

É um método baseado no agrupamento de elementos por dígitos, iniciando pelo dígito menos significativo.

Primeiro, ordenamos pela unidade. Depois, ordenamos pela dezena, centena, milhar, etc... até o número de dígitos do máximo elemento na lista. Por exemplo, considere a seguinte lista de inteiros:

[121, 432, 564, 23, 1, 45, 788]

Como, o máximo elemento é 788, temos que o número máximo de dígitos é $k = 3$.

1. Unidade: [121, 001, 432, 023, 564, 045, 788]
2. Dezena: [001, 121, 023, 432, 045, 564, 788]
3. Centena: [001, 023, 045, 121, 432, 564, 788]

Uma boa opção consiste em adotar uma versão modificada do Countingsort para o Radixsort, uma vez que estaremos lidando apenas com dígitos de 0 a 9 (m é pequeno e o algoritmo CountingSort é muito eficiente!). A seguir apresentamos o pseudocódigo do algoritmo Countingsort_R, usado no Radixsort.

```
Countingsort_R(L, n, d) {
    S = zeros(n)           # qual dígito é selecionado: uni, dez,...
    m = (L[0]/d) % 10      # vetor com mesmo tamanho de L
                           # dígito d do primeiro elemento
    for i = 1 to n - 1 {
        if (L[i]/d) % 10 > m
            m = L[i]       # m é o maior dos dígitos d
    }
    C = zeros(m+1)
    for i = 0 to n - 1
        C[(L[i]/d)%10] += 1 # número de ocorrências
    for i = 1 to m
        C[i] += C[i-1]     # soma acumulada
    for i = 0 to n - 1 {
        C[(L[i]/d)%10] -= 1
        S[C[(L[i]/d)%10]] = L[i]
    }
    for i = 0 to n - 1
        L[i] = S[i]
}
```

A seguir apresentamos o algoritmo Radixsort. A ideia consiste em executar d vezes o Countingsort_R, onde d é o número de dígitos do maior elemento da lista L .

```
Radixsort(L, n) {
    m = max(L)
```

```

d = 1                                # d começa com 1 (unidade)
while m/d > 0 {
    Countingsort_R(L, n, d)
    d = d*10                          # a cada iteração multiplica por 10
}

```

Para analisar a complexidade do algoritmo Radixsort, primeiramente, devemos analisar a complexidade da versão modificada do Countingsort, denominada Countingsort_R. É possível notar que temos 5 estruturas de repetição (FOR) sequenciais. Sendo assim, a função $T(n)$ referente ao Countingsort_R é dada por:

$$T(n) = \sum_{i=1}^{n-1} 1 + \sum_{i=0}^{n-1} 1 + \sum_{i=1}^m 1 + 2 \sum_{i=0}^{n-1} 1 + \sum_{i=0}^{n-1} 1 = O(n) + O(n) + O(m) + O(n) + O(n) = O(n+m)$$

Porém, neste caso, o valor de m é no máximo 9, o que faz com que n seja dominante, levando a complexidade $O(n)$.

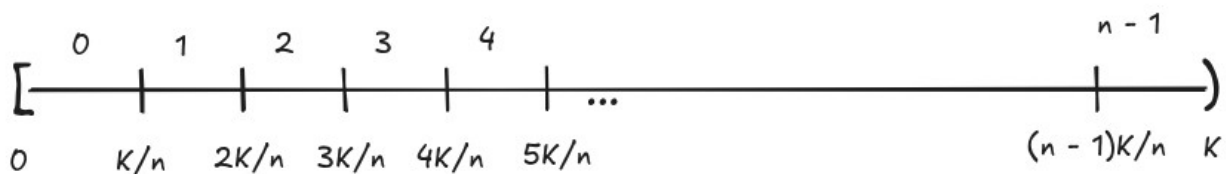
Prosseguindo para a função Radixsort, podemos escrever a função $T(n)$ como:

$$T(n) = O(n) + \sum_{i=1}^k [O(n) + 1] = O(n) + kO(n) + k$$

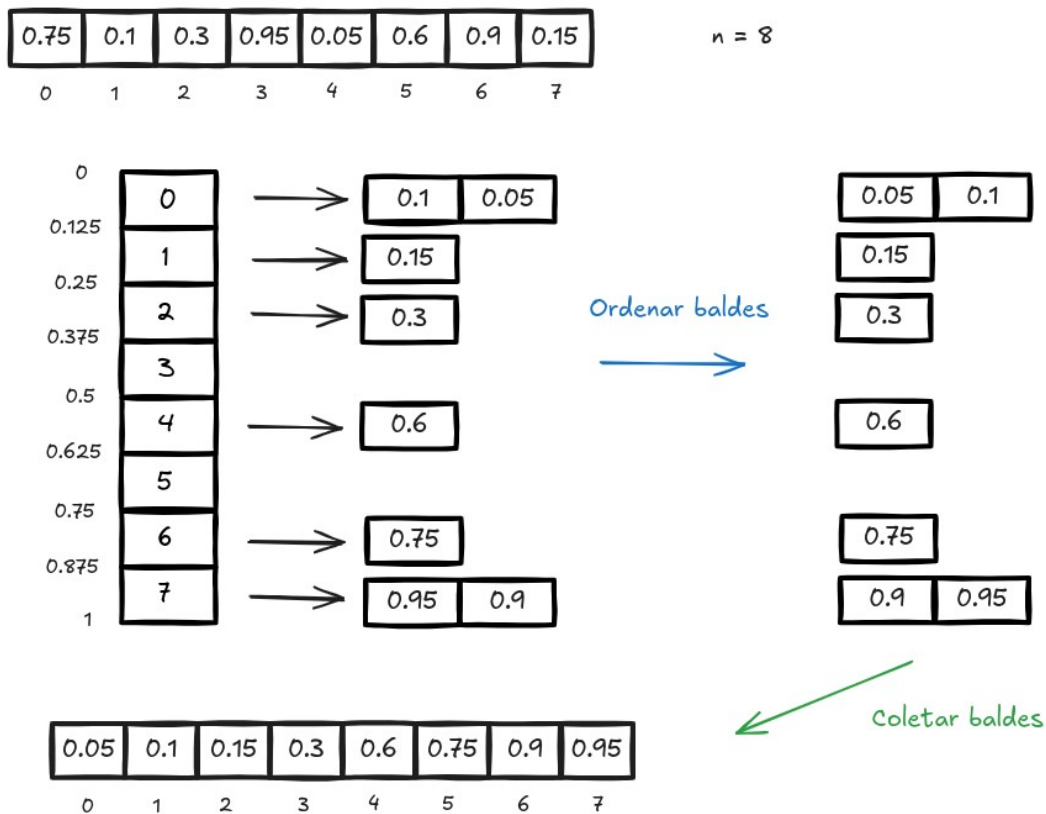
onde k denota o o número de dígitos do maior inteiro em L . Como em geral k é uma constante muito menor que n , temos que a complexidade resultante é $O(n)$.

Bucketsort

É um algoritmo de ordenação eficiente para ordenar um conjunto de n chaves distribuídas uniformemente em um intervalo (hipótese). Não é baseado em comparações, por isso não precisa realizar trocas de posições. Seja $I = [0, K)$ um intervalo arbitrário. O primeiro passo do algoritmo consiste em dividir o intervalo de tamanho K em n baldes (buckets), cada um com tamanho K/n . A ideia é simples: devemos colocar cada número da lista L a ser ordenada em seu respectivo balde. Sob a hipótese de uniformidade, o número de elementos por balde será pequeno e constante, fazendo com que o custo computacional da ordenação de cada balde seja $O(1)$.



Por fim, percorremos os baldes na ordem em que aparecem copiando os elementos ordenados de cada balde para a lista final. A figura a seguir ilustra um simples exemplo.



A seguir apresentamos o algoritmo Bucketsort escrito como um pseudocódigo para que seja possível analisar sua complexidade.

```

Bucketsort(L, n) {
    for i = 0 to n - 1
        B[i] = NIL
    for i = 0 to n - 1 {
        if L[i] == 1
            index = n - 1
        else {
            index = floor(n*L[i])
            insert(B[index], L[i])
        }
    }
    for i = 0 to n - 1
        sort(B[i])
    S = concatenate(B[0], B[1], ..., B[n-1])
    return S
}
  
```

Cria n baldes vazios

Uma observação relevante é que, se os dados estão uniformemente distribuídos no intervalo [a, b), se calcularmos:

$$y_i = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

então os dados terão distribuição uniforme no intervalo [0, 1). De modo similar, ao fazer

$$x_i = x_{\min} + (x_{\max} - x_{\min}) y_i$$

devolvemos os dados para o intervalo original $[a, b)$.

Análise da complexidade

Note que utilizamos 3 primitivas básicas no algoritmo Bucketsort.

i) $\text{insert}(p, x)$: insere x na lista apontada pela referência p . Possui complexidade $O(1)$.

ii) $\text{sort}(p)$: ordena a lista apontada pela referência p . Se utilizarmos um algoritmo baseado em comparações, sabe-se que a complexidade seria entre $O(n \log n)$ e $O(n^2)$.

iii) $\text{concatenate}(B, n)$: retorna a lista obtida pela concatenação das listas $B[0], B[1], \dots, B[n-1]$. Possui complexidade $O(kn)$, onde k é uma constante que representa o tamanho médio dos baldes, o que resulta em $O(n)$.

O ponto crítico é a análise da primitiva $\text{sort}()$, sendo que devemos realizar uma análise probabilística. Seja X_i o número de elementos na lista $B[i]$. Seja ainda a variável binária:

$$X_{ij} = \begin{cases} 1, & \text{se o elemento } j \text{ de } L \text{ foi para a lista } B[i] \\ 0, & \text{se o elemento } j \text{ de } L \text{ não foi para a lista } B[i] \end{cases}$$

Note que $X_i = \sum_j X_{ij}$.

Iremos denotar por Y_i o número de comparações necessárias para ordenar a lista $B[i]$. Observe que $Y_i \leq X_i^2$, pois no pior caso a ordenação será $O(n^2)$. Logo como desejamos analisar o caso médio, podemos escrever:

$$E[Y_i] \leq E[X_i^2] = E\left[\left(\sum_j X_{ij}\right)^2\right]$$

Mas podemos desenvolver o valor esperado como:

$$E\left[\left(\sum_j X_{ij}\right)^2\right] = E\left[\left(\sum_j X_{ij}\right)\left(\sum_k X_{ik}\right)\right] = E\left[\sum_j \sum_k X_{ij} X_{ik}\right] = E\left[\sum_j X_{ij}^2 + \sum_j \sum_{k \neq j} X_{ij} X_{ik}\right]$$

onde a última igualdade é válida pois o somatório duplo inclui os termos a seguir:

(1, 1) (1, 2) (1, 3), ..., (1, n)
(2, 1) (2, 2) (2, 3), ..., (2, n)
(3, 1) (3, 2) (3, 3), ..., (3, n)
...
(n, 1) (n, 2) (n, 3), ..., (n, n)

Note que para $k = j$ temos os elementos da diagonal e para os termos $k \neq j$ temos os elementos fora da diagonal. Dessa forma, podemos escrever:

$$E[Y_i] \leq \sum_j E[X_{ij}^2] + \sum_j \sum_{k \neq j} E[X_{ij} X_{ik}]$$

Como X_{ij} é uma variável aleatória binária:

$$P(X_{ij}=0) + P(X_{ij}=1) = 1$$

Pela definição de valor esperado, temos:

$$E[X_{ij}^2] = \sum_{x \in \{0,1\}} X_{ij}^2 P(X_{ij}=x) = 0^2 P(X_{ij}=0) + 1^2 P(X_{ij}=1) = P(X_{ij}=1)$$

Assumindo que a probabilidade de um elemento cair em um dos n baldes é uniforme, isto é, todos os baldes possuem a mesma probabilidade, chega-se a:

$$E[X_{ij}^2] = P(X_{ij}=1) = \frac{1}{n}$$

Para calcular o valor esperado do produto, note que para $j \neq k$ as duas variáveis aleatórias são independentes, ou seja:

$$E[X_{ij} X_{ik}] = E[X_{ij}] E[X_{ik}] = \frac{1}{n} \frac{1}{n} = \frac{1}{n^2}$$

Isso nos leva a:

$$E[Y_i] \leq \sum_{j=1}^n \frac{1}{n} + \sum_{j=1}^n \sum_{k \neq j} \frac{1}{n^2} = n \frac{1}{n} + \sum_{j=1}^n (n-1) \frac{1}{n^2} = 1 + \frac{n(n-1)}{n^2} = 1 + \frac{n^2 - n}{n^2} = 2 - \frac{1}{n}$$

Esse é o número médio de comparações para ordenar o balde $B[i]$. Como temos n baldes, a complexidade total do Bucketsort é:

$$E[Y] = E\left[\sum_{i=1}^n Y_i\right] = E\left[n\left(2 - \frac{1}{n}\right)\right] = E[2n - 1] = 2n - 1$$

o que indica complexidade $O(n)$. Por essa razão, a complexidade total do Buckesort é:

$$n * O(1) + O(n) + O(n) + O(n)$$

o que finalmente resulta em $O(n)$. Vimos que o Bucketsort é um algoritmo de ordenação linear no caso médio. Mas e no pior caso? Esse cenário ocorre quando todos os elementos caem no mesmo balde. Sendo assim, se optarmos pela aplicação de um algoritmo baseado em trocas, a complexidade poderia variar de $O(n \log n)$ a $O(n^2)$. Em resumo, a tabela a seguir faz uma comparação das complexidades dos algoritmos de ordenação apresentados.

Algoritmo	Melhor	Médio	Pior
BubbleSort	$O(n)$	$O(n^2)$	$O(n^2)$
InsertionSort	$O(n)$	$O(n^2)$	$O(n^2)$
SelectionSort	$O(n^2)$	$O(n^2)$	$O(n^2)$

MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Countingsort	$O(n+m)$	$O(n+m)$	$O(n+m)$
Radixsort	$O(nd)$	$O(nd)$	$O(nd)$
Bucketsort	$O(n)$	$O(n)$	$O(n \log n)$

Para os interessados em aprender mais sobre o assunto, a internet contém diversos materiais sobre algoritmos de ordenação. Outro detalhe interessante está relacionado com a estabilidade. Um algoritmo de ordenação é considerado estável ele mantém a ordem relativa dos registros em caso de igualdade de chaves. Importante quando ordenamos strings pelo primeiro caracter (i.e., Processamento de Linguagem Natural).

Algoritmo	Estável
Bubblesort	Sim
Selectionsort	Não
Insertionsort	Sim
Quicksort	Não
Mergesort	Sim
Heapsort	Não
Countingsort	Sim
Bucketsort	Sim

A seguir indicamos alguns links interessantes:

15 sorting algorithms in 6 minutes (Animações sonorizadas muito boas para visualização)

<https://www.youtube.com/watch?v=kPRA0W1kECg>

Animações passo a passo dos algoritmos

<https://visualgo.net/en/sorting>

Algoritmos de ordenação como danças em grupo

<https://www.youtube.com/user/AlgoRythmics>

"If you feel like you're losing everything, remember that trees lose their leaves every year and they still stand tall and wait for better days to come."

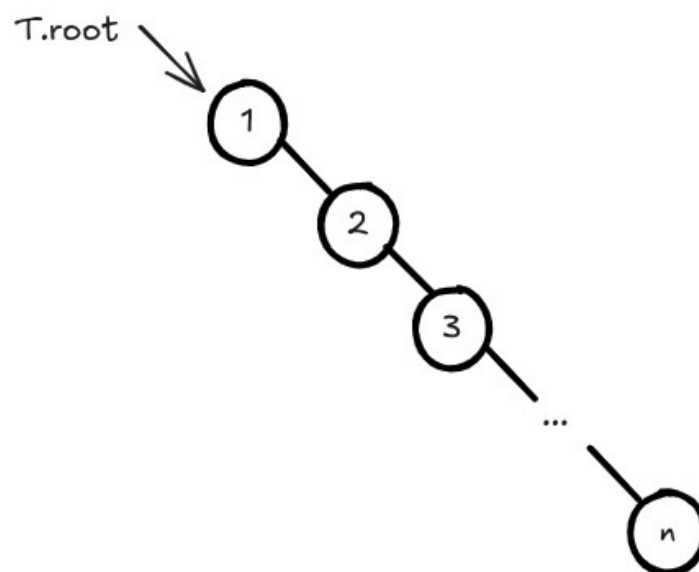
-- Author Unknown

Árvores AVL (Balanceadas)

Vimos anteriormente que as operações de busca, inserção e remoção de nós em árvores binárias de busca possuem complexidade $O(h)$, em que h é a altura da árvore T .

Um dos problemas com as árvores binárias de busca é que seu padrão de crescimento pode se tornar bastante irregular, no sentido de que, após a criação da raiz da árvore, todos os elementos inseridos à árvore são maiores que o anterior. Isso faz com que a árvore se degenere para uma lista encadeada.

Claramente, a situação descrita é um caso extremo, mas não é incomum que alguns ramos da árvore cresçam muito mais que outros, tornando a árvore completamente desbalanceada, o que faz com que as operações de inserção, pesquisa e remoção de nós sejam menos eficientes.



Para contornar esse problema, foram propostas as árvores AVL, em homenagem aos seus criadores, Georgy Adelson-Velsky e Landis. Essas árvores também são conhecidas como árvores binárias de busca balanceadas. Para cada nó x definimos o fator de balanço como:

$$x.b = h_L - h_R$$

onde h_L é a altura da subárvore à esquerda e h_R é a altura da subárvore à direita.

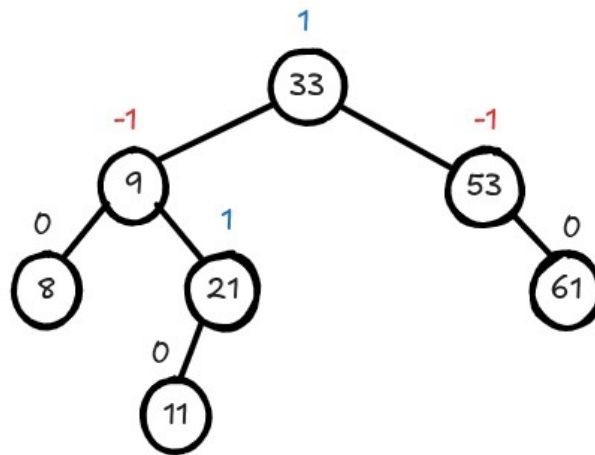
Def: Uma árvore T é dita AVL ou balanceada se:

$$\forall x \in T (x.b \in \{-1, 0, 1\})$$

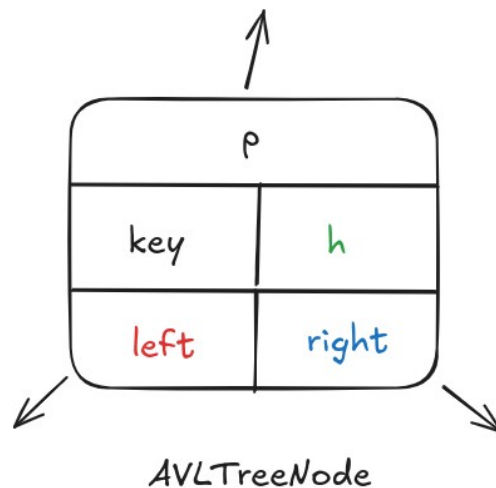
ou seja, não é permitido que nenhum nó tenha fator de balanço maior que 1 ou menor que -1.

A figura a seguir ilustra um exemplo de árvore AVL.

Para garantir que após a inserção ou remoção de um nó, a árvore em questão seja balanceada, devemos executar algumas operações de modo a garantir que o fator de balanço dos nós esteja entre -1 e +1. A seguir discutimos um pouco dessas operações.



Para otimizar os cálculos das alturas dos nós da árvore AVL é comum criarmos um campo `h` para armazenar essa informação e evitar o recálculo. Assim, definimos o nó de uma árvore AVL como:



```

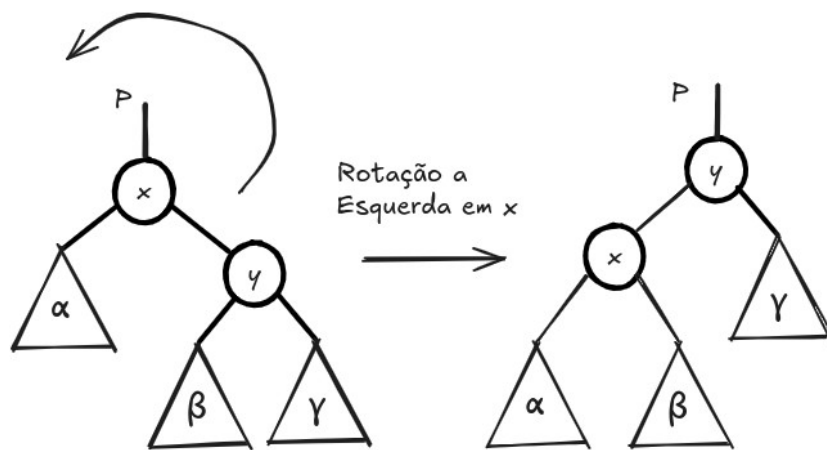
# Função auxiliar que retorna a altura do nó z
# Altura é o número de nós no maior caminho da raiz a um nó folha
height(z) {
    if z == NIL
        return 0
    else
        return z.h
}
  
```

Operações em árvores AVL

Se uma árvore AVL torna-se desbalanceada após a remoção ou inserção de um nó, devemos executar uma operação chamada rotação. Há dois tipos básicos de rotação: rotação a esquerda e rotação a direita. É importante mencionar que tais operações preservam a propriedade chave das árvores binárias de busca.

Rotação a esquerda

Seja x um nó desbalanceado com $x.b < -1$, conforme indica a figura a seguir (a altura da subárvore a direita é maior que a altura da subárvore a esquerda). Após uma rotação a esquerda em x , a altura do nó x na árvore é decrementada.



A função leftRotate a seguir mostra os passos lógicos dessa operação.

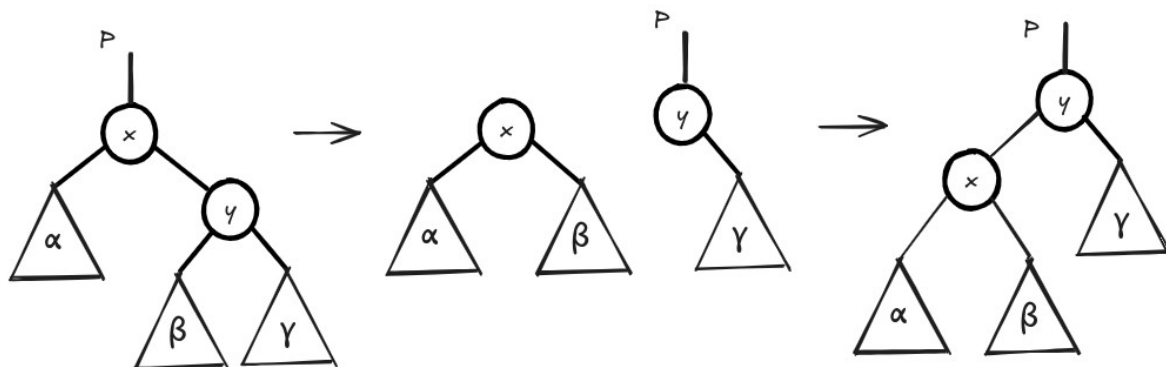
```

leftRotate(x) {
    y = x.right
    beta = y.left
    x.right = beta
    beta.p = x
    y.left = x
    y.p = x.p
    if x == x.p.left
        x.p.left = y
    else
        x.p.right = y
    x.p = y
    x.h = 1 + max(height(x.left), height(x.right))
    y.h = 1 + max(height(y.left), height(y.right))
    return y
}

```

Note que em resumo temos os seguintes passos:

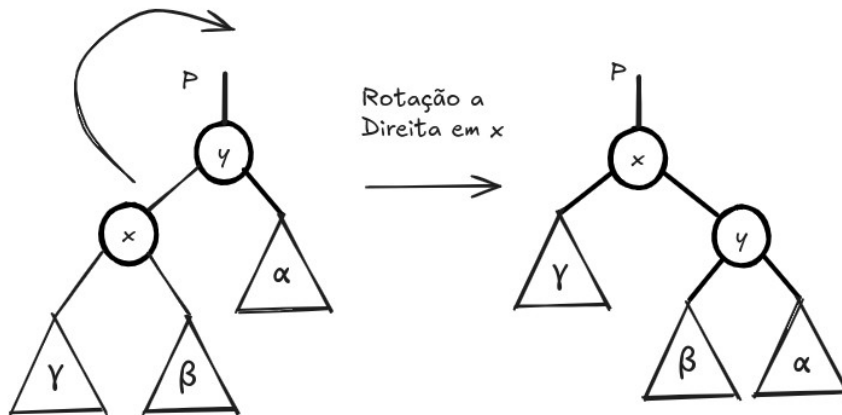
1. Se y possui uma subárvore a esquerda β , então o filho a direita de x deve apontar para β .
2. Se x é a raiz da árvore, então faça y ser a nova raiz da árvore. Senão, se x é o filho a esquerda do nó p, faça y ser o filho a esquerda de p. Senão, y deve ser o filho a direita de p.
3. Faça y ser o pai de x.



Note que as chaves em α permanecem menores que x, as chaves de β permanecem menores que y e maiores que x e as chaves de γ permanecem maiores que y (a propriedade chave é mantida!)

Rotação a direita

Seja y um nó desbalanceado com $y.b > 1$, conforme indica a figura a seguir (a altura da subárvore a esquerda é maior que a altura da subárvore a direita). Após uma rotação a direita em y , a altura do nó y na árvore é decrementada.

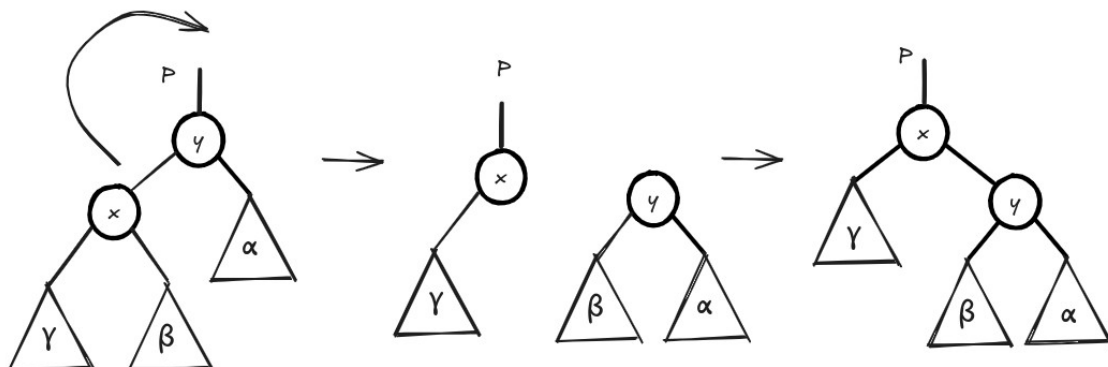


A função `rightRotate` a seguir mostra os passos lógicos dessa operação.

```
rightRotate(y) {  
    x = y.left  
    beta = x.right  
    y.left = beta  
    beta.p = y  
    x.right = y  
    x.p = y.p  
    if y == y.p.left  
        y.p.left = x  
    else  
        y.p.right = x  
    y.p = x  
    y.h = 1 + max(height(y.left), height(y.right))  
    x.h = 1 + max(height(x.left), height(x.right))  
    return x  
}
```

Note que em resumo temos os seguintes passos:

1. Se x possui uma subárvore a direita β , então o filho a esquerda de y deve apontar para β .
2. Se y é a raiz da árvore, então faça x ser a nova raiz da árvore. Senão, se y é o filho a direita do nó p , faça x ser o filho a direita de p . Senão, x deve ser o filho a esquerda de p .
3. Faça x ser pai de y .

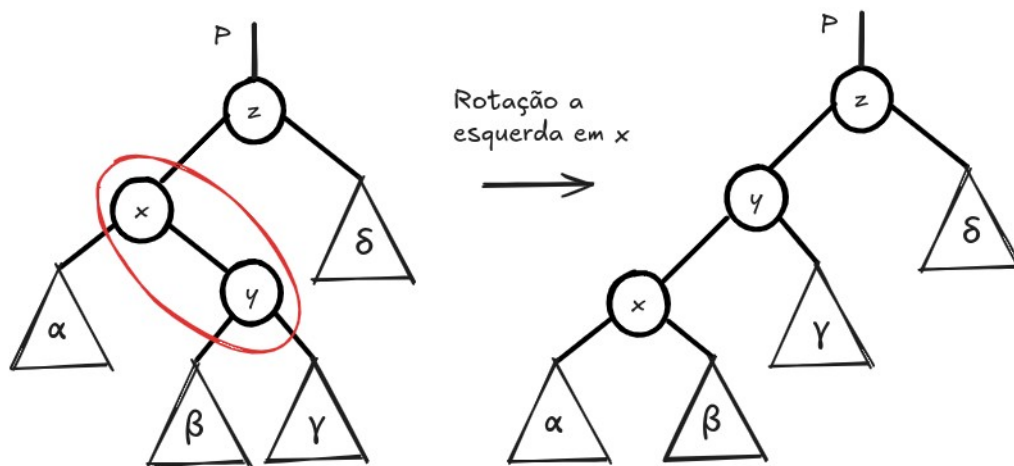


Note que após a rotação as chaves em α permanecem maiores que y , as chaves de β permanecem menores que y e maiores que x e as chaves de γ permanecem menores que y (a propriedade chave é mantida!)

Em alguns casos mais complexos, pode ser necessário realizar duas rotações básicas em sequência após a inserção ou remoção de um nó na árvore AVL: são as operações de rotação esquerda-direita e rotação direita-esquerda, que são definidas a seguir.

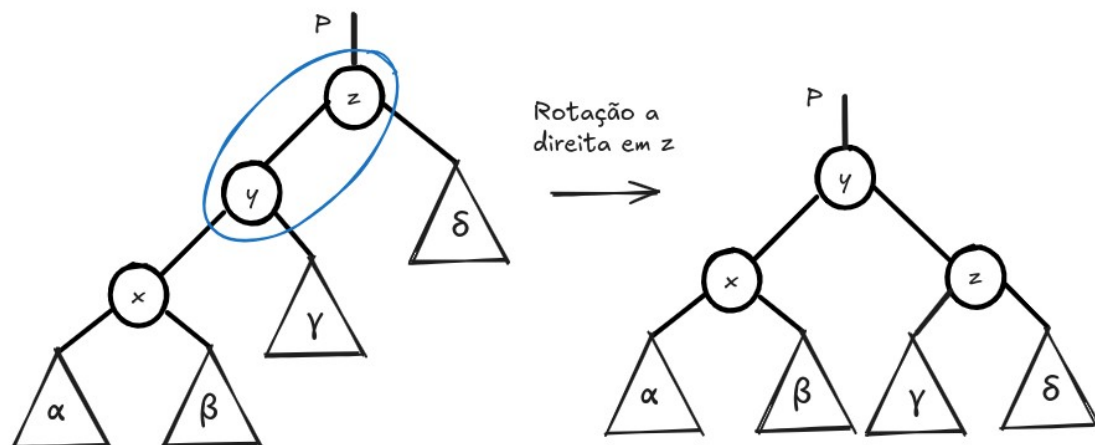
Rotação esquerda-direita

1. Como o nome sugere, primeiro devemos realizar uma rotação a esquerda em x .



Puxando o y para cima de x

2. Em seguida, devemos realizar uma rotação a direita em z .



Puxando y para cima de z

A função a seguir ilustra os passos para uma rotação esquerda-direita.

```

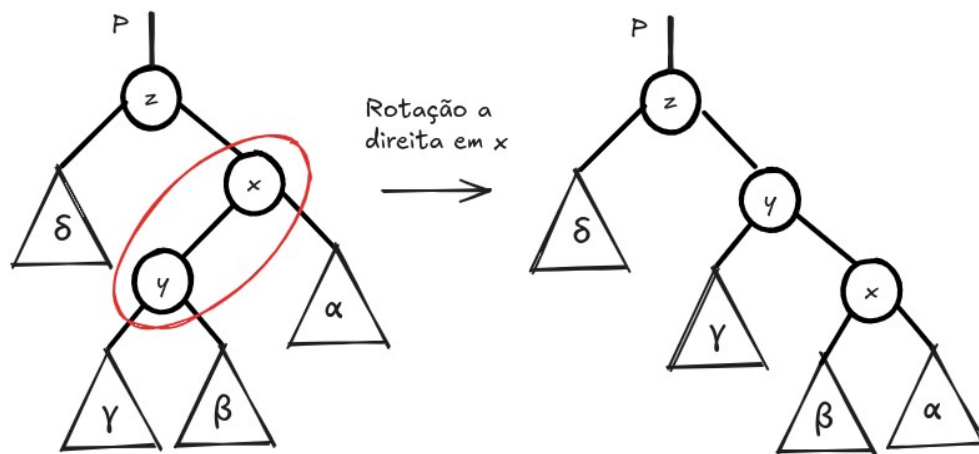
leftRightRotate(z) {
  x = z.left
  u = leftRotate(x)
  w = rightRotate(z)
  return w
}

```

De forma análoga, podemos realizar uma rotação direita-esquerda.

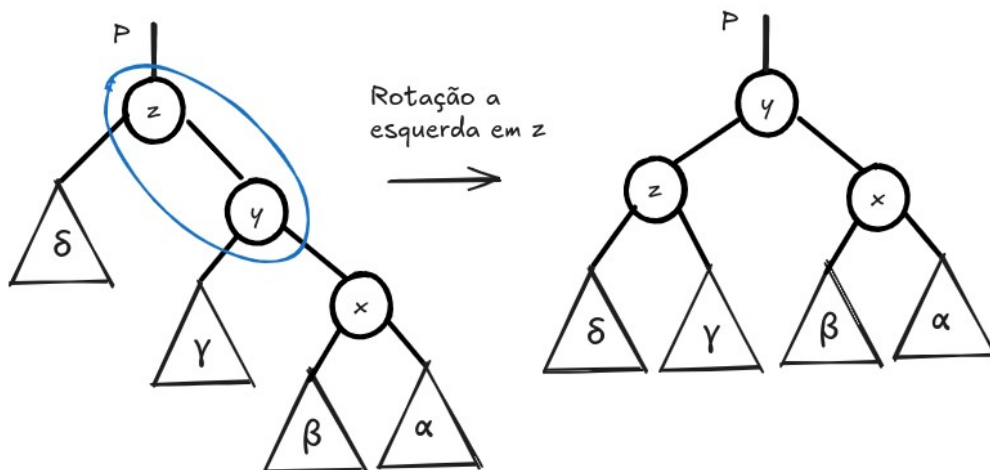
Rotação direita-esquerda

1. Novamente, como o nome sugere, primeiro devemos realizar uma rotação a direita em x.



Puxando o y para cima de x

2. Em seguida, devemos realizar uma rotação a esquerda em z.



Puxando y para cima de z

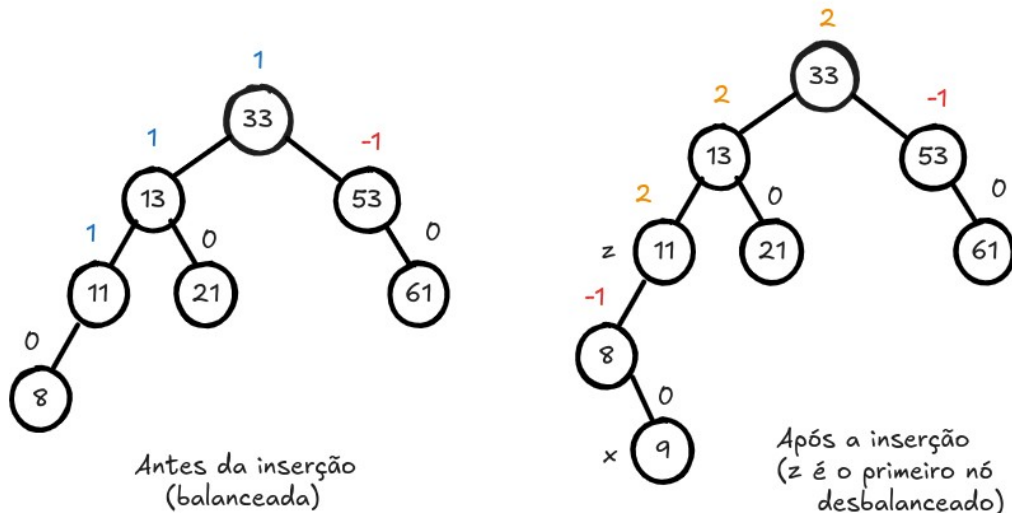
```

rightLeftRotate(z) {
  x = z.right
  u = rightRotate(x)
  w = leftRotate(z)
  return w
}

```

Inserção de nós em árvores AVL

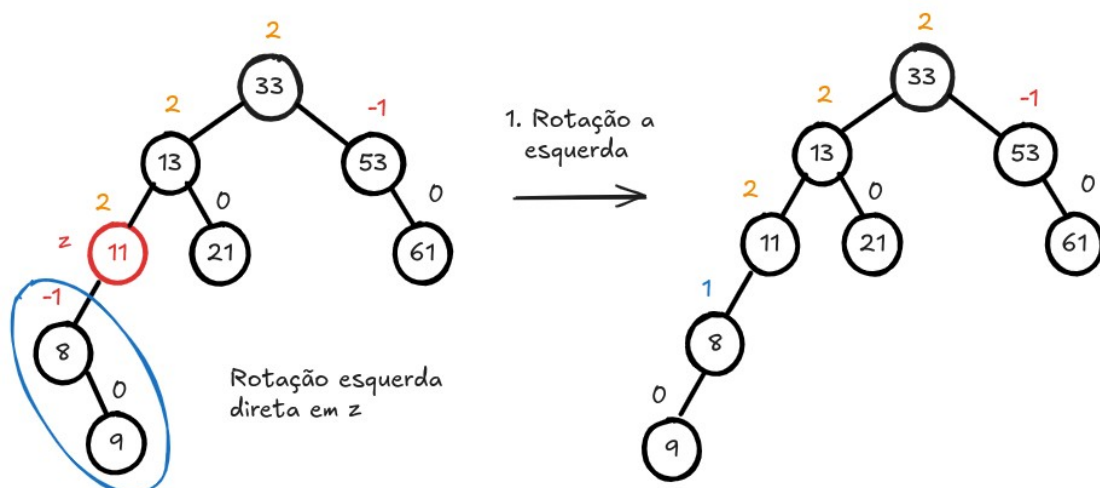
Um novo nó inserido na árvore AVL será sempre uma folha, que possui fator de balanço nulo. Porém, ele pode tornar algum outro nó desbalanceado, requerendo assim uma operação de rotação. Considere por exemplo, a inserção da chave 9 na árvore AVL a seguir.

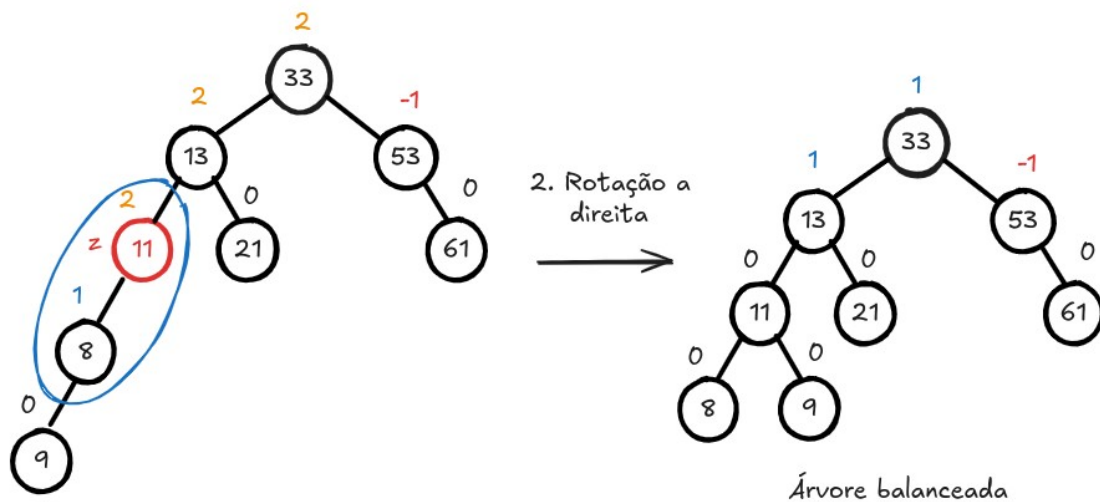


A inserção de um único nó desbalanceou vários outros nós. Como proceder? A operação de rotação deve ser aplicada no nó desbalanceado mais próximo. Partindo do princípio que todo nó inserido é uma folha da árvore (denotaremos de x), devemos percorrer um caminho bottom-up até encontrar o nó desbalanceado mais próximo ou chegar em NIL. Para o caso de fator de balanço positivo, deve-se aplicar a seguinte regra (supondo que z é o primeiro nó desbalanceado no caminho bottom-up):

```
if z.b > 1 {  
    if x.key < z.left.key  
        rightRotate(z)  
    else  
        leftRightRotate(z)  
}
```

Após o balanceamento, a árvore AVL resultante encontra-se perfeitamente balanceada.

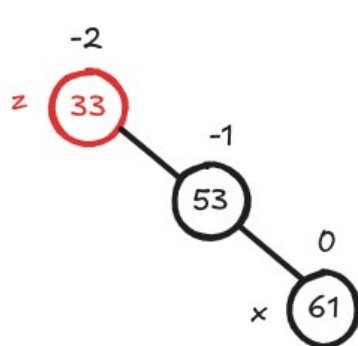




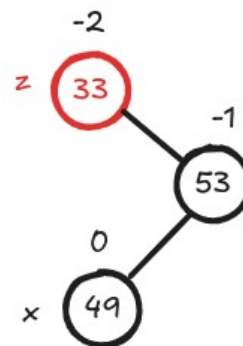
Analogamente, temos o caso simétrico, quando o fator de balanço é negativo.

```
if z.b < -1 {
    if x.key > z.right.key
        leftRotate(z)
    else
        rightLeftRotate(z)
}
```

A figura a seguir ilustra um exemplo simples.



Rotação a esquerda



Rotação direta-esquerda

Sendo assim, a operação completa de rebalanceamento de uma árvore AVL pode ser realizada de acordo com o algoritmo a seguir. A função auxiliar balance retorna o fator de balanço de um nó.

```
# Função para retornar o fator de balanço de um nó
balance(z) {
    if z == NIL
        return 0
    else
        return height(z.left) - height(z.right)
}
# Função recursiva para inserir um nó em uma árvore AVL
```

```

# Deve-se passar T.root como parâmetro em node
AVL_Tree_Insert(node, key) {
    if node == NIL {
        node = AVLTreeNode(key)
        return node
    }
    # Na ida da recursão, caminho TOP-DOWN
    if key < node.key
        node.left = AVL_Tree_Insert(node.left, key)
    elif key > node.key
        node.right = AVL_Tree_Insert(node.right, key)
    else # se chave já existem em T, não insere novamente
        return node
    # Caminho BOTTOM-UP iniciando no pai do nó inserido
    # Na volta da recursão
    # atualiza as alturas dos antecessores
    node.h = 1 + max(height(node.left), height(node.right))
    # calcula o fator de balanço
    b = balance(node)
    # Temos que analisar 4 casos possíveis
    # Caso 1: RE
    if b > 1 and key < node.left.key
        return rightRotate(node)
    # Caso 2: RD
    if b < -1 and key > node.right.key
        return leftRotate(node)
    # Caso 3: RED
    if b > 1 and key > node.left.key
        return leftRightRotate(node)
    # Caso 4: RDE
    if b < -1 and key < node.right.key
        return rightLeftRotate(node)
    # Se estiver balanceado, retorna o nó sem modificações
    return node
}

```

Pode-se mostrar que, após uma inserção, são necessárias no máximo 2 rotações para restaurar o balanço da árvore AVL.

Remoção de nós em árvores AVL

Assim como vimos anteriormente, durante a remoção de um nó em uma árvore binária de busca, há 3 casos principais a serem observados:

1. O nó a ser removido é uma folha
2. O nó a ser removido possui um único filho
3. O nó a ser removido possui 2 filhos

Por conveniência, denominaremos de w o nó a ser removido da árvore.

Caso 1: após a remoção, devemos apenas checar o fator de balanço dos antecessores.

Iniciando no pai de w (denotado por y), devemos percorrer um caminho ascendente até encontrar o primeiro nó cujo fator de balanço é maior que 1 ou menor que -1 (note que tal nó pode ser o próprio pai de w , ou seja y) ou chegar em NIL. Denominaremos esse primeiro nó desbalanceado de z .

Caso 2: devemos substituir o nó removido pelo seu único filho e checar o fator de balanço dos antecessores.

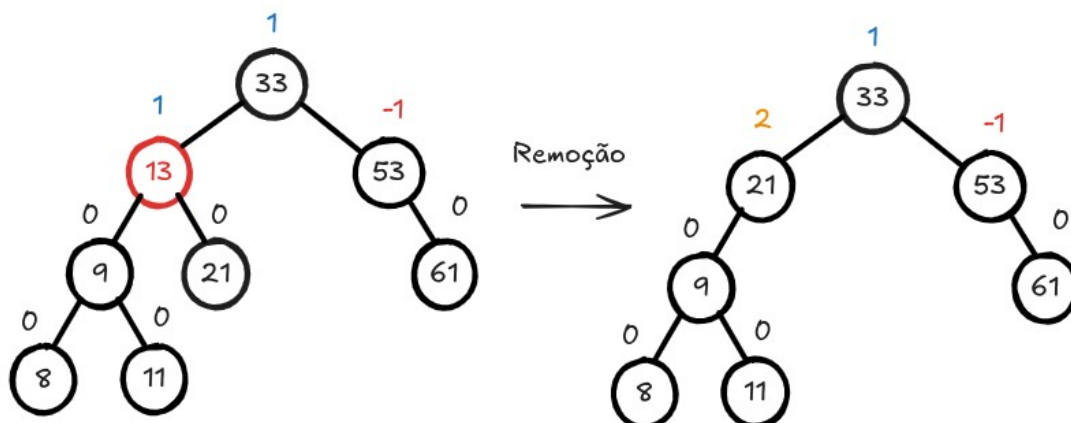
Iniciando no nó substituído, denotado por y , devemos percorrer um caminho ascendente até encontrar o primeiro nó desbalanceado z .

Caso 3: devemos encontrar o sucessor do nó na árvore e como ele sempre terá um único filho (menor chave da subárvore a direita), podemos substituir o nó removido por ele. Em seguida, devemos checar o fator de balanço dos antecessores.

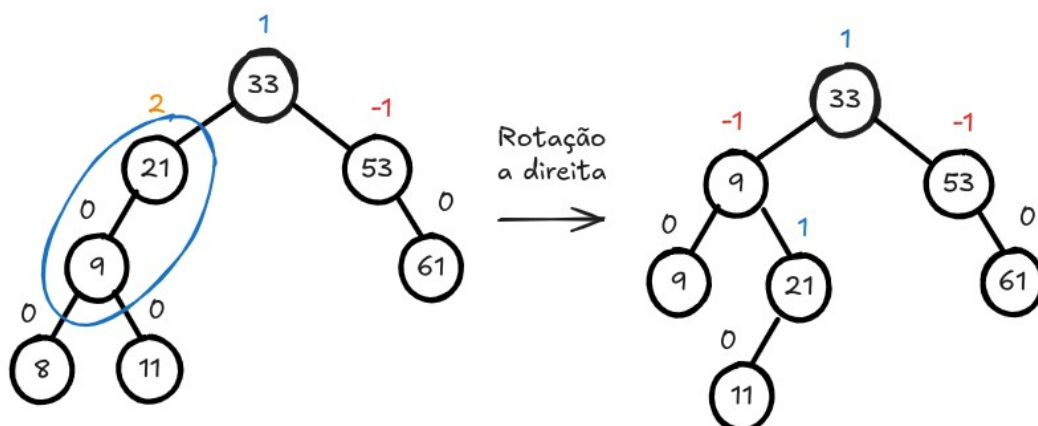
Iniciando no nó substituído, denotado por y , devemos percorrer um caminho ascendente até encontrar o primeiro nó desbalanceado z . A regra para o rebalanceamento da árvore consiste em rebalancear o ancestral mais próximo como:

```
if z.b > 1 {  
    if z.left.b ≥ 0  
        rightRotate(z)  
    else  
        leftRightRotate(z)  
}
```

As figuras a seguir ilustram um exemplo didático de remoção em árvores AVL.



Sendo assim, é preciso realizar uma rotação a direita, conforme ilustra a figura a seguir.



Para o caso simétrico, temos uma regra similar.

```

if z.b < -1 {
    if z.right.b ≤ 0
        leftRotate(z)
    else
        rightLeftRotate(z)
}

```

A função a seguir mostra como podemos realizar a remoção de um nó em uma árvore AVL de maneira recursiva.

```

AVL_Tree_Delete(root, key) {
    # Caminho TOP-DOWN para encontrar nó
    if root == NIL
        return root
    if key < root.key
        root.left = AVL_Tree_Delete(root.left, key)
    elif key > root.key
        root.right = AVL_Tree_Delete(root.right, key)
    else {
        # Se entrar aqui, achou o nó a ser removido!
        if root.left == NIL or root.right == NIL {
            if root.left ≠ NIL
                temp = root.left
            if root.right ≠ NIL
                temp = root.right
            # Se nó removido é folha
            if temp == NIL
                root = NIL
            else
                # Nó a ser removido tem um único filho
                root = temp
        }
        else {
            # Nó a ser removido possui 2 filhos
            # Encontrar o sucessor (menor da subárvore a direita)
            temp = Tree_Minimum(root.right)
            root.key = temp.key
            # Remove o sucessor (sempre terá 1 único filho)
            root.right = AVL_Tree_Delete(root.right, temp.key)
        }
    }
    # Condição de parada da recursão (atingiu NIL)
    if root == NIL
        return root
    # Caminho BOTTOM-UP para balanceamento (volta da recursão)
    # Na volta da recursão, atualiza as alturas dos antecessores
    root.h = 1 + max(height(root.left), height(root.right))
    # Fator de balanço dos ancestrais
    b = balance(root)
    # Temos que analisar 4 casos possíveis
    # Caso 1: RD
    if b > 1 and balance(root.left) ≥ 0
        return rightRotate(node)
    # Caso 2: RED
    if b > 1 and balance(root.left) < 0
        return leftRightRotate(node)
}

```

```
# Caso 3: RE
if b < -1 and balance(root.right) ≤ 0
    return leftRotate(node)
# Caso 4: RDE
if b < -1 and balance(root.right) > 0
    return rightLeftRotate(node)
# Se estiver balanceado, retorna o nó sem modificações na recursão
return root

}
```

Existem outros tipos de árvores aproximadamente balanceadas que não precisam de operações de rotação. Os exemplos mais conhecidos são as árvores rubro negras e as árvores de busca digitais (Tries). No próximo capítulo, apresentaremos uma breve introdução às árvores rubro-negras.

"You are a piece of the puzzle of someone else's life. You may never know where you fit, but others will fill the holes in their lives with pieces of you."
-- Bonnie Arbon

Estruturas de Dados: Árvores de Busca Digitais (Tries)

Nas árvores AVL, a posição de um nó na estrutura depende essencialmente da comparação entre as chaves. Nas árvores de busca digitais (Tries), a posição de um nó depende da comparação entre os bits que compõem as chaves. Esse tipo de estrutura de dados possui vantagens e desvantagens, sendo que as principais delas são:

- *Vantagens: implementação simples pois não exige operações de rotações.
- * Desvantagens: desempenho depende do tamanho das chaves (quanto maior, pior).

Suponha um conjunto finito de chaves Ω dado por:

$$\Omega = \{A, B, C, D, \dots, W, X, Y, Z\}$$

Considere que as chaves são codificadas em binário de acordo com a sua posição no conjunto. Como temos 26 símbolos, são necessários 5 bits para representá-los ($2^5 = 32$).

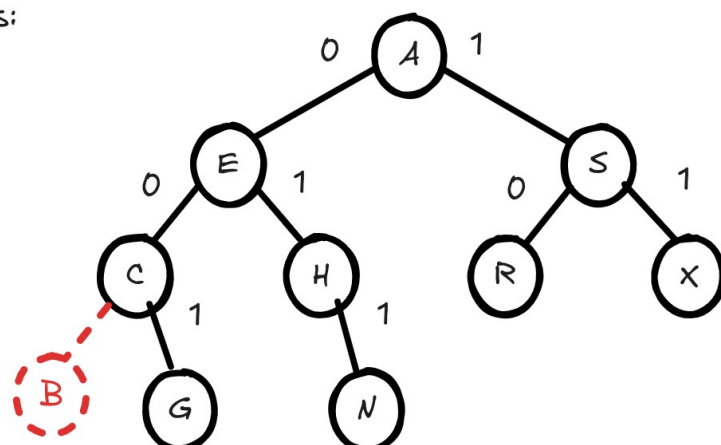
A: 00001	B: 00010	C: 00011	D: 00100	E: 00101	F: 00110	G: 00111
H: 01000	I: 01001	J: 01010	K: 01011	L: 01100	M: 01101	N: 01110
O: 01111	P: 10000	Q: 10001	R: 10010	S: 10011	T: 10100	U: 10101
V: 10110	W: 10111	X: 11000	Y: 11001	Z: 11001		

Busca em árvores binárias digitais

A estratégia para a busca em árvores binárias digitais consiste em comparar um bit da chave buscada com um bit do nó correto. Mas as chaves são compostas por vários bits, como saber quais deles devemos comparar? Depende da profundidade em que o nó corrente se encontra! Se estamos na raiz da árvore, comparamos o primeiro bit (da esquerda para direita), se estamos no segundo nível, comparamos o segundo bit, e assim sucessivamente. Por essa razão, a profundidade da árvore está diretamente relacionada com o número de bits necessários para codificar as chaves. A figura a seguir ilustra um exemplo.

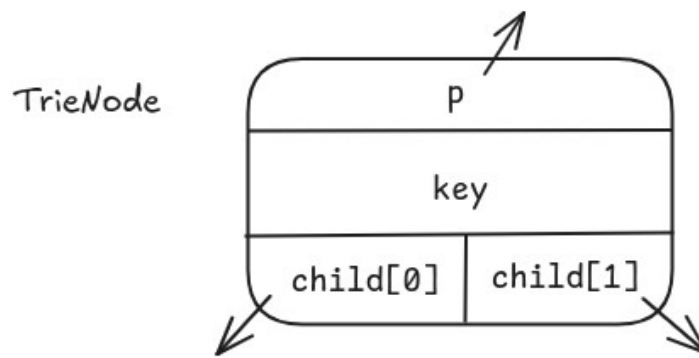
Buscar as seguintes chaves:

G: 00111 (ok)
R: 10010 (ok)
B: 00010 (x)

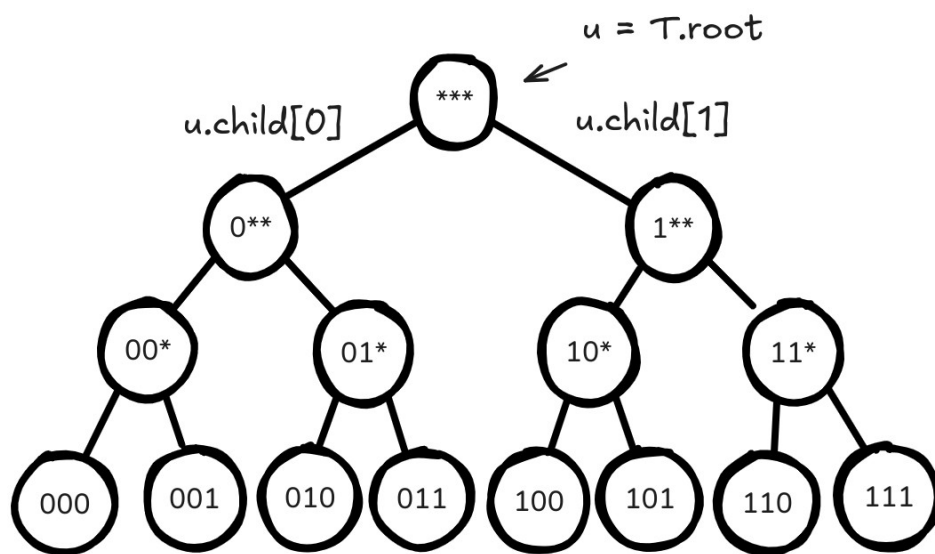


Em resumo, suponha que desejamos buscar a chave G (00111). Partindo da raiz, devemos visitar o filho a esquerda, pois o primeiro bit é zero. No segundo nível devemos novamente visitar o filho a esquerda, pois o segundo bit também é zero. No terceiro, visitamos o filho a direita pois o bit é 1.

Podemos definir o nó de uma árvore de busca digital como:



Para projetarmos um algoritmo de busca em árvores binárias digitais, considere um cenário em que as chaves possuem 3 bits. Note que nesse caso não há como ter caminhos com tamanho maior que 3, o que implica que qualquer chave deverá estar a no máximo uma distância 3 da raiz.



Com base nessa ideia, o algoritmo de busca em uma árvore digital de w bits é dado a seguir.

```

Trie_Search(T, x) {
    u = T.root
    for i = 0 to w-1 {
        c = (x.key >> (w-1)-i) & 1 # extrai bit a ser comparado
        if u.key ≠ x.key
            u = u.child[c]          # desce um nível
        else
            return True              # encontrou
        if u == NIL
            return False             # não encontrou
    }
}

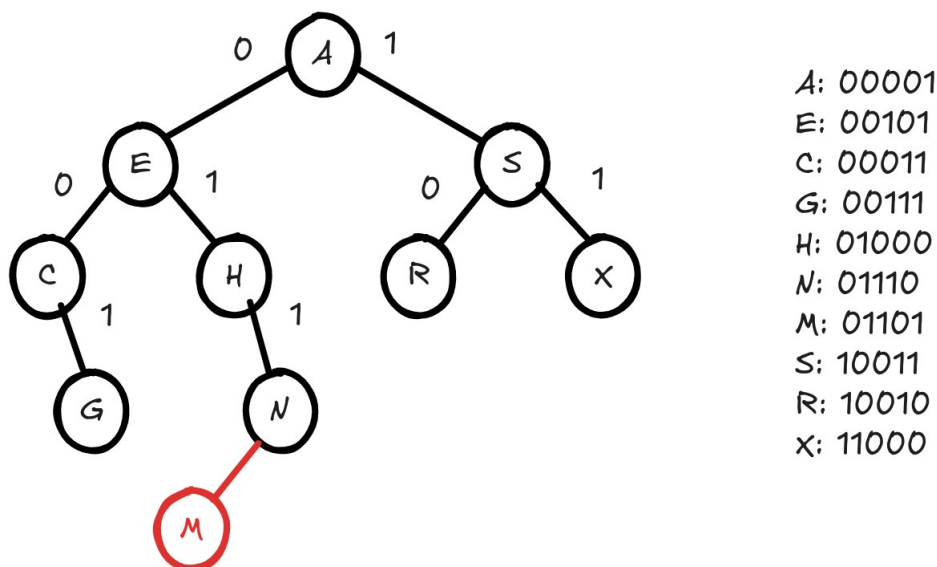
```

O comando para extrair o bit a ser analisado usa operadores binários. Suponha $w = 3$ bits e que a chave a ser buscada é 100. A ideia é que se estamos na raiz, o bit a ser analisado é o 1. Na raiz, o valor de $i = 0$ e assim, devemos deslocar os bits de $x.key$ duas vezes para direita gerando 001 e fazer utilizar o operador lógico & (and) com o valor 1 para obter um bit 1. Antes de apresentarmos a inserção em árvores binárias digitais, veremos uma propriedade fundamental.

Propriedade chave: Toda chave de uma árvore de busca digital está em algum ponto do caminho definido pelos seus bits.

Inserção em árvores binárias digitais

Seja M a chave do nó a ser inserido, $M = 01101$. Logo, essa chave deverá ser inserida na primeira posição livre nesse caminho. A figura a seguir ilustra uma sequência de inserções em uma árvore de busca digital.



Note, porém, que a árvore não mantém as chaves em ordem: $E > A$ e está na subárvore a esquerda de A. Entretanto, as chaves da subárvore a esquerda são todas menores que as chaves da subárvore a direita! Outro detalhe é que todas as chaves em uma subárvore no nível K possuem todos os K-1 bits iniciais idênticos.

A ideia é que nas árvores de busca digitais não é preciso realizar balanceamento, pois após a inserção de todas as chaves a árvore se manterá aproximadamente balanceada por conta da propriedade chave.. Dessa forma, eliminamos a necessidade de operações de rotação.

Sob hipótese de que todas as chaves são distintas e possuem w bits, o número total de elementos que podem ser armazenados é $n = 2^w$. Isso porque a altura máxima da árvore é justamente w. Se as chaves são uniformes, a altura da árvore é $h = \log_2 n = w$.

O algoritmo de inserção em uma árvore digital de w bits é dado a seguir.

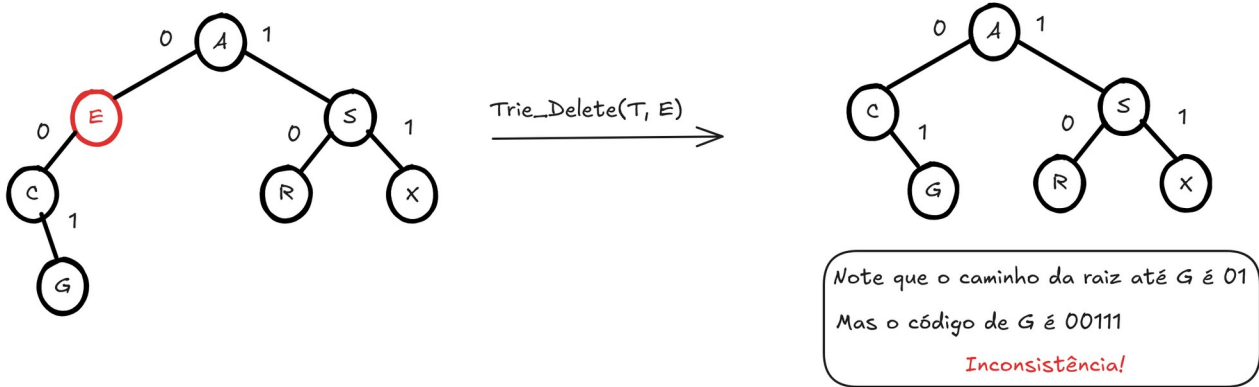
```

Trie_Insert(T, x) {
    u = T.root
    for i = 0 to w-1 {
        c = (x.key >> (w-1)-i) & 1 # extrai bit a ser comparado
        if u.key == x.key
            return False           # chave já pertence a T
        if u.child[c] == NIL
            break                   # achou posição da chave
        u = u.child[c]
    }
    u.child[c] = x
    x.p = u                        # pai de x é u
}

```

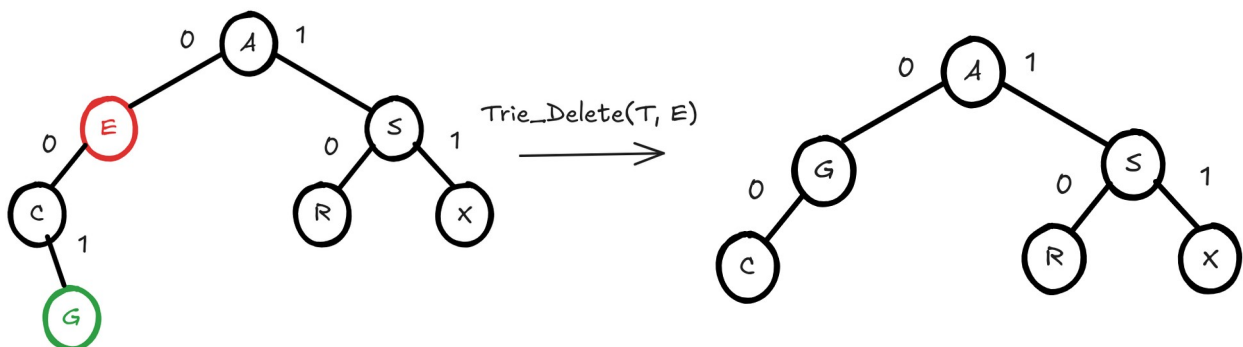
Remoção em árvores binárias digitais

A remoção de nós em árvores binárias digitais pode ser problemática, uma vez que em certos casos, a propriedade chave pode ser desfeita. O caso mais simples ocorre quando o nó a ser removido é uma folha, ou seja, não possui filhos. Neste caso, basta remover o nó desligando as referências com o nó pai. Porém, quando o nó a ser removido possui um ou dois filhos, podem surgir problemas. Vejamos um exemplo ilustrativo. Considere a árvore binária digital a seguir.



Suponha que deseja-se remover a chave E. Após a simples remoção, a chave C passa a ser filho a esquerda de A. Porém, note que de acordo com a codificação G: 00111, mas note que o caminho da raiz até G é 01. Inconsistência! O problema aqui é que 01 não é um prefixo de G, de modo que temos uma violação da propriedade chave. O que fazer?

Devemos substituir o nó removido (E) por algum descendente que seja uma folha! Depois, basta remover a folha. Isso resolve o problema, pois um nó folha pode ser colocado em qualquer posição do caminho da raiz até ele (propriedade chave). Note que G: 00111 tem o mesmo prefixo que E:00101. A árvore a seguir mostra o resultado final.



O algoritmo de remoção em uma árvore

digital de w bits é dado a seguir.

```
Trie_Delete(T, x) {  
    u = T.root  
    for i = 0 to w-1 {  
        c = (x.key >> (w-1)-i) & 1      # extrai bit a ser comparado  
        if u.key == x.key                # encontrou chave a ser removida  
            break  
        if u.child[c] == NIL  
            return False                 # chave não pertence a T  
        u = u.child[c]  
    }  
}
```

```

if i == w-1                                # se nó é folha
    u.p.child[c] = NIL                      # desaloca o nó
else {                                     # se nó não é folha
    z = u
    while z.child[0] ≠ NIL or z.child[1] ≠ NIL {
        if z.child[0] ≠ NIL {
            z = z.child[0]
            c = 0                            # vim da esquerda
        }
        else {
            z = z.child[1]
            c = 1                            # vim da direita
        }
    }
    u.key = z.key                          # atualiza chave
    z.p.child[c] = NIL                      # desaloca o nó
}

```

"One day you'll give someone advice you once needed and realize that it no longer applies to you. That is when you know you've grown."
 -- Author Unknown

Estruturas de Dados: Árvores Rubro-Negras

Árvores rubro-negras são árvores binárias de busca balanceadas que, embora utilizem operações de rotação, seus nós não possuem fator de balanço. Mas como é possível criar uma árvore balanceada sem fator de balanço?

Veremos que, pela definição de uma árvore rubro-negra, o balanceamento é uma consequência lógica.

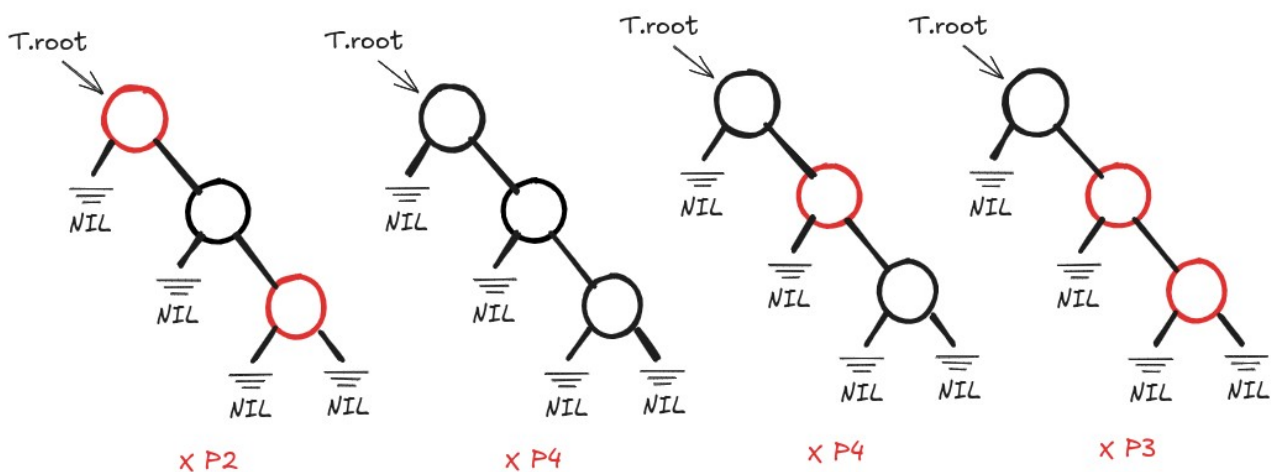
Definição: Uma árvore T é rubro-negra se:

1. Cada nó é **vermelho** ou **preto**.
2. A raiz sempre é **preta**.
3. Dois nós **vermelhos** não podem ser adjacentes (um nó **vermelho** só pode ter filhos **pretos**).
4. Todo caminho da raiz até um apontador NIL (caminho raiz-NIL) tem o mesmo número de nós **pretos** (pense nesses caminhos como aqueles percorridos em buscas mal sucedidas).

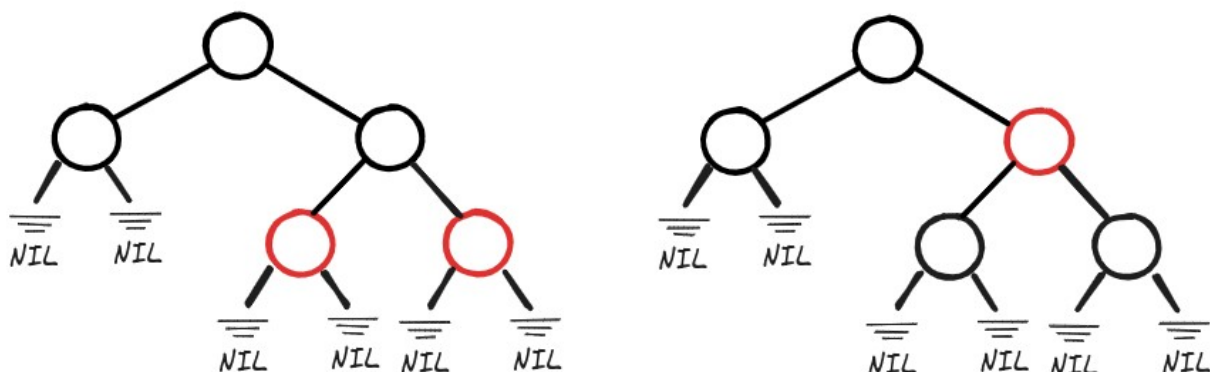
Uma conclusão lógica das premissas anteriores é: se um nó possui um único filho, ele deve ser vermelho! Caso contrário, iria ferir propriedade 4.

Iremos analisar intuitivamente porque essas propriedades nos levam a uma árvore balanceada.

Iniciamos pelas menores árvores possíveis: árvores de 3 nós.



Note que todas as árvores mostradas na figura acima ferem alguma das 4 propriedades, ou seja, nenhuma delas é uma árvore rubro-negra válida. Exemplos de árvores rubro-negras válidas são mostradas a seguir.



Note que nas duas árvores anteriores, o número de nós pretos em qualquer caminho raiz-NIL é sempre igual a 2.

Uma pergunta interessante é: qual é a altura máxima de uma árvore rubro-negra? A seguir iremos analisar como responder a essa questão.

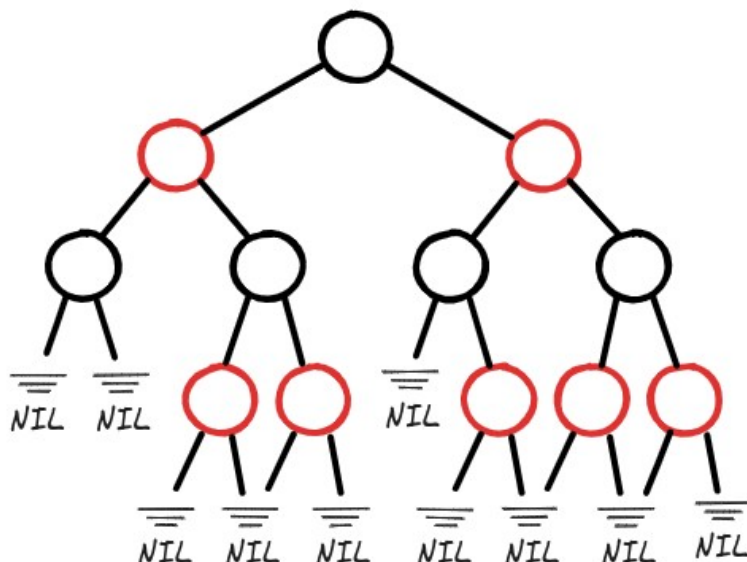
Altura máxima de árvores rubro-negras

Lembre-se que a altura de uma árvore é igual ao número de arestas do maior caminho raiz-NIL. Sendo assim, desejamos limitar superiormente o comprimento de caminhos em árvores rubro-negras.

Teorema: Se todo caminho raiz-NIL de uma árvore rubro-negra T possui pelo menos k nós, então os primeiros k níveis da árvore T devem estar completos.

É simples verificar isso pela contrapositiva: $p \rightarrow q \equiv \neg q \rightarrow \neg p$

Se os primeiros k níveis de T não estão completos, então existe um caminho raiz-NIL com menos de k nós.



Note que na árvore T acima, como o nível $k = 3$ não está completo, existem caminhos raiz-NIL de tamanho $k - 1 = 2$. Lembre-se que o comprimento de um caminho C é $|C| - 1$, onde $|C|$ denota o número de nós no caminho C .

Supondo que os k primeiros níveis da árvore estão completos, então o número de nós em T é maior ou igual o número de nós de uma árvore binária completa de altura $k - 1$, ou seja:

$$n \geq 2^0 + 2^1 + 2^2 + \dots + 2^{k-1} = \sum_{i=0}^{k-1} 2^i = 2^k - 1$$

o que nos leva a

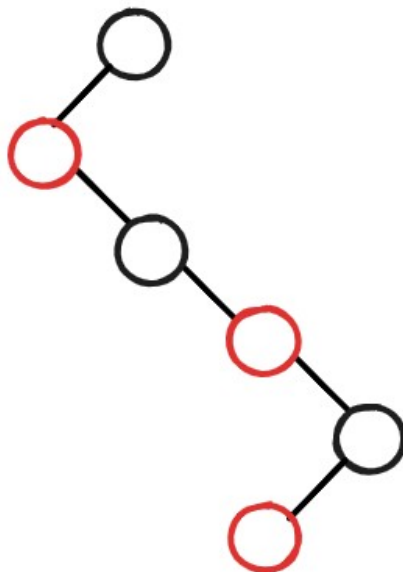
$$2^k \leq n + 1$$

ou seja, $k \leq \log_2(n + 1)$

o que mostra que k é $O(\log n)$, ou seja, a altura é proporcional ao log do número de nós.

Pela propriedade 4 da definição, todo caminho raiz-NIL tem o mesmo número de nós pretos. Digamos que esse número é p . Então, temos um limitante superior para o número de nós pretos em qualquer caminho raiz-NIL, pois o valor de p não pode exceder a altura k , ou seja, $p \leq \log_2(n+1)$.

No entanto, desejamos encontrar um limitante para o número total de nós da árvore T em qualquer caminho, já que isso limita a altura da árvore. Note que pelas propriedades 2 e 3 da definição, todo caminho tem no máximo um nó vermelho para cada nó preto, pois a raiz é preta e não são permitidos dois nós vermelhos consecutivos.



Assim, o número total de nós t em qualquer caminho raiz-NIL da árvore é no máximo duas vezes o número de nós pretos, ou seja, $t = 2p \leq 2 \log_2(n+1)$.

Esse resultado é fundamental, pois nos permite caracterizar árvores rubro-negras pela sua propriedade chave:

=> O comprimento do caminho da raiz até a folha mais distante é no máximo duas vezes o comprimento do caminho da raiz até a folha mais próxima.

Esse resultado garante que árvores rubro-negras são balanceadas! Embora não tão balanceadas quanto as árvores AVL, mas mesmo assim garantem que as operações de busca, inserção e remoção são todas $O(\log n)$.

Inserção em árvores rubro-negras

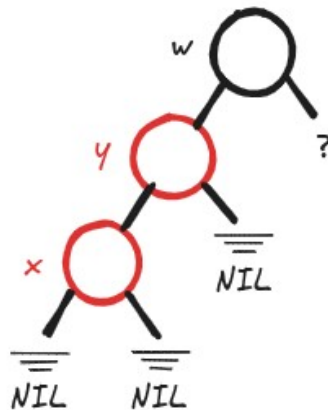
A ideia básica inserir um novo nó como uma folha **vermelha** após percorrer um caminho descendente na árvore, iniciando pela raiz. Em seguida devemos utilizar recolorações e rotações para reestabelecer as propriedades da árvore rubro-negra, percorrendo um caminho ascendente.

A seguir iremos analisar alguns casos que podem ocorrer na volta da recursão, quando percorrermos o caminho ascendente, sempre considerando que o nó corrente é o nó x .

Caso 1: se y , o pai de x não for vermelho, OK. Propriedades estão mantidas!

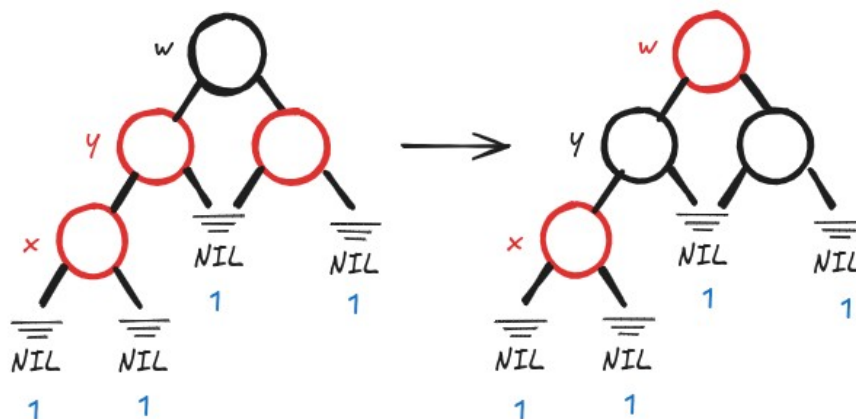


Caso 2: se y (pai de x) é **vermelho**, temos que restaurar a propriedade 3. Sabemos que, por conta da propriedade 2, y não é a raiz de T (raiz é **preta**). E que w , o pai de y , é preto, devido a propriedade 3.



Analisaremos a seguir, dois subcasos que podem ocorrer.

Caso 2.1: w tem outro filho z de cor **vermelha**



Neste caso, vamos recolorir y e z para **preto** e w para **vermelho**.

Note que a propriedade 4 está preservada, pois todos os caminhos raiz-NIL passam pelo mesmo número de vértices pretos. Porém, a propriedade 3, violada entre os nós x e y , foi restaurada.

Ao continuar a subida na árvore, será necessário analisar a situação de w (que ficou vermelho), pois essa alteração pode fazê-lo violar a propriedade 3 em relação a seu pai. Se w for a raiz da árvore,

basta mudar a cor dele para preto. Porque isso é permitido? Pois a raiz é o único nó da árvore que está presente em todos os caminhos raiz-NIL!

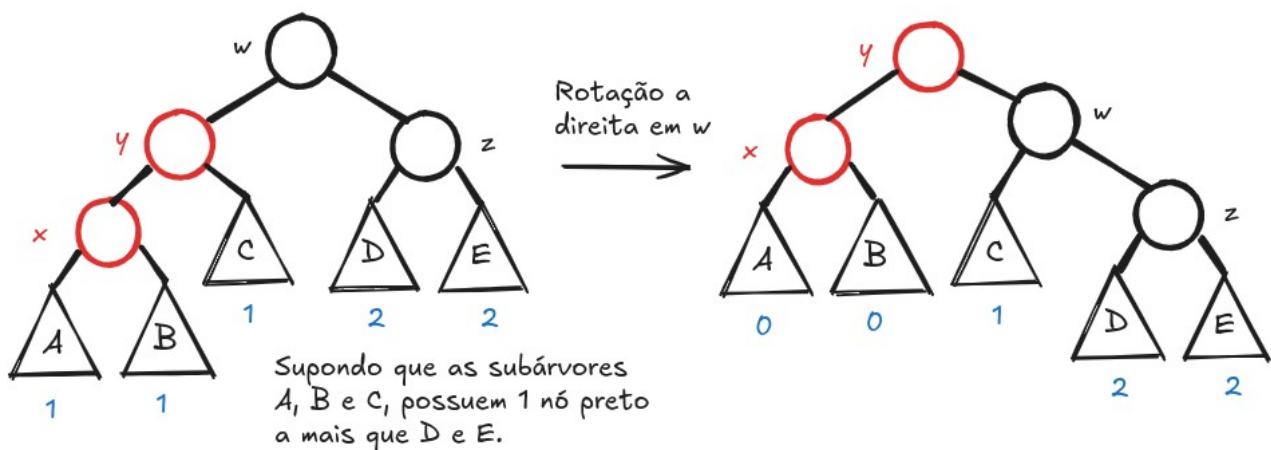
Caso 2.2: w não tem outro filho z de cor **vermelha**

Então, w pode ter outro filho **preto** z ou mesmo não ter outro filho

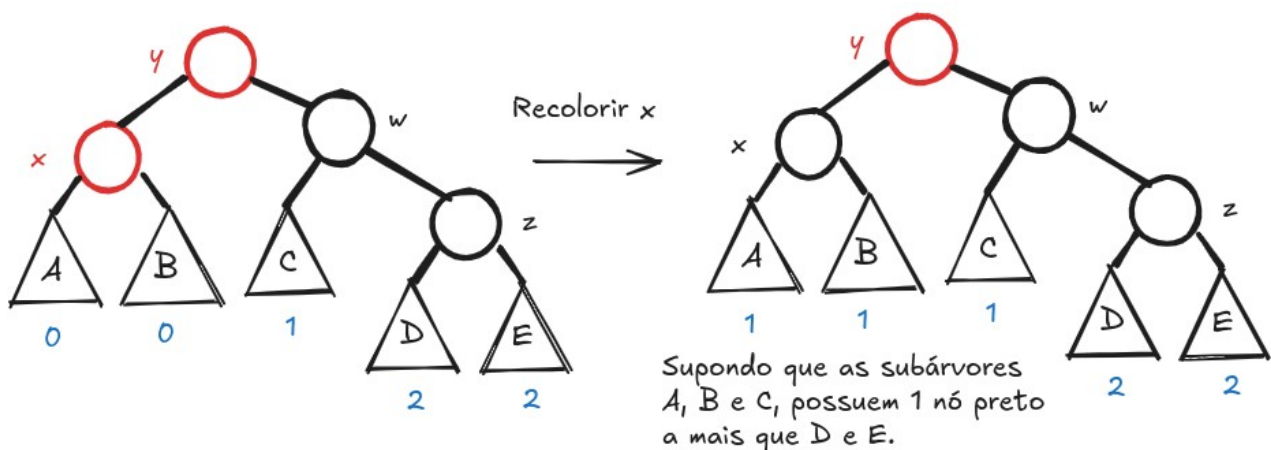
Vamos considerar a situação a) como:

a) w possui um outro filho **preto** z

Devemos realizar uma rotação a direita para inverter a relação entre w e y.



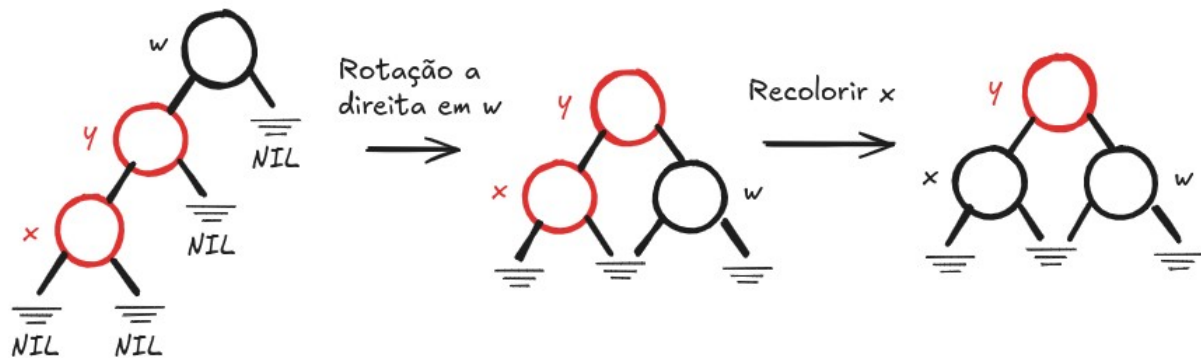
Porém, note que após a rotação, o número de nós pretos nos caminhos que vão da raiz até os filhos de x é reduzido! Para contornar esse problema, devemos mudar a cor de x para preto, o que resolve tanto o problema do número de nós pretos nos caminhos da árvore quanto os nós vermelhos adjacentes.



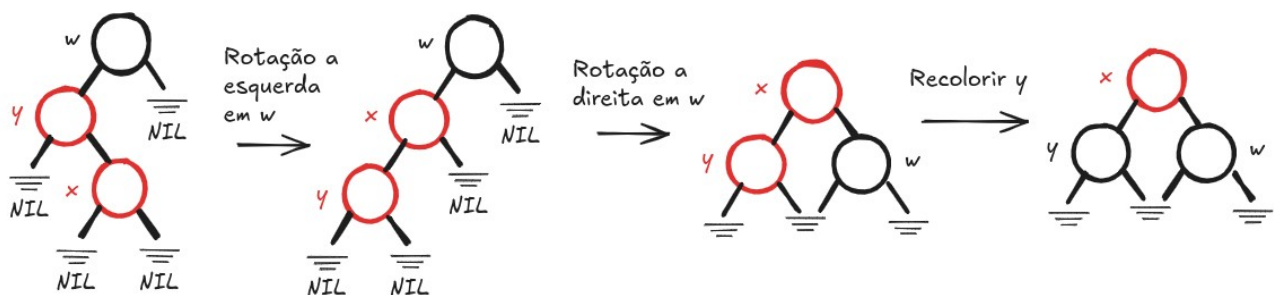
Ao continuar a subida na árvore (caminhos ascendente), será necessário analisar a situação de y, que é vermelho e se tornou a raiz da subárvore, pois ele pode violar a propriedade 3 com relação a seu pai. Se y for a nova raiz da árvore, basta recolorir y de preto (pois todo caminho raiz-NIL inclui a raiz).

b) w não possui outro filho

A mesma solução descrita anteriormente também funciona quando x é filho a esquerda de y.



Mas e se x é filho a direita de y?



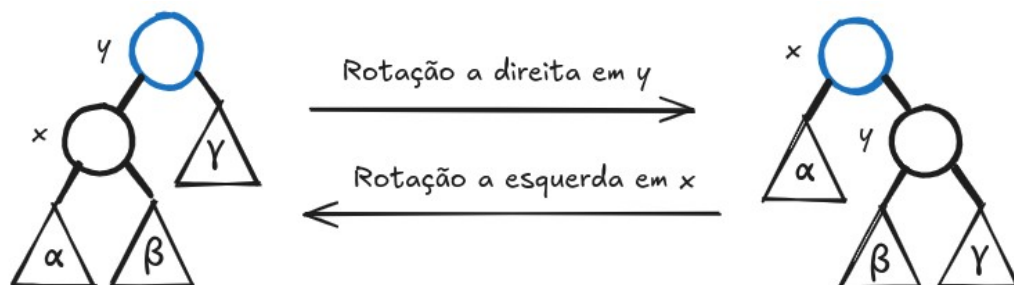
A seguir veremos os algoritmos necessários para a inserção em uma árvore rubro-negra. Iniciaremos lembrando a rotação a esquerda.

```

Left_Rotate(T, x) {
    y = x.right
    x.right = y.left
    if y.left != NIL
        y.left.p = x
    y.p = x.p
    if x.p == NIL
        T.root = y
    elif x == x.p.left
        x.p.left = y
    else
        x.p.right = y
    y.left = x
    x.p = y
}

```

subárv esq. de y vira subárv dir. de x
 # se subárvore esq. De y não é vazia
 # x torna-se o pai dessa subárvore
 # pai de x se torna pai de y
 # se x era a raiz
 # y se torna nova raiz
 # senão, se x era filho a esquerda
 # então y se torna filho a esquerda
 # senão, x era filho a direita
 # e y é filho a direita agora
 # faz x se tornar filho a esquerda de y
 # faz y se tornar pai de x



Com base no algoritmo acima, escreva a função para rotação a direita. A seguir apresentamos a função para inserção em uma árvore rubro-negra (percurso top-down).

```
Red_Black_Insert(T, z) {
    x = T.root           # nó que será comparado com z (buscar)
    y = NIL              # será o pai de z
    while x ≠ NIL {      # descer até a posição correta
        y = x
        if z.key < x.key
            x = x.left
        else
            x = x.right
    }
    z.p = y              # achou posição correta
    if y == NIL
        T.root = z       # árvore estava vazia
    elif z.key < y.key
        y.left = z        # z é filho a esquerda
    else
        y.right = z       # z é filho a direita
    z.left = NIL
    z.right = NIL
    z.color = RED         # todo nó inserido inicia vermelho
    Red_Black_Insert_Fixup(T, z) # faz ajustes para preservar árvore
}
```

A função a seguir é responsável por realizar os ajustes necessários para que após a inserção de um novo nó, as propriedades da árvore rubro-negra permaneçam válidas (percurso bottom-up).

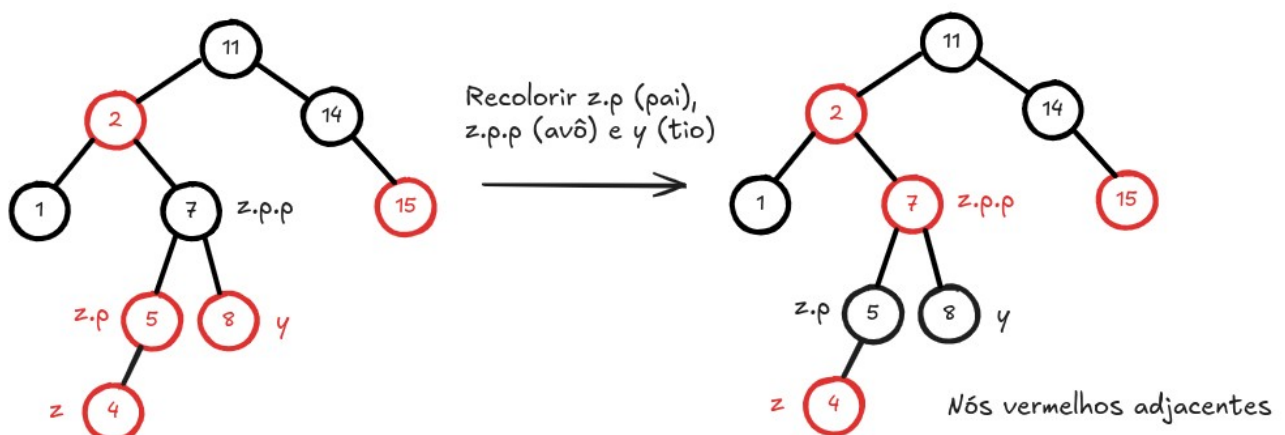
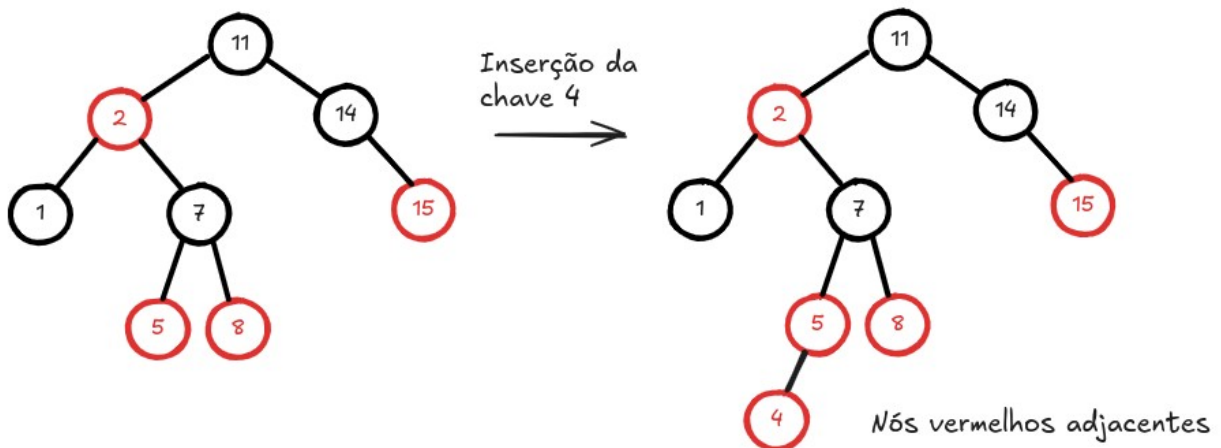
```
Red_Black_Insert_Fixup(T, z) {
    while z.p.color == RED {
        if z.p == z.p.p.left {          # o pai de z é filho a esq. ?
            y = z.p.p.right              # y é tio de z
            if y.color == RED {          # pai e tio de z são RED
                z.p.color = BLACK
                y.color = BLACK
                z.p.p.color = RED        # raiz da subárvore fica RED
                z = z.p.p
            }
        } else {
            if z = z.p.right {            # z é filho a dir.
                z.p.color = BLACK
                z = leftRightRotate(z.p.p)
            } else {
                z.color = BLACK
                z = rightRotate(z.p.p)
            }
        }
    }
    else { # o espelhamento do código anterior
        y = z.p.p.left                  # y é tio de z
        if y.color == RED {             # pai e tio de z são RED
            z.p.color = BLACK
            y.color = BLACK
            z.p.p.color = RED            # raiz da subárvore fica RED
        }
    }
}
```

```

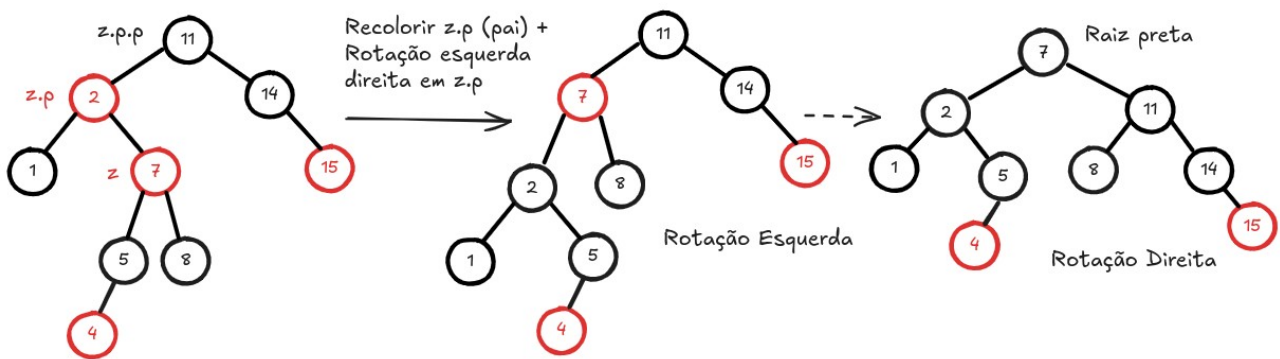
        z = z.p.p
    }
    else {
        if z = z.p.left {           # z é filho a esq.
            z.p.color = BLACK
            z = rightLeftRotate(z.p.p)
        }
        else {
            z.color = BLACK
            z = leftRotate(z.p.p)
        }
    }
}
T.root.color = BLACK
}

```

As figuras a seguir mostram um exemplo ilustrativo, em que a chave 4 é inserida em uma árvore rubro-negra. Note que após a inserção, as propriedades são violadas, mas aplicando as operações vistas anteriormente, as propriedades são restauradas.



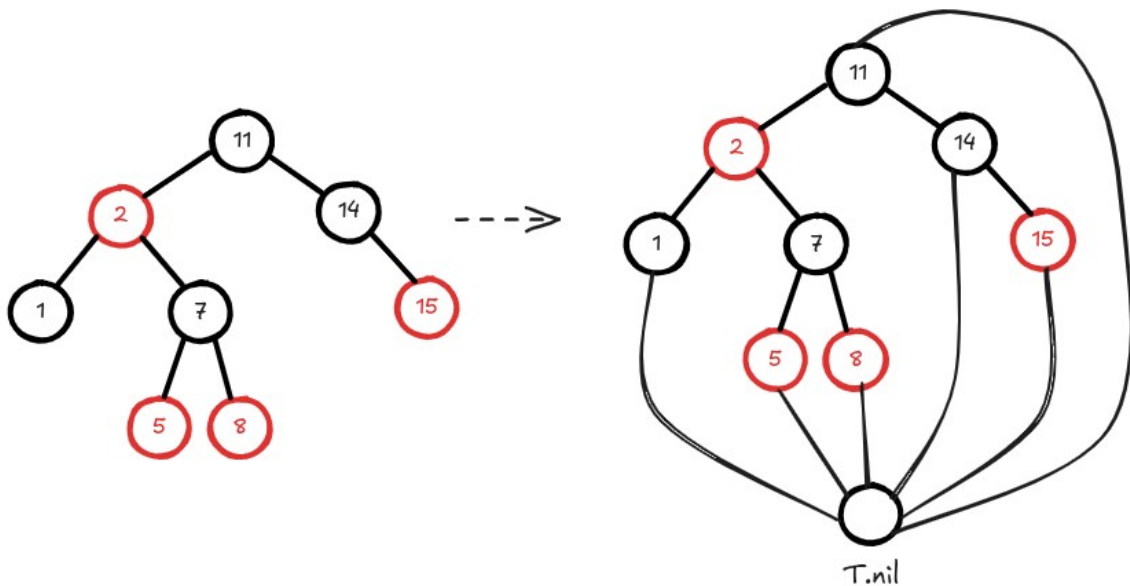
Neste ponto, devemos continuar subindo no caminho ascendente fazendo z apontar para seu avô, ou seja, fazendo $z = z.p.p$



Assim, finalmente a árvore está com todas as quatro propriedades restauradas. Note que ela encontra-se balanceada, como era de se esperar.

Remoção em árvores rubro-negras

Na remoção de nós em árvores rubro-negras, é preciso entender a organização lógica de uma árvore rubro-negra de maneira ligeiramente diferente. Ao invés de utilizarmos NIL como uma referência vazia, iremos denotar por $T.nil$, um nó preto na parte de baixo da árvore que representa a referência vazia. Por exemplo, a árvore a seguir ilustra esse conceito.



A remoção de um nó de uma árvore rubro-negra é mais complicado que a inserção de um novo nó. A ideia básica consiste em se inspirar na remoção de um nó em uma árvore binária de busca. Para tanto, primeiramente, iremos definir uma primitiva auxiliar RB-Transplant, que tem a mesma função do algoritmo Transplant na remoção de um nó de uma árvore binária de busca. Note que v pode ser inclusive $T.nil$.

```
RB-Transplant(T, u, v) {
    if u.p == T.nil
        T.root = v
    elif u == u.p.left
        u.p.left = v
    # nó u a ser substituído é a raiz de T
    # u é filho a esquerda de alguém
```

```

else                                     # u é filho a direita de alguém
    u.p.right = v
    v.p = u.p
}

```

A função RB-Delete é muito similar à função Tree_Delete utilizada para remover um nó de uma árvore binária de busca. A diferença principal é que após a remoção, devemos restaurar as propriedades da árvore rubro-negra, o que pode ser bastante complicado, devido aos diversos subcasos que devem ser analisados.

```

RB-Delete(T, z) {
    y = z
    y_orig_color = y.color
    if z.left == T.nil {
        x = z.right
        # substitui z pelo seu único filho a direita ou T.nil
        RB-Transplant(T, z, z.right)
    }
    elif z.right == T.nil {
        x = z.left
        # substitui z pelo seu único filho a esquerda
        RB-Transplant(T, z, z.left)
    }
    else {
        # se entrou aqui é porque z tem 2 filhos
        # encontra a menor chave da subárvore a direita (sucessor)
        y = Tree_Minimum(z.right) # pode ou não ser filho a dir
        y_orig_color = y.color
        x = y.right
        if y.p == z      # sucessor y é filho a direita de z
            x.p = y
        else {          # sucessor y está lá embaixo na árvore
            # Substitui y por seu filho a dir e ajusta refs
            RB-Transplant(T, y, y.right)
            y.right = z.right
            y.right.p = y
        }
        RB-Transplant(T, z, y)      # substitui z por y
        y.left = z.left             # filho a esquerda de z para y
        y.left.p = y
        y.color = z.color
    }
    if y_orig_color == BLACK # se nó removido era preto, problema!
        RB-Delete-Fixup(T, x) # x é o nó que substitui o removido
}

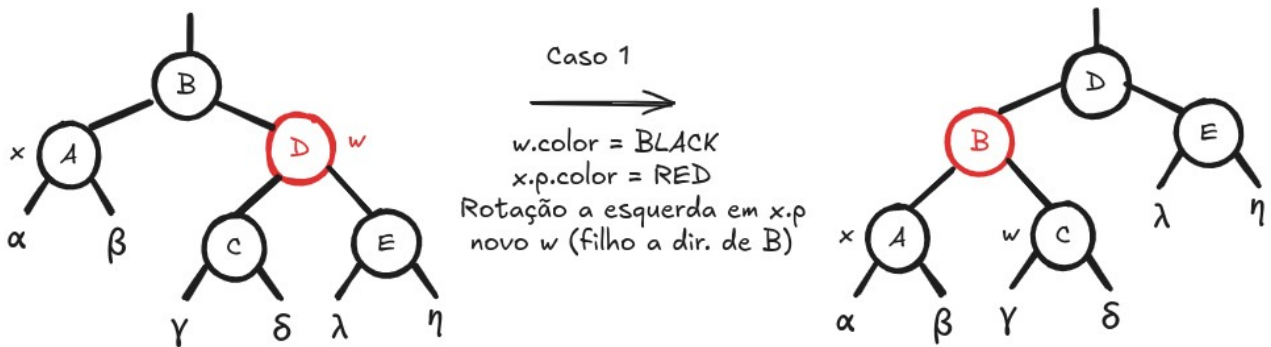
```

Note que se o nó y a ser removido é **BLACK**, uma ou mais propriedades da árvore rubro-negra podem ser violadas após sua remoção. A função RB-Delete-Fixup restaura as propriedades da árvore após a remoção. É importante salientar que se o nó a ser removido y é **RED**, as propriedades da árvore são preservadas, pois:

- i) O número de nós pretos nos caminhos raiz-NIL permanece inalterado.
- ii) Os nós vermelhos não se tornam adjacentes ao se remover um nó vermelho.

Em resumo, há 4 casos que podem ocorrer quando o nó removido y da árvore rubro-negra é substituído (por outro nó x ou por NIL). A seguir iremos ilustrar cada um deles.

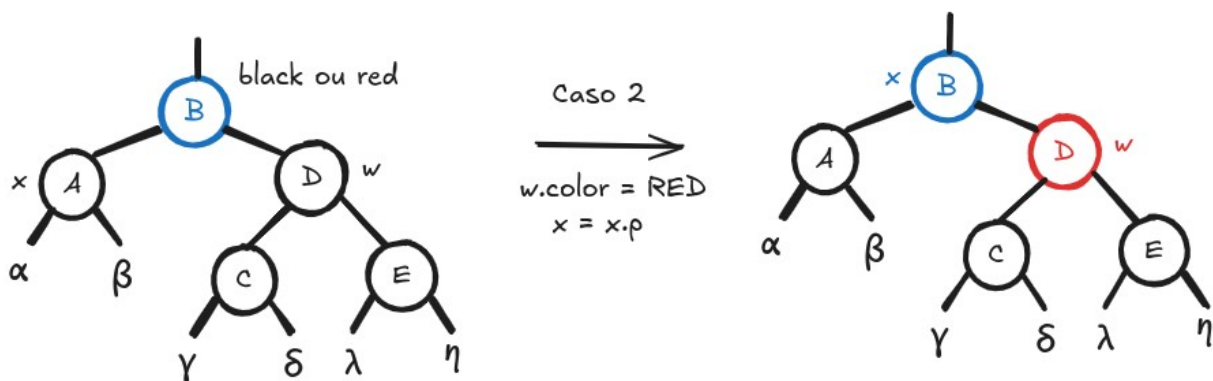
Caso 1: o nó x é filho a esquerda, é **BLACK** e possui um irmão **RED** w



- a) w fica **BLACK**
 $x.p$ fica **RED**
 Rotação a Esquerda em $x.p$
 novo w deve ser o filho a direita de B

Dessa forma, o caso 1 é transformado em caso 2, 3 ou 4, dependendo das cores dos vizinhos.

Caso 2: o nó x é filho a esquerda, seu irmão w é **BLACK** com os 2 filhos **BLACK**



- b) w fica **RED**
 x aponta para $x.p$

Caso 3: o nó x é filho a esquerda, seu irmão w é **BLACK** com o filho a direita **BLACK** e o filho a esquerda **RED**



- c) filho a esquerda de w fica **BLACK**
w fica **RED**
Rotação a Direita em w
novo w fica o filho a direita do pai de x

Caso 4: o nó x é filho a esquerda, seu irmão w é **BLACK** com o filho a direita **RED** (filho a esquerda de w pode ser qualquer cor)



- d) w fica com a cor do pai de x
o pai de x fica **BLACK**
o filho a direita de w fica **BLACK**
Rotação a Esquerda em x.p

Note que em todos os 4 casos descritos anteriormente, x é filho a esquerda de um nó arbitrário. Sendo assim, devemos tratar o caso simétrico quando x é filho a direita de alguém, o que nos leva portanto a um total de 8 casos distintos. Segue o algoritmo para a função RB-Delete-Fixup.

```
RB-Delete-Fixup(T, x) {
  while x ≠ T.root and x.color == BLACK {
    if x == x.p.left {
      # x é filho a esquerda
      # w é irmão de x
      w = x.p.right
      # Caso 1
      if w.color == RED {
        w.color = BLACK
        x.p.color = RED
        z = leftRotate(x.p)
        w = x.p.right
      }
      # Caso 2
      if w.left.color == BLACK and w.right.color == BLACK {
        w.color = RED
        x = x.p
      }
      else {
        # Caso 3
        if w.right.color == BLACK {
          w.left.color = BLACK
          w.color = RED
          z = rightRotate(w)
          w = x.p.right
        }
        # Caso 4
        w.color = x.p.color
      }
    }
  }
}
```

```

        x.p.color = BLACK
        w.right.color = BLACK
        z = leftRotate(x.p)
        x = T.root          # para sair do loop while
    }
}
else {    # x é filho a direita, igual mas troca left por right
    w = x.p.left          # w é irmão de x
    # Caso 5
    if w.color == RED {
        w.color = BLACK
        x.p.color = RED
        Right-Rotate(T, x.p)
        w = x.p.left
    }
    # Caso 6
    if w.right.color == BLACK and w.left.color == BLACK {
        w.color = RED
        x = x.p
    }
    else {
        # Caso 7
        if w.left.color == BLACK {
            w.right.color = BLACK
            w.color = RED
            Left-Rotate(T, w)
            w = x.p.left
        }
        # Caso 8
        w.color = x.p.color
        x.p.color = BLACK
        w.left.color = BLACK
        Right-Rotate(T, x.p)
        x = T.root        # para sair do loop while
    }
}
}
x.color = BLACK
}

```

Na teoria, as árvores AVL e rubro-negras possuem a mesma complexidade em suas operações de inserção, remoção e busca: $O(\log n)$. Em termos práticos, a árvore AVL é mais rápida na operação de busca, e mais lenta nas operações de inserção e remoção.

A principal vantagem das árvores rubro-negras é justamente nas inserções e remoções mais rápidas do que as inserções e remoções em árvores AVL. A principal desvantagem é que a busca em árvores AVL, em geral, é mais rápida do que em árvores rubro-negras, pois as árvores AVL são um pouco mais balanceadas do que as árvores rubro-negras. Portanto, se a operação prevalente for busca, árvores AVL são melhores, mas se as operações prevalentes forem inserção e remoção, árvores rubro-negras são melhores (as rotações são menos frequentes nas árvores rubro-negras).

"Once you stop learning you start dying."
Albert Einstein

Estruturas de Dados: Skip Lists

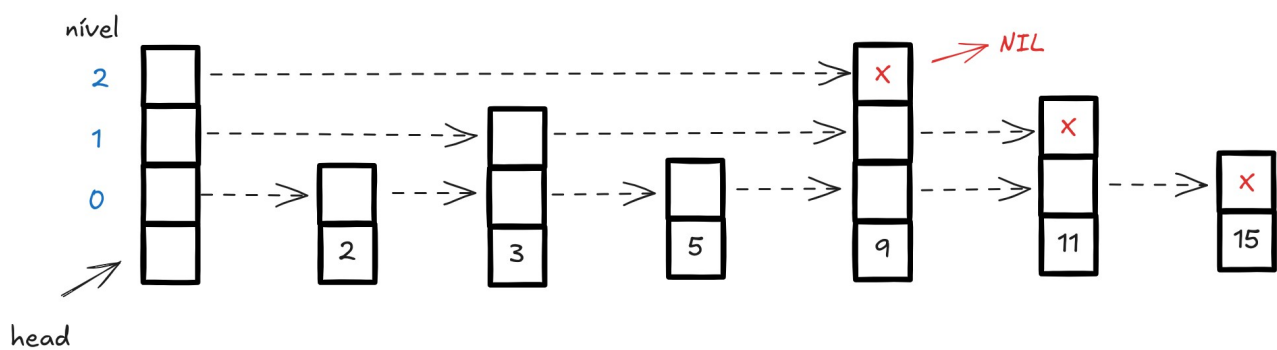
Skip Lists, ou listas com saltos, são estruturas de dados probabilísticas que permitem que as operações de busca, inserção e remoção de nós sejam realizadas com complexidade $O(\log n)$. Essas estruturas possuem as vantagens de um vetor ordenado (como na busca binária), mas com uma estrutura dinâmica similar a das listas encadeadas.

* Uma Skip List é composta por uma hierarquia de listas encadeadas de modo que quanto maior o nível, maior a esparsidade, ou seja, menos elementos aparecem.

* Uma Skip List utiliza um parâmetro conhecido como fator de dispersão ($d > 1$) para controlar a probabilidade de que um elemento do nível i seja replicado para o nível $i+1$.

Em resumo, uma Skip List é construída em camadas de modo que:

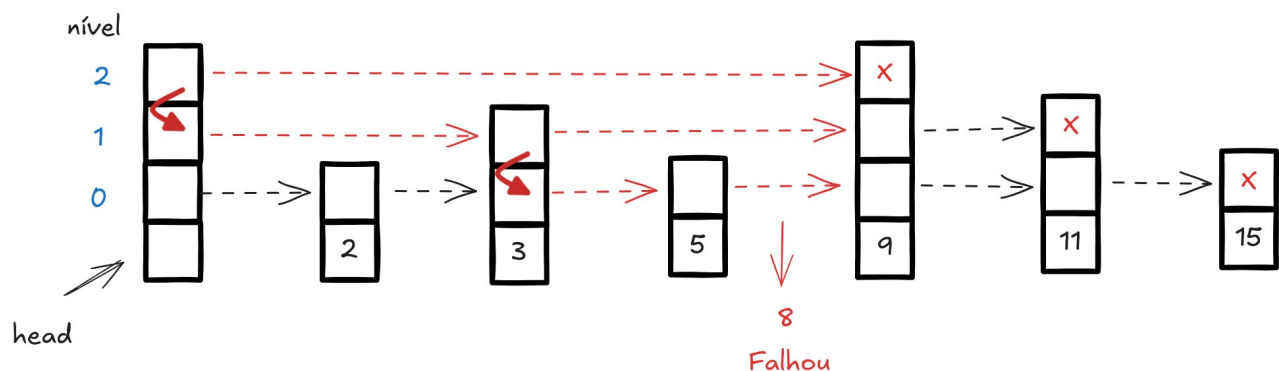
- i) a camada inferior é uma lista encadeada ordenada tradicional.
- ii) cada camada superior é uma “via expressa”, onde um elemento da camada i aparece na camada $i+1$ com probabilidade $q=1/d$. Note que se $d=2$, temos 50% de chances de aparecer (é como se jogássemos uma moeda e: se o resultado for cara o elemento da camada i será replicado na camada $i+1$, senão ele não é replicado).



Pode-se mostrar que, em média, cada elemento aparece em $d/(d-1)$ listas. O primeiro nó é um vetor de referências (ponteiros) que serve como as cabeças das diferentes listas encadeadas.

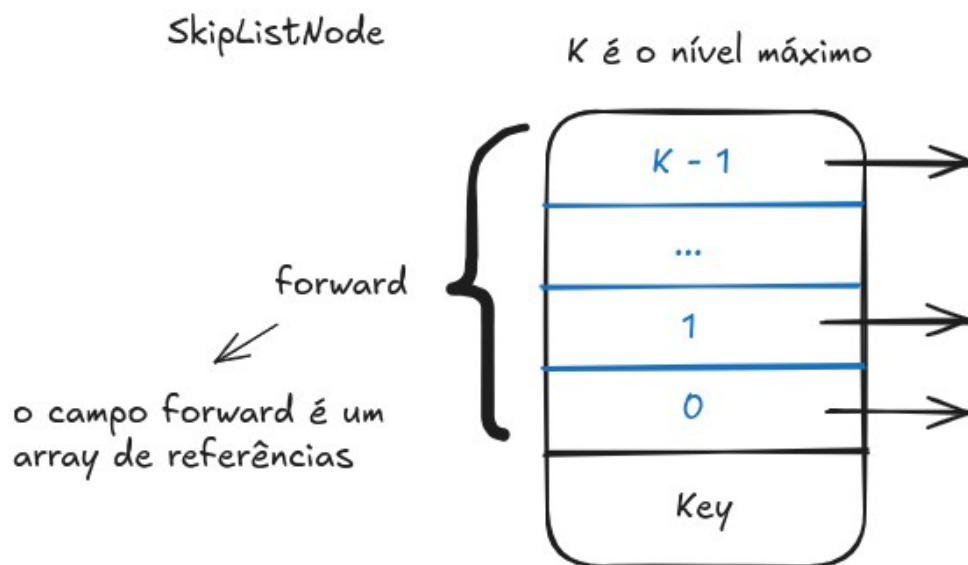
Busca em Skip Lists

Deve-se sempre iniciar a busca na lista superior (a mais esparsa) e caso não encontramos a chave, a busca deve continuar no nível abaixo. A figura a seguir ilustra a busca pela chave 8.



A lógica da busca pela chave 8 consiste em iniciar no nível 2 e percorrer a lista até que $x.key > 8$ ou chegar até o final da lista. Se não encontrou a chave, desce para o nível 1, passa pelas chave 3 e chega na chave 9. Novamente não encontrou, então desce para o nível 0, reiniciando do último elemento visitado no nível superior (3), passa pela chave 5 e chega na chave 9. Como estamos no nível mais baixo, a busca deve retornar que a chave 8 não pertence a Skip List. A função a seguir ilustra o algoritmo de inserção em uma Skip List.

Antes de apresentar o algoritmo de busca em Skip Lists, é preciso entender como são organizadas as informações em um nó arbitrário. A figura a seguir ilustra um nó de uma Skip List. Note que além do campo key, temos um array de referências para outros nós, o que nos permite ligar um nó a vários outros.



A seguir apresentamos o algoritmo para buscar uma chave em uma Skip List.

```
SkipList_Search(L, key) {
    x = L.head
    for i = L.level downto 0 {
        while x.forward[i].key < key
            x = x.forward[i]
    }
    # Aqui, estamos apontando para o antecessor da chave buscada
    x = x.forward[0]
    if x.key == key
        return x.key
    else
        return False
}
```

Análise da complexidade

O fator de dispersão representa a probabilidade de um elemento do nível i ser replicado para o nível $i+1$. Em outras palavras, podemos dizer que ao passar para o próximo nível, o número de nós na lista ligada cai, em média de $q=1/d$. Note que matematicamente temos:

nível	número de nós
0	n
1	n/d
2	n/d^2
3	n/d^3
...	
k	n/d^k

Mas no último nível, temos apenas 1 elemento, ou seja, $\frac{n}{d^k} = 1$, o que implica em $n = d^k$.

Logo, $k = \log_d n$. Sendo assim, uma skip list com n itens deve ter no máximo $1 + \log_d n$ níveis.

Além disso, é esperado que para cada nó do nível $i+1$ existam d nós no nível i . Por essa razão, espera-se que na busca por um elemento, em média, sejam necessários $d/2$ saltos em um nível antes de descer para o nível seguinte.

Portanto, pelo produto entre o número esperado de níveis e o número esperado de saltos por nível, o tempo médio de busca é $\frac{d}{2}(1 + \log_d n) = O(\log_d n)$ para d constante.

Eficiência de espaço

Desejamos agora verificar, em média, quantos nós são armazenados em uma skip list. Note que no primeiro nível temos n nós, no segundo n/d , no terceiro n/d^2 , e assim sucessivamente. Portanto, a quantidade total de nós é:

$$S = n + \frac{n}{d} + \frac{n}{d^2} + \frac{n}{d^3} + \dots = n \left(1 + \frac{1}{d} + \frac{1}{d^2} + \frac{1}{d^3} + \dots \right)$$

A expressão entre parêntesis corresponde a soma de uma PG infinita com $a_0 = 1$ e razão $q = 1/d$. A soma pode ser expressa como:

$$S = n(1 + q + q^2 + q^3 + \dots)$$

Seja a soma S' dada por:

$$S' = qS = n(q + q^2 + q^3 + \dots)$$

Então, calculando a diferença $S - S'$, temos:

$$S - S' = (1 - q)S = n$$

o que nos leva a

$$S = \frac{n}{1 - q} = \frac{n}{1 - \frac{1}{d}} = \frac{n}{\frac{d-1}{d}} = \frac{nd}{d-1} = O(n)$$

Portanto, temos eficiência de espaço linear com o tamanho da entrada, o que é muito bom! Assintoticamente equivalente a uma lista encadeada tradicional.

Relação entre tempo e espaço

Vamos comparar skip lists com diferentes fatores de dispersões d .

a) Suponha que $d' = 2$

$$\text{Tempo: } \frac{2}{2} \log_2 n = \log_2 n$$

$$\text{Espaço: } \frac{2n}{(2-1)} = 2n$$

b) Suponha que $d'' = 10$

$$\text{Tempo: } \frac{10}{2} \log_{10} n = 5 \left(\frac{\log_2 n}{\log_2 10} \right) = \frac{5}{3.32} \log_2 n \approx 1.5 \log_2 n$$

$$\text{Espaço: } \frac{10n}{(10-1)} = \frac{10}{9} n \approx 1.11 n$$

Portanto, podemos perceber que quanto maior o fator de dispersão d , mais lenta é a busca e menos espaço em memória ocupa a skip list.

Análise da probabilidade

A cada novo nível temos menos nós, mais especificamente $1/d$ do número de nós do nível anterior. Para isso, precisamos utilizar escolhas aleatórias, de modo que um nó pertença ao nível i com probabilidade $(1/d)^i$. Por questões didáticas, iremos considerar o caso em que $d = 2$.



A ideia consiste em gerar um número uniforme entre 0 e 1 e se:

- cair na região de 0 a 1/2, vai para **nível 1**
- cair na região de 1/2 a 3/4 vai para **nível 2**
- cair na região de 3/4 a 7/8, vai para **nível 3**
- cair na região de 7/8 a 15/16, vai para **nível 4**

...

Assim, as probabilidades de um valor cair em cada subintervalo é exatamente o limite superior menos o limite inferior do intervalo.

Probabilidades

1	0.5	0.25	0.125	0.0625	...
---	-----	------	-------	--------	-----

Níveis

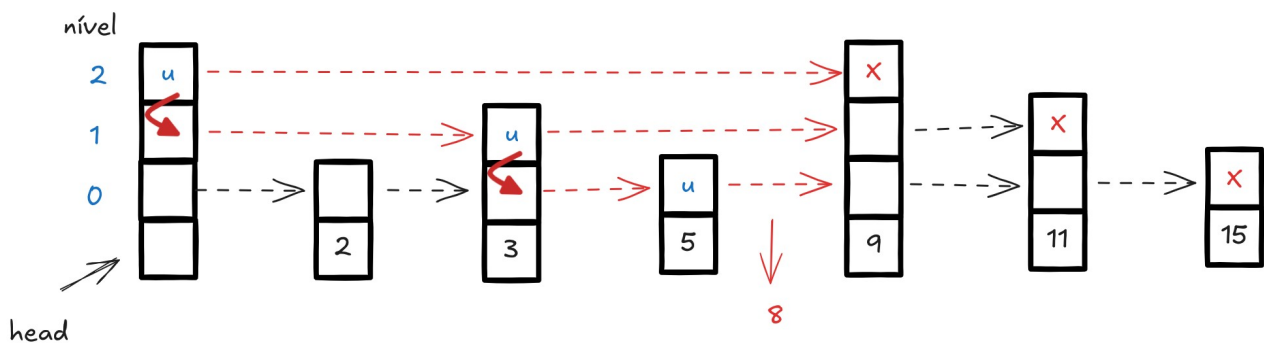
0 1 2 3 4 5 ...

Inserção em Skip Lists

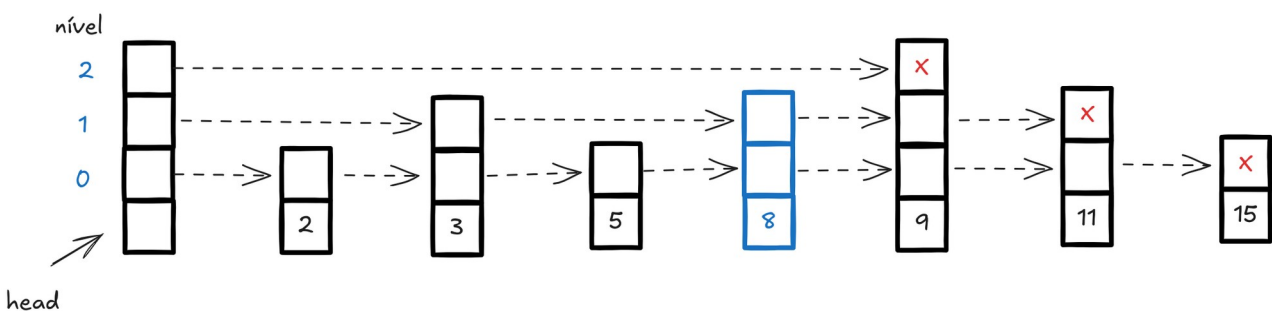
A ideia consiste em sortear um nível aleatório para o novo nó e encontrar a posição correta do nó no conjunto, o que é muito similar com o processo de busca visto anteriormente. A função a seguir ilustra o algoritmo para sortear um nível aleatório para um novo nó.

```
Random_Level(q) {      # o parâmetro q = 1/d deve ser definido previamente
    level = 1
    # É usual definir um nível máximo para evitar níveis vazios
    while random() < q and level < maxlevel
        level = level + 1
    return level
}
```

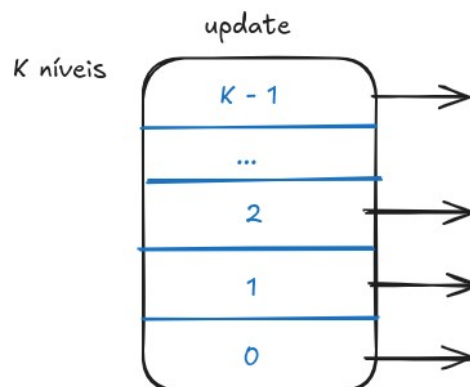
Note que se temos $q=0.5$, o processo é equivalente a: prosseguir jogando uma moeda e enquanto for saindo cara, continua subindo de nível até atingir o nível máximo. A figura a seguir ilustra o processo de inserção da chave 8 em uma Skip List.



Ao encontrar a posição do novo nó, devemos ajustar as referências denotadas por u (update) para todos os níveis.



A variável update deve ser um array de K referências, uma para cada nível da Skip List.



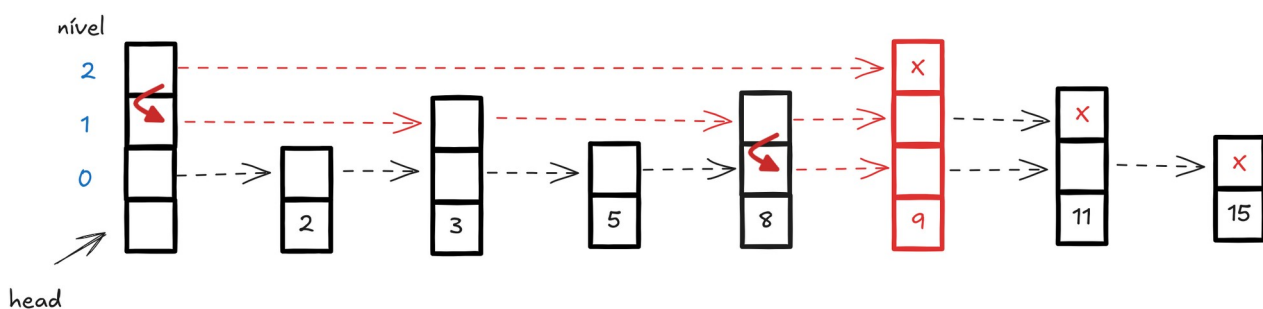
A função a seguir descreve o algoritmo para a inserção de um nó em uma Skip List. O algoritmo funciona da seguinte maneira: se searchKey não existe, ela é inserida em sua posição correta. Se ela já existe, atualize-a para o valor newKey

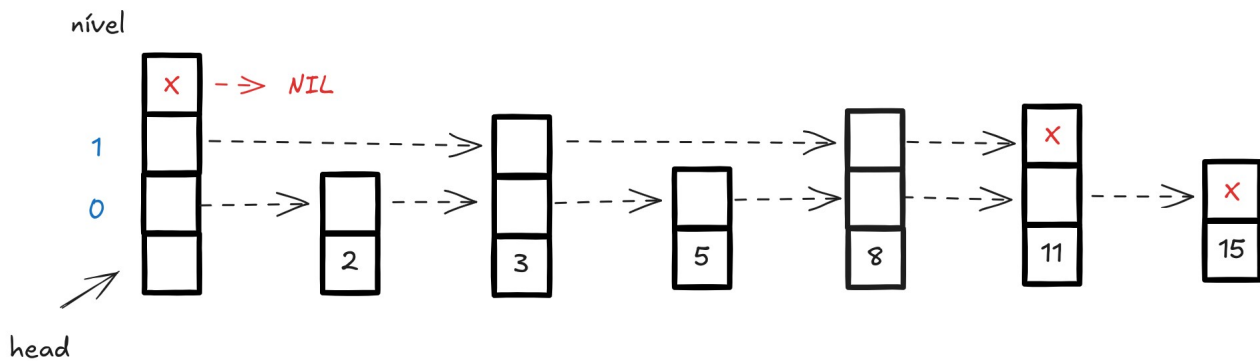
```
SkipList_Insert(L, searchKey, newKey) {
    Let update[0..K-1] be an array of references
    x = L.head
    # Encontra a posição correta do nó
    for i = L.level downto 0 {
        while x.forward[i].key < searchKey
            x = x.forward[i]
        update[i] = x
    }
    # Aqui, x.forward[0] aponta para o local da inserção!
    x = x.forward[0]
    # Se chave searchKey já existe, atualiza para newKey
    if x.key == searchKey
        x.key = newKey
    else {
        # se chave searchKey não existe, insere
        level = Random_Level(q) # sorteia número aleatório
        if level > L.level { # se nível sorteado é maior que atual
            for i = L.level+1 to level
                update[i] = L.head
            L.level = level
        }
        x = Make_Node(level, searchKey)
        for i = 0 to level {
            x.forward[i] = update[i].forward[i]
            update[i].forward[i] = x
        }
    }
}
```

Como Skip Lists são estruturas de dados aleatorizadas, a inserção de um conjunto fixo de n chaves nunca será da mesma forma, ou seja, a distribuição das chaves pelos níveis muda a cada execução.

Remoção em Skip Lists

Ao remover um nó da Skip List, além de acertar as referências, devemos nos atentar se o nó removido é o último do nível k. Caso seja, devemos deletar todo o nível após sua remoção. Isso ocorre por exemplo se formos remover a chave 9 da Skip List a seguir.





Note que após a sua remoção, o último nível da Skip List é uma lista vazia. Sendo assim, ela deve ser removida, de forma que o número de níveis total é diminuído em uma unidade. A função a seguir descreve o algoritmo para a remoção de um nó em uma Skip List.

```
SkipList_Delete(L, searchKey) {
    Let update[0..K-1] be an array of references
    x = L.head
    for i = L.level downto 0 {
        while x.forward[i].key < searchKey
            x = x.forward[i]
        update[i] = x
    }
    # Aqui, x.forward[0] aponta para o nó a ser removido!
    x = x.forward[0]
    if x.key == searchKey
        for i = 0 to L.level {
            if update[i].forward[i] != x    # se passa por cima do nó
                break
            update[i].forward[i] = x.forward[i]
        }
        free(x)    # desaloca o nó x
        # todo nível que estiver vazio, desconsidera
        while L.level > 0 and L.header.forward[L.level] == NIL
            L.level = L.level - 1
    }
}
```

Análise da complexidade

Tanto a inserção quanto a remoção de nós em Skip Lists realizam uma busca para encontrar a posição correta das referências a serem ajustadas, seguidas de modificações nas referências, o que leva tempo constante (pois é proporcional ao número de níveis da Skip List).

$$O(\log_d n) + O(k)$$

mas como $k \ll n$, o termo dominante é $O(\log_d n)$.

Ná prática, costuma-se observar que a inserção e a remoção em skip lists é mais rápida que a inserção e a remoção em árvores AVL, em especial a remoção, em que devemos encontrar o sucessor do nó a ser removido. Além disso, uma vantagem da skip list é armazenar as chaves ordenadas, de modo que encontrar o menor/maior elemento pode ser feito em tempo constante.

Com base em testes empíricos, observou-se que Skip List é geralmente pode ser mais rápida que árvores AVL para algumas operações, às vezes de forma significativa. Skip List é uma excelente escolha ao trabalhar com chaves já ordenadas (em ordem crescente ou decrescente). Porém, para chaves com ordem aleatória, a Skip List mostra-se muito eficiente na contagem de elementos menores que um determinado valor e na exclusão de elementos. Para chaves não ordenadas com operações de inserção e remoção, árvore AVL é preferida se a operação primária for a busca. Por outro lado, Skip List é muito mais eficiente quando há muitas exclusões e remoções na estrutura. Levando em consideração que implementar uma Skip List em uma linguagem de programação é mais fácil e rápido, pode-se dizer que Skip List supera árvore AVL em várias situações. As operações de rotação e balanceamento são bastante caras e desafiadoras de implementar em comparação com a implementação de uma Skip List, que é muito mais simples. Concluindo, implementar uma Skip List é mais simples do que uma implementar uma árvore AVL.

Aplicações

Skip Lists são muito utilizadas em sistemas distribuídos para implementar filas de prioridades concorrentes altamente escaláveis. Também são muito utilizadas em programação concorrente e paralela.

“The most frequently used implementation of a binary search tree is a red-black tree. The concurrent problems come in when the tree is modified it often needs to rebalance. The rebalance operation can affect large portions of the tree, which would require a mutex lock on many of the tree nodes. Inserting a node into a skip list is far more localized, only nodes directly linked to the affected node need to be locked.”

Em sistemas operacionais, Skip Lists tem sido empregadas no kernel do Linux como alternativa a árvores AVL e rubro-negras. Skip Lists também são empregadas em motores de busca (search engines), como por exemplo o Apache Lucene (open source).

"If an egg is broken by outside force, Life ends. If broken by inside force, Life begins. Great things always begin from inside."
-- Jim Kwik

Estruturas de Dados: Tabelas de Espalhamento (Hash Tables)

Tabelas de Espalhamento, ou Hash Tables, são estruturas de dados otimizadas para operações de busca, inserção e remoção. Trata-se de uma estrutura de dados muito eficiente para a implementação de dicionários.

Pode-se mostrar que, sob um certas condições, a complexidade da busca por um elemento em uma Hash Table é $O(1)$ no caso médio.

A ideia básica de uma Hash Table generaliza a ideia de um array tradicional. Como sabemos, arrays são estruturas estáticas que permitem endereçamento direto, o que permite o acesso a qualquer elemento do conjunto com complexidade $O(1)$.

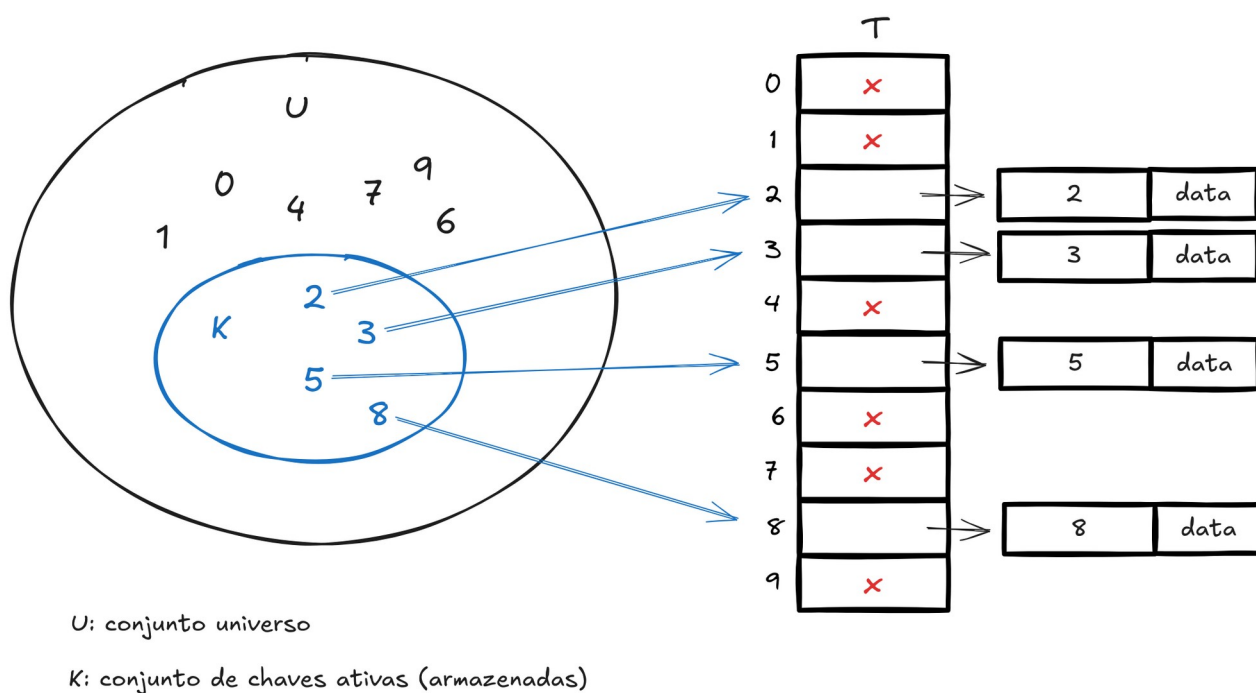
A vantagem das Hash Tables é que quando o número de chaves utilizado é relativamente pequeno em relação ao número total de possíveis chaves, essas estruturas de dados não utilizam a chave propriamente dita como índice para o acesso direto, mas sim uma **função hashing**.

Tabelas de acesso direto

O acesso direto é uma técnica que funciona bem quando o universo U de chaves é razoavelmente pequeno. Suponha uma aplicação que necessita de um conjunto dinâmico em que cada elemento possui uma chave retirada de um conjunto universo $U = \{0, 1, \dots, m-1\}$, onde m não é tão grande.

Dois elementos distintos do conjunto não podem ter a mesma chave. Para representar esse conjunto dinâmico, utilizamos uma **Tabela de Acesso Direto** (TAD), denotada por $T[0..m-1]$, na qual cada posição ou slot, corresponde a uma chave no conjunto U .

Cada chave k é mapeada diretamente para o slot k da tabela de acesso direto. Se não há elemento com chave k , então $T[k] = \text{NIL}$.



As operações de busca, inserção e remoção são todas $O(1)$.

```
TAD-Search(T, k)
    return T[k]
```

```
TAD-Insert(T, x)
    T[x.key] = x
```

```
TAD-Delete(T, x)
    T[x.key] = NIL
```

Mas então qual é o problema?

O problema com Tabelas de Acesso Direto é simples: se o conjunto universo é muito grande, armazenar a tabela T de tamanho $|U|$ na memória do computador pode ser inviável, ou até mesmo impossível!

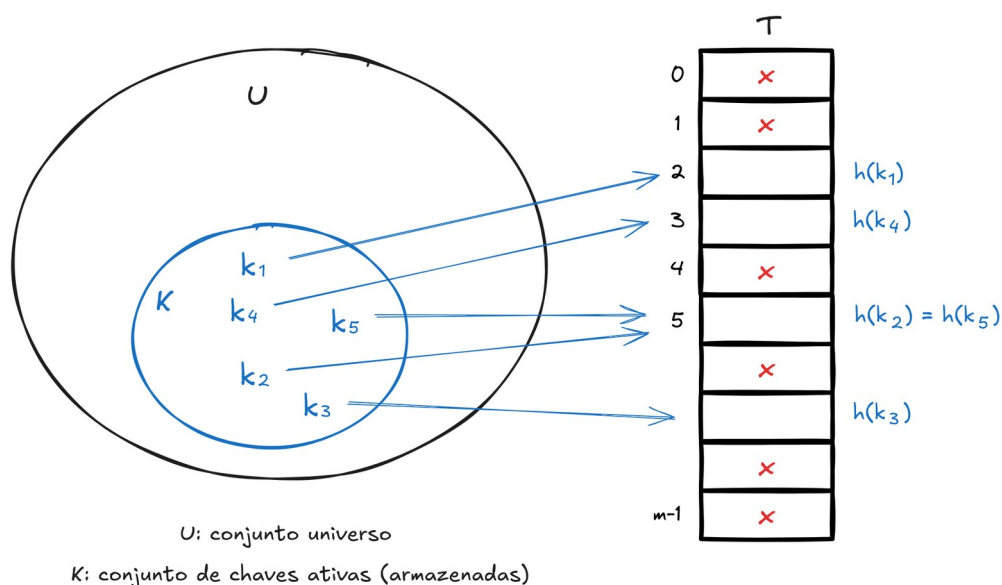
Além disso, nesse caso, o conjunto K de chaves ativas, ou seja, utilizadas de fato para o endereçamento, pode ser tão pequeno em relação ao conjunto U que a grande maioria do espaço de memória alocado para T seria um enorme desperdício.

Tabelas de Espalhamento (Hash Tables)

Quando o conjunto de chaves ativas K é muito menor que o conjunto universo de todas as possíveis chaves U , Tabelas de Espalhamento requerem muito menos espaço de armazenamento do que Tabelas de Acesso Direto, reduzindo a complexidade de espaço de $O(|U|)$ para $O(|K|)$, mantendo a complexidade da busca em $O(1)$.

Nas Tabelas de Acesso Direto o elemento com a chave k é mapeada para o slot k , enquanto que nas Tabelas de Espalhamento o elemento com a chave k é mapeada para o slot $h(k)$, onde $h(.)$ é a função hash que mapeia os elementos do conjunto universo U nos possíveis slots da Tabela de Espalhamento $T[0..m-1]$, ou seja, $h: U \rightarrow \{0, 1, \dots, m-1\}$, sendo que o tamanho m da Tabela de Espalhamento é muito menor que o tamanho do conjunto universo U .

Nesse contexto, dizemos que $h(k)$ é o valor de hash da chave k . A grande vantagem é que ao contrário das Tabelas de Acesso direto, que possuem o mesmo tamanho do conjunto universo, as Tabelas de Espalhamento possuem tamanho m , o que é bem menor que $|U|$.



Problema: Podem ocorrer colisões! Duas chaves distintas mapeadas para o mesmo slot!

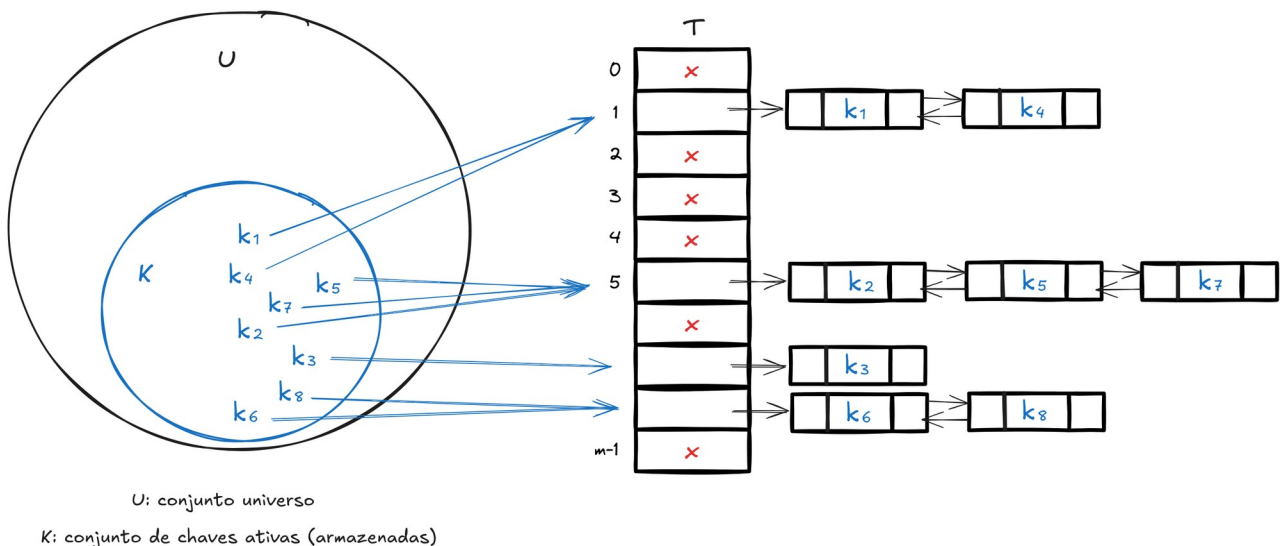
A ideia é fazer a função hash h parecer o mais “aleatória” possível, de modo que o espalhamento das chaves nos slots da tabela seja quase uniforme.

Mas como $|U| > m$, no entanto, deve haver pelo menos duas chaves que tenham o mesmo hash valor; evitar colisões por completo é, portanto, impossível. Assim, ainda precisamos de um método para resolver as colisões que ocorrem.

Tratamento de colisões por encadeamentos lógicos

A estratégia mais comum para o tratamento de colisões consiste em criar um encadeamento lógico envolvendo todos os elementos que possuam o mesmo valor hash.

Em outras palavras, devemos criar uma lista encadeada com todos os elementos cujos valores hash sejam iguais. Em resumo, o slot k contém uma referência para a cabeça de uma lista encadeada que armazena todos os elementos com valor hash $h(k)$. Se não há elemento com valor hash $h(k)$, o slot k aponta para NIL.



Assim, pode-se definir as seguintes primitivas básicas:

Insira x na cabeça da lista encadeada em $T[h(x.key)]$

```
TH-Insert( $T, x$ ) {  
    Insert_Head( $T[h(x.key)], x$ )  
}
```

Busque por um elemento com chave k na lista encadeada $T[h(k)]$

```
TH-Search( $T, k$ ) {  
    Search( $T[h(k)], k$ )  
}
```

Remova x da lista encadeada $T[h(x.key)]$

```
TH-Delete( $T, x$ ) {  
    Delete( $T[h(x.key)], x$ )  
}
```

Note que em termos de complexidade temos os seguintes fatos:

- A inserção de uma nova chave tem complexidade $O(1)$ (cabeça da lista).
- A busca por uma chave k depende do tamanho da lista $T[h(k)]$ (analisaremos mais adiante).
- A remoção de uma chave, após ela ser localizada, também pode ser realizada em $O(1)$, uma vez que podemos implementar listas duplamente encadeadas, o que faz com que o reajuste das referências não necessite de uma referência auxiliar (a primitiva TH-Delete assume que já temos uma referência para o nó a ser removido).

Análise da complexidade da busca

A seguir, iremos analisar a operação de busca, que é a mais complexa.

Seja T uma tabela de Espalhamento com m slots que armazena um total de n chaves.

Define-se o fator de carga de T como $\alpha = n/m$, que a média de elementos armazenados por uma lista encadeada.

As análises de complexidade serão feitas em termos de α que pode ser menor, igual ou maior que 1. O pior caso é a situação em que todos os n elementos possuem o mesmo valor hash, ou seja, são armazenados na mesma lista encadeada, o que nos leva claramente a uma complexidade $O(n)$ para a busca.

A complexidade da busca depende de quão bem a função hash consegue espalhar as chaves pelos m slots de T . Iremos assumir a hipótese de *hashing simples uniforme* (HSU):

$$\forall k \in U \left(\forall m \in T \left(p(k \in m) = \frac{1}{m} \right) \right)$$

ou seja, a probabilidade de uma chave cair em um slot é sempre igual a $1/m$ (mesma chance de cair em qualquer um dos m slots).

Seja n_k o comprimento da lista encadeada $T[h(k)]$, para $k = 0, 1, \dots, m-1$. Então, $n = n_0 + n_1 + \dots + n_{m-1}$. Note que o valor esperado de n_k é justamente $E[n_k] = \alpha = n/m$.

O cálculo da função hash pode ser feito em $O(1)$, uma vez que ela requer operações matemáticas simples. Além disso, o tempo requerido para buscar um elemento com chave k depende linearmente do tamanho da lista encadeada $T[h(k)]$, ou seja, de n_k .

Devemos considerar 2 casos distintos:

- 1) quando a busca não encontra o elemento com chave k ($k \notin T$); e
- 2) a busca termina com sucesso, encontrando o elemento k em uma das listas encadeadas ($k \in T$)

Teorema: Em uma T.H. de encadeamento lógico, a busca por uma chave $k \notin T$ tem complexidade $O(1 + \alpha)$ (constante), no caso médio.

Prova:

1. Sob a hipótese de hashing simples uniforme, qualquer chave k que não está armazenada na estrutura tem probabilidade igual de cair em qualquer um dos m slots.

2. O tempo esperado para a busca sem sucesso de uma chave arbitrária k é o tempo esperado de se buscar um elemento até o fim da lista $T[h(k)]$, cujo tamanho esperado é exatamente $E[n_k] = \alpha$

3. Portanto, temos uma complexidade total $O(1 + \alpha)$ (pois 1 refere-se ao custo da função hash).

Note que, quanto maior o fator de carga de T , maior o custo da busca (porém, é linear no fator de carga, o que ainda é bom).

A situação para uma busca que termina com sucesso é ligeiramente diferente, uma vez que cada lista não tem a mesma probabilidade de ser buscada (o elemento pertence a uma das listas). A probabilidade de que uma lista encadeada seja pesquisada é proporcional ao número de elementos que ela contém (quanto maior, mais provável)

Teorema: Em uma T.H. de encadeamento lógico, a busca por uma chave $k \in T$ tem complexidade $O(1 + \alpha)$ (constante), no caso médio.

Prova:

1. Iniciamos assumindo que o valor a ser buscado tem probabilidade igual de ser qualquer um dos n elementos armazenados na estrutura, ou seja, $1/n$.

2. O número de elementos examinados durante a busca com sucesso por uma chave k é 1 (cálculo da função hash é $O(1)$) mais o número de elementos que aparecem antes de k na sua lista encadeada $T[h(k)]$.

3. Como novos elementos são adicionados no início da lista, as chaves que vem antes de k em $T[h(k)]$, foram adicionadas depois que k foi inserida.

4. Para encontrar o número esperado de elementos examinados, nós tomamos a média sobre as n chaves na Tabela de Espalhamento, de 1 mais o número esperado de elementos adicionados à lista $T[h(k)]$ depois que k foi adicionada na lista.

5. Note que a probabilidade de uma chave ser adicionada a lista de k é $1/m$ (pela hipótese de hashing simples uniforme), o que nos leva ao seguinte número médio de operações:

$$T(n) = \frac{1}{n} \sum_{i=1}^n \left(1 + \sum_{j=i+1}^n \frac{1}{m} \right) = \frac{1}{n} \sum_{i=1}^n 1 + \frac{1}{nm} \sum_{i=1}^n \sum_{j=i+1}^n 1 = \frac{n}{n} + \frac{1}{nm} \sum_{i=1}^n (n-i) = 1 + \frac{1}{nm} \sum_{i=1}^n n - \frac{1}{nm} \sum_{i=1}^n i$$

É fácil notar que o primeiro somatório é igual a n vezes n , ou seja, n^2 . Como vimos em aulas anteriores, temos que:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2}$$

o que nos leva a:

$$T(n) = 1 + \frac{n}{m} - \frac{(n+1)}{2m} = 1 + \frac{n}{m} - \frac{n}{2m} - \frac{1}{2m} = 1 + \frac{n}{m} - \frac{1}{2} \frac{n}{m} - \frac{1}{2n} \frac{n}{m}$$

Mas como sabemos que o fator de carga é $\alpha = n/m$, temos:

$$T(\alpha) = 1 + \alpha - \frac{\alpha}{2} - \frac{\alpha}{2n} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

o que é linear em α . Portanto, temos que a complexidade no caso médio da operação de busca em uma Tabela de Espalhamento é $O(1 + \alpha)$.

Mas o que essa análise nos diz em termos práticos?

Se o número de chaves na Hash Table é proporcional ao número de slots, ou seja, $n = cm$, então temos que $n = O(m)$ e por consequência $\alpha = O(m)/m = O(1)$.

Portanto, no caso médio, a busca na Hash Table pode ser realizada em tempo constante!

Funções Hash

As funções hash assumem que o universo de chaves é o conjunto dos naturais $N = \{0, 1, 2, 3, \dots\}$. Assim, se as chaves não são números naturais, devemos encontrar uma maneira de interpretá-las como números naturais. Por exemplo, com caracteres e strings, podemos utilizar o código ASCII para definir uma chave numérica (inteira).

O método da divisão

No método da divisão, a ideia consiste em mapear uma chave k para um dos m slots calculando o resto da divisão de k por m (lembre-se que o resto da divisão de k por m sempre será um número entre 0 e $m - 1$). Ou seja, a função hash é dada por:

$$h(k) = k \bmod m$$

Por exemplo, se a Tabela de Espalhamento possui tamanho $m = 12$ e temos a chave $k = 100$, então ela será mapeada para o slot 4, pois:

$$\begin{array}{r} 100 \mid 12 \\ \underline{4 8} \end{array}$$

Como a função hash requer apenas uma operação matemática ela é muito rápida de ser realizada, com complexidade $O(1)$. É importante ressaltar que ao utilizar o método da divisão, devemos evitar alguns valores de m . Por exemplo, deve-se evitar escolher m como uma potência de 2. Note que se $m = 2^p$, a função hash torna-se:

$$h(k) = k \bmod 2^p$$

Note que toda potência de 2 quando expressa em binário, tem a forma:

$$\begin{aligned} 2^1 &= 10 \\ 2^2 &= 100 \\ 2^3 &= 1000 \\ 2^4 &= 10000 \\ 2^5 &= 100000 \\ 2^6 &= 1000000 \\ &\dots \end{aligned}$$

o que faz com que o valor da função hash seja exatamente os p bits de ordem inferiores de k . Esse padrão gera códigos que não são igualmente prováveis, ou seja, tende-se a ter uma concentração grande em alguns valores da função hash.

Números primos são excelentes escolhas! Particularmente quando são próximos de alguma potência de 2. Lembre-se que números primos são divisíveis apenas por 1 e por ele mesmo, o que garante um bom espalhamento das chaves.

O método da multiplicação

O método da multiplicação para criar funções hash opera em dois passos principais. Primeiramente, nós multiplicamos a chave k por uma constante A no intervalo $0 < A < 1$ e extraímos a parte fracionária de kA .

Então, multiplicamos esse valor por m e tomamos o piso do resultado. Em resumo, a função hash é dada por:

$$h(k) = \lfloor m(kA \bmod 1) \rfloor$$

onde operação $kA \bmod 1$ denota a parte fracionária de kA , que é dada por $kA - \lfloor kA \rfloor$.

Uma possível vantagem desse método é que a escolha do valor de m (tamanho da Tabela de Espalhamento) não é crítico. Diferentemente do método da divisão, define-se m como sendo uma potência de 2, ou seja, $m = 2^p$, para p inteiro.

Uma observação importante é que, embora esse método funcione com qualquer valor de A , ele funciona melhor com alguns valores de A do que outros. A escolha do valor ótimo de A depende das características dos dados que serão armazenados (chaves). Um valor particularmente adequado e sugerido por Knuth é

$$A = \frac{\sqrt{5}-1}{2} \approx 0.6180339887\dots$$

Hashing universal

Suponha que alguém conheça a função hash de uma Tabela de Espalhamento T . Então, essa pessoa pode, de forma maliciosa, escolher chaves de forma que todos os valores hash são iguais, fazendo com que todas sejam mapeadas para o mesmo slot, o que irá tornar a complexidade da busca $O(n)$ no caso médio. Em outras palavras, é possível “quebrar” a Tabela de Espalhamento!

Para evitar esse tipo de problema, foi proposta a estratégia de hashing universal, que basicamente consiste escolher uma função de hash aleatoriamente de modo a torná-la independente das chaves que serão armazenadas na estrutura de dados. Desse modo, tornamos a Tabela de Espalhamento mais robusta a ataques maliciosos e adversos.

Seja H uma coleção finita de funções hash que mapeiam um dado universo U de chaves no intervalo $\{0, 1, \dots, m-1\}$. Dizemos que H é universal se para cada par de chaves distintas $k, l \in U$, o número de funções hash $h \in H$ para as quais $h(k) = h(l)$ é no máximo $|H|/m$.

Em outras palavras, com uma função hash selecionada aleatoriamente de H , a probabilidade de uma colisão entre chaves distintas $k, l \in U$ não é maior que $1/m$ (o que é equivalente a hipótese de

hashing simples uniforme, ou seja, se $h(k)$ e $h(l)$ fossem escolhidos aleatoriamente do conjunto $\{0, 1, \dots, m-1\}$ a partir de uma distribuição uniforme.

A pergunta que surge é: como desenvolver uma classe universal de funções hash?

Iniciamos escolhendo um número primo p que seja grande o suficiente para que toda possível chave $k \in [0, p-1]$. Defina os conjuntos Z_p e Z_p^* como:

$$Z_p = \{0, 1, 2, \dots, p-1\}$$

$$Z_p^* = \{1, 2, \dots, p-1\}$$

Podemos definir agora uma função hash h_{ab} parametrizada para $a \in Z_p^*$ e $b \in Z_p$ utilizando uma transformação linear seguida de uma redução módulo p e outra redução módulo m :

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m$$

A família de todas as funções hash é definida por:

$$H_{pm} = \{h_{ab} : a \in Z_p^* \wedge b \in Z_p\}$$

Cada uma das funções hash dessa família mapeia uma chave $k \in Z_p$ para um slot $s \in Z_m$. Essa classe de funções hash tem a desejável propriedade de que o tamanho m da Tabela de Espalhamento pode ser escolhido arbitrariamente, não precisa necessariamente ser primo.

Note que como temos $(p-1)$ escolhas para a e p escolhas para b , a família de funções hash H_{pm} contém $p(p-1)$ funções hash, o que representa um número elevado, uma vez que p deve ser um número primo grande o suficiente.

Mas como o hashing universal funciona na prática?

Se a cada inserção de nova chave utilizarmos valores distintos de a e b , como iremos saber os valores no momento de efetuar uma busca? Seria totalmente caótico!

A ideia consiste em escolher uma função hash h_{ab} arbitrária e seguir utilizando essa função até que, por algum critério objetivo, seja detectado que o espalhamento das chaves não está bom o suficiente (chaves concentram-se em poucos slots). A partir desse momento, deve-se escolher outra função hash h'_{ab} arbitrariamente, realizar o rehash de todas as chaves já armazenadas na Tabela Hash (reespalhamento) e então utilizar h'_{ab} para futuras consultas.

Endereçamento aberto (open addressing)

No endereçamento aberto, todas as chaves são armazenadas diretamente na Tabela de Espalhamento, sendo que não é permitido ter múltiplas chaves mapeadas para um mesmo slot (cada chave deve ter um slot específico). Dessa forma, a Tabela de Espalhamento pode “encher-se de chaves” de modo que não seja possível fazer mais inserções. Uma consequência disso é que o fator de carga α nunca pode exceder 1, ou seja, temos $\alpha \leq 1$. Os algoritmos a seguir ilustram como são os processos de inserção e busca em Tabelas de Espalhamento de endereçamento aberto.

A ideia básica da inserção de uma chave consiste em parametrizar a função hash utilizando uma variável contadora i , de modo que na primeira tentativa atingimos um slot k , se ele estiver ocupado,

na segunda tentativa atingimos um outro slot k' , se ele estiver ocupado, na terceira tentativa atingimos um novo slot k'' , e assim sucessivamente.

```

Hash_Insert(T, k) {
    i = 0
    repeat {
        q = h(k, i)
        if T[q] == NIL {
            T[q] = k
            return q
        }
        else
            i = i + 1
    } until i == m
    error "overflow"
}

Hash_Search(T, k) {
    i = 0
    repeat {
        q = h(k, i)
        if T[q] == k
            return q
        i = i + 1
    } until T[q] == NIL or i == m
    return NIL
}

```

A questão primordial aqui é como definir a função hash $h(k, i)$. Veremos a seguir 3 métodos distintos: linear probing, quadratic probing e double hashing.

Linear probing (sondagem linear)

A ideia dessa abordagem consiste em utilizar uma função hash auxiliar $h': U \rightarrow \{0, 1, \dots, m-1\}$ na definição da função hash $h(k, i)$:

$$h(k, i) = (h'(k) + i) \bmod m$$

para $i = 0, 1, \dots, m-1$. Se uma colisão ocorre em $h(k, 0)$, então verificamos $h(k, 1)$ e assim sucessivamente. Dessa forma, sempre que uma posição está ocupada, devemos tentar a posição imediatamente posterior.

Um problema com a sondagem linear é que ela causa um efeito chamado de primary clustering (agrupamento primário), que é a formação de longas sequências de chaves, o que prejudica a inserção de novas chaves em T (aumenta o tempo gasto para percorrer toda sequência).

Quadratic probing (sondagem quadrática)

A ideia é similar a sondagem linear, mas com a utilização de incrementos não lineares em $h(k, i)$

$$h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$$

onde $h'(k)$ é uma função hash auxiliar, c_1 e c_2 são duas constantes auxiliares e $i = 0, 1, \dots, m-1$. A posição inicial a ser sondada é $T[h'(k)]$ e caso ela esteja ocupada as próximas posições a serem sondadas terão offsets (deslocamentos) que dependem de forma quadrática do parâmetro i . Ou seja, ela tende a ser melhor que a sondagem linear para espalhar mais as chaves ao longo da estrutura. Porém, assim como a sondagem linear, se duas chaves tiverem a mesma sonda inicial posição, então suas sequências de sonda são as mesmas, uma vez que $h(k_1, 0) = h(k_2, 0)$ implica $h(k_1, i) = h(k_2, i)$ para todo i . Essa propriedade leva a uma forma mais branda de agrupamento, chamada secondary clustering (agrupamento secundário).

Double hashing (espalhamento duplo)

O espalhamento duplo oferece uma das melhores estratégias para o endereçamento aberto em Hash Tables porque as permutações produzidas por ele possuem muitas propriedades de permutações escolhidas aleatoriamente e conseguem atenuar os efeitos de agrupamentos primário e secundário. A função hash neste caso é definida como:

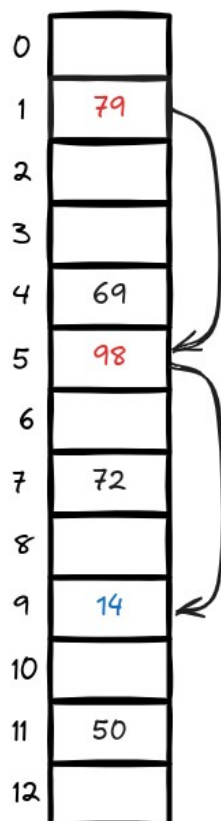
$$h(k,i) = (h_1(k) + i h_2(k)) \bmod m$$

onde temos duas funções hash auxiliares $h_1(k)$ e $h_2(k)$. A sondagem inicial é realizada na posição $T[h_1(k)]$ (pois $i=0$). As próximas posições de sondagem são deslocamentos da posição anterior de tamanho proporcional ao resto de $h_2(k)$ por m . Ou seja, diferentemente das sondagens linear e quadrática, aqui a posição inicial e os deslocamentos (offsets) são variáveis, o que contribui e muito para evitar colisões. É interessante que o valor de $h_2(k)$ seja relativamente primo com o valor do tamanho da Tabela de Espalhamento, m (não tenham ou tenham poucos divisores em comum). Uma forma simples de garantir isso é fazer m igual a uma potência de 2 e definir a função $h_2(k)$ de modo que ela sempre retorne um número ímpar. Outra alternativa consiste em fixar m primo e definir $h_2(k)$ de modo que ela sempre retorne um inteiro positivo menor que m . Por exemplo:

$$h_1(k) = k \bmod m$$

$$h_2(k) = 1 + (k \bmod m')$$

com m' sendo um pouco menor que m , como $m' = m - 1$. A figura a seguir ilustra um exemplo de espalhamento duplo.



Double Hashing

chave = 14

$m = 13$

$h_1(k) = k \bmod 13$

$h_2(k) = 1 + (k \bmod 11)$

Aqui, temos:

$$m=13$$

$$h_1(k)=k \bmod 13$$

$$h_2(k)=1+(k \bmod 11) \text{ .}$$

Desejamos inserir a chave 14. No $i=0$, como $14 \bmod 13 = 1$, tentamos na posição 1, que está ocupada. Sendo assim, no $i=1$, o próximo slot a ser examinado será $1 + 14 \bmod 11 = 1 + 3 = 4$, o que resulta em $(1 + 4) \bmod 13 = 5$. Tentamos no slot 5, que novamente está ocupado. Assim, em $i=2$, temos $(1 + 2*4) \bmod 13 = 9 \bmod 13 = 9$. A posição está livre e a inserção finaliza com sucesso.

Exercício: Considere a inserção das chaves 10, 22, 31, 4, 15, 28, 17, 88, 59 em uma Tabela de Espalhamento de tamanho $m = 11$, utilizando endereçamento aberto com uma função auxiliar de hash $h'(k) = k$. Explique o processo de inserção ilustrando com um diagrama como fica a estrutura se utilizarmos:

- a) Sondagem linear (linear probing)
- b) Sondagem quadrática (quadratic probing)

Análise da complexidade no endereçamento aberto

A seguir, veremos resultados importantes acerca do espalhamento por endereçamento aberto.

Teorema: Dada uma Tabela de Espalhamento com endereçamento aberto e um fator de carga $\alpha = n/m < 1$, o número esperado de sondagens na busca por uma chave $k \notin T$ (não pertence ao conjunto) é no máximo $1/(1 - \alpha)$, sob hipótese de hashing uniforme.

Prova:

1. Se $\alpha < 1$, existe ao menos 1 slot vazio na Tabela Hash.
2. Assim, em uma busca sem sucesso, todos os acessos com exceção do último, são feitos em slots ocupados.
3. Seja X a variável aleatória que denota o número de sondagens em slots ocupados na busca. Defina A_i , $i=1,2,\dots$, como o seguinte evento: a i -ésima sondagem ocorre e retorna um slot ocupado.
4. Então, o evento $\{X \geq i\}$ é dado por $A_1 \cap A_2 \cap \dots \cap A_{i-1}$.
5. Logo, temos:

$$P(A_1 \cap A_2 \cap \dots \cap A_{i-1}) = P(A_1)P(A_2|A_1)P(A_3|A_1 \cap A_2) \dots P(A_{i-1}|A_1 \cap A_2 \cap \dots \cap A_{i-2})$$

6. Como há n elementos e m slots, $\frac{n}{m}$.

7. Para $j > 1$, $P(A_j|A_1 \cap A_2 \cap \dots \cap A_{j-1}) = \frac{n-(j-1)}{m-(j-1)}$ (probabilidade da j -ésima sondagem ser em um slot ocupado, dado que as $j - 1$ primeiras também estavam ocupadas).

8. Então, temos:

$$P(X \geq i) = \frac{n}{m} \frac{n-1}{m-1} \frac{n-2}{m-2} \dots \frac{n-(i-2)}{m-(i-2)}$$

o que nos permite escrever a desigualdade:

$$P(X \geq i) \leq \frac{n}{m} \frac{n}{m} \frac{n}{m} \dots \frac{n}{m} = \left(\frac{n}{m}\right)^{i-1} = \alpha^{i-1}$$

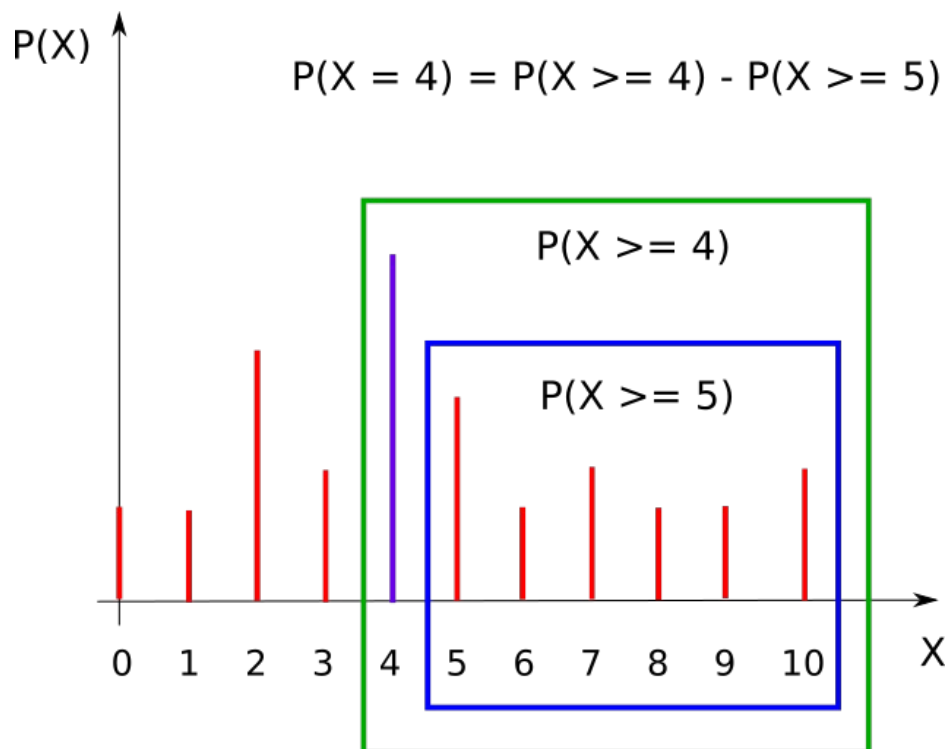
pois como $n < m$, temos $\frac{(n-j)}{(m-j)} \leq \frac{n}{m}$ para $0 \leq j \leq m$.

9. Lembre-se que o valor esperado de X é:

$$E[X] = \sum_{i=0}^{\infty} i P(X=i)$$

e como X é uma variável aleatória discreta:

$$P(X=i) = P(X \geq i) - P(X \geq i+1)$$



Então, podemos escrever o valor esperado como:

$$E[X] = \sum_{i=0}^{\infty} i [P(X \geq i) - P(X \geq i+1)]$$

Denotando $P_i = P(X \geq i)$, note que o somatório nada mais é que:

$$0(P_0 - P_1) + 1(P_1 - P_2) + 2(P_2 - P_3) + 3(P_3 - P_4) + 4(P_4 - P_5) + \dots = P_1 + P_2 + P_3 + P_4 \dots$$

ou seja:

$$E[X] = \sum_{i=1}^{\infty} P(X \geq i)$$

10. Como temos n chaves em T, há no máximo n slots ocupados, de modo que:

$$E[X] = \sum_{i=1}^{\infty} P(X \geq i) = \sum_{i=1}^n P(X \geq i) + \sum_{i \geq n+1} P(X \geq i) = \sum_{i=1}^n P(X \geq i) + 0$$

o que nos leva a:

$$E[X] = \sum_{i=1}^n \alpha^{i-1} \leq \sum_{i=1}^{\infty} \alpha^{i-1}$$

Por uma simples substituição de variáveis, seja $k = i - 1$. Então, temos que o limite inferior do somatório é $k = 0$. Dessa forma, vale a seguinte desigualdade:

$$E[X] \leq \sum_{k=0}^{\infty} \alpha^k = \alpha^0 + \alpha^1 + \alpha^2 + \dots$$

O lado direito da desigualdade é a soma de uma P. G. infinita de razão α . Logo, temos que:

$$E[X] \leq \frac{1}{1 - \alpha}$$

Se α é constante, o que significa que $m = cn$, a busca possui complexidade $O(1)$. Por exemplo, se tabela está 50% ocupada, são necessários no máximo 2 acessos, se a tabela está 75% ocupada, são necessários no máximo 4 acessos, se a tabela está 90% ocupada, são necessários no máximo 10 acessos, e assim sucessivamente (mesmo que n seja um valor alto).

Corolário: A inserção de uma chave k em uma Tabela Hash com endereçamento aberto com fator de carga $\alpha < 1$ requer no máximo $\frac{1}{1 - \alpha}$ sondagens, no caso médio.

Esse resultado decorre diretamente do teorema anterior. A ideia é que para um elemento ser inserido, primeiramente deve-se verificar se a chave não está na estrutura e inseri-la na posição subsequente, o que é justamente o resultado do teorema anterior.

Teorema: Dada uma Tabela de Espalhamento com endereçamento aberto e fator de carga $\alpha < 1$, o número esperado de sondagens em uma busca terminada com sucesso (encontra o elemento no conjunto) é no máximo

$$\frac{1}{\alpha} \log \frac{1}{1 - \alpha}$$

assumindo hashing simples uniforme.

Prova:

A busca pela chave k é equivalente a sequência de sondagens usada na inserção da chave k .

Se k é a $(i+1)$ -ésima chave a ser inserida em T , então o fator de carga nesse momento é:

$$\alpha = \frac{i}{m}$$

o que implica que número esperado de sondagens é no máximo:

$$E[X] \leq \frac{1}{1-\alpha} = \frac{1}{1-\frac{i}{m}} = \frac{1}{\frac{m-i}{m}} = \frac{m}{m-i}$$

Tomando a média sobre todas as n chaves da tabela:

$$\frac{1}{n} \sum_{i=0}^{n-1} \left(\frac{m}{m-i} \right) = \frac{m}{n} \sum_{i=0}^{n-1} \left(\frac{1}{m-i} \right) = \frac{1}{\alpha} \sum_{i=0}^{n-1} \left(\frac{1}{m-i} \right)$$

Por uma substituição de variáveis, seja $k = m - i$. Assim, temos:

$$\begin{aligned} i=0 &\rightarrow k=m \\ i=n-1 &\rightarrow k=m-(n-1) \end{aligned}$$

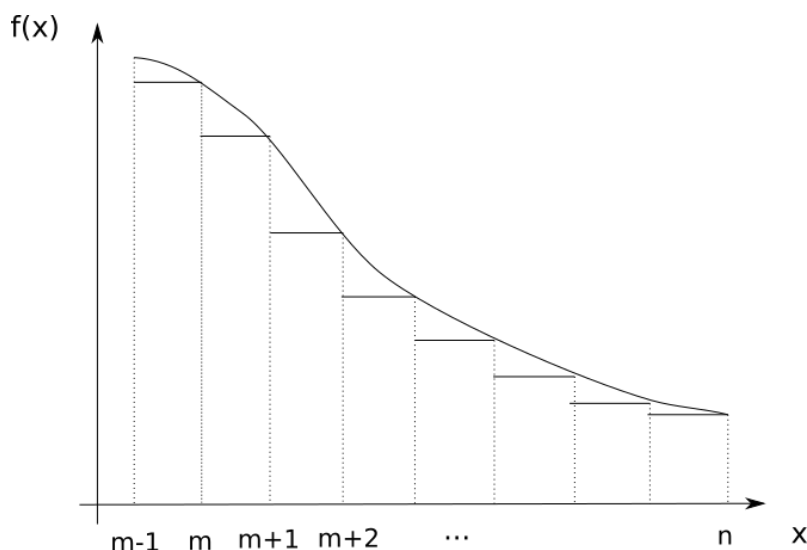
Desa forma, o somatório pode ser expresso como:

$$\frac{1}{\alpha} \sum_{k=m-n+1}^m \frac{1}{k}$$

Pode-se mostrar que para funções monotonicamente decrescentes, temos:

$$\sum_{k=m}^n f(k) \leq \int_{m-1}^n f(x) dx$$

A figura a seguir ilustra uma intuição por trás dessa desigualdade (área sob a curva é maior).



Então, podemos escrever:

$$\frac{1}{\alpha} \sum_{k=m-n+1}^m \left(\frac{1}{k} \right) \leq \frac{1}{\alpha} \int_{m-n}^m \frac{1}{x} dx = \frac{1}{\alpha} \ln x \Big|_{m-n}^m = \frac{1}{\alpha} [\ln m - \ln(m-n)] = \frac{1}{\alpha} \ln \left(\frac{m}{m-n} \right)$$

Dividindo o numerador e o denominador do logaritmo por m, finalmente chegamos em:

$$\frac{1}{\alpha} \ln \left(\frac{1}{1-\alpha} \right)$$

Esse resultado é muito bom, pois é um limite ainda menor que o obtido no caso de $k \notin T$.

Por exemplo:

- Se a Tabela Hash está 50% cheia, número esperado de sondagens é aproximadamente 1.386
- Se a Tabela Hash está 75% cheia, número esperado de sondagens é aproximadamente 1.848
- Se a Tabela Hash está 90% cheia, número esperado de sondagens é aproximadamente 2.558

Na utilização de Tabelas Hash, como podemos evitar colisões? Para responder a essa questão, iremos estudar um pouco sobre probabilidades de colisão.

Calculando probabilidades de colisões

Considere uma Tabela de Espalhamento de tamanho m com n chaves.

Pergunta-se: Qual a probabilidade de que ocorra pelo menos uma colisão no conjunto de n chaves?

Para responder a essa pergunta, precisamos esclarecer um equívoco muito comum: de que colisões em uma tabela hash só acontecem quando ela está quase cheia. Por exemplo, alguns podem acreditar que se a tabela hash estiver 25% cheia, as colisões ocorreriam com uma probabilidade de 25%. No entanto, essa suposição é incorreta!

O paradoxo do aniversário

Suponha um grupo de pessoas. Qual é o número mínimo de indivíduos necessários neste grupo para que haja mais de 50% de probabilidade de duas pessoas fazerem aniversário no mesmo dia? Sabemos que deve haver $365+1=366$ pessoas para ter uma probabilidade de 100%. Então, somos levados a pensar que devem ter ao menos 183 pessoas no grupo para termos uma probabilidade maior que 50%, certo? Incorreto!

Surpreendentemente, 23 pessoas em um grupo são o suficiente para ter uma probabilidade maior que 50% de que duas pessoas compartilhem o mesmo aniversário! É difícil de acreditar? Vamos ver o porque disso.

A probabilidade de você compartilhar o mesmo aniversário com alguém é $1/365$, e a probabilidade de ter aniversários diferentes é $364/365$. No entanto, quando você considera todos os pares possíveis, a probabilidade cresce muito mais rápido. Lembre-se, não se trata de encontrar alguém que tenha o mesmo aniversário que você, mas sim de encontrar quaisquer duas pessoas no grupo que compartilhem o mesmo aniversário. Em um grupo de 23 pessoas, o número de pares possíveis é $(23 \times 22 / 2) = 253$.

A probabilidade de duas pessoas não terem o mesmo aniversário é $364/365$. Temos 253 pares, então a probabilidade de nenhum dos pares compartilhar o mesmo aniversário é exatamente $(364/365)^{253}$. Consequentemente, a probabilidade de pelo menos um par compartilhar o mesmo aniversário é:

$$1 - \left(\frac{364}{365}\right)^{253} \approx 50.05\%.$$

Vamos explorar como o paradoxo de aniversário funciona com tabelas hash e qual é a probabilidade de colisões em uma tabela hash. Será que a probabilidade de colisão de 25% ocorre quando a tabela hash está 25% preenchida? Vamos descobrir.

Seja m é o número de posições na tabela hash e n é o número de chaves a serem inseridas. Então a probabilidade de que haja pelo menos uma colisão entre n chaves inseridas aleatoriamente é

$$P_{m,n}(\text{colisão}) = 1 - P_{m,n}(\text{não colisão})$$

$$P_{m,n}(\text{colisão}) = 1 - P_{m,n}(\text{sem colisão na primeira chave}) \times \dots \times P_{m,n}(\text{sem colisão na } n\text{ésima chave})$$

$$P_{m,n}(\text{colisão}) = 1 - 1 \times \frac{m-1}{m} \times \frac{m-2}{m} \times \frac{m-3}{m} \times \dots \times \frac{m-n+1}{m} = \frac{m!}{(m-n)!m^n}$$

Usando essa equação, se tomarmos $m = 365$ e $n = 23$, obtemos a probabilidade de colisões como 50,07%, que é a probabilidade de duas pessoas compartilharem o mesmo aniversário em um grupo de 23 pessoas. E se você tomar $n = 57$, a probabilidade é 99,01%, o que significa que é quase certo que duas pessoas podem compartilhar o mesmo aniversário em um grupo de 57 pessoas.

Agora, considere uma tabela hash com 1000 localizações, ou seja, $m = 1000$. Vamos examinar a probabilidade de colisões se a tabela estiver apenas 25% preenchida. ou seja, $n = 250$. A probabilidade de colisões é incrivelmente de 99,9999999999984%! Então é quase certo que haverá uma colisão. Isso indica que uma colisão é certa quando a tabela está apenas 25% preenchida.

Calculando alguns valores de probabilidade para diferentes valores de n , temos:

$$n = 38 \rightarrow P = 50.93\%$$

$$n = 50 \rightarrow P = 71.22\%$$

$$n = 75 \rightarrow P = 94.19\%$$

$$n = 100 \rightarrow P = 99.40\%$$

$$n = 125 \rightarrow P = 99.97\%$$

Como podemos ver, 38 chaves em uma tabela hash de tamanho 1000 produz uma probabilidade de colisão maior que 50%. Com uma ocupação de 125 chaves, o que equivale a 25% de ocupação, nos leva a uma probabilidade de colisão de 99.97%.

Número total de colisões

Inserir uma chave em uma tabela hash é como escolher um item aleatório do conjunto de n itens e jogá-lo em um conjunto de m caixas. Não podemos prever de antemão em qual caixa o item vai parar. É possível que o item que jogamos acabe em uma caixa que já tenha um item (o que significa uma colisão).

A primeira chave que inserimos acabou em alguma caixa X . A probabilidade de que a segunda chave acabe na mesma caixa X que o primeiro item é $1/m$. Então a probabilidade de que os itens restantes $(n - 1)$ acabem na mesma caixa X que o primeiro item é $(n - 1)/m$.

Suponha que não haja colisões na primeira caixa onde a primeira chave foi inserida. A segunda chave que inserimos acabou em alguma outra caixa Y. A probabilidade de que a terceira chave acabe na mesma caixa Y é $1/m$. Então a probabilidade de que as chaves restantes $(n - 2)$ acabem na mesma caixa Y é $(n - 2)/m$.

Assim, o número médio de colisões é dado por:

$$N_m = \sum_{i=1}^{n-1} \frac{i}{m} = \frac{1}{m} \sum_{i=1}^{n-1} i = \frac{n(n-1)}{2m}$$

Vamos ver o número total médio de colisões possíveis quando $m = 1000$, e o número de itens a serem inseridos é n .

$n = 100 \rightarrow \# \text{ colisões} \approx 5$
 $n = 250 \rightarrow \# \text{ colisões} \approx 31$
 $n = 500 \rightarrow \# \text{ colisões} \approx 125$
 $n = 750 \rightarrow \# \text{ colisões} \approx 281$
 $n = 1000 \rightarrow \# \text{ colisões} \approx 500$

Então, se tivermos uma tabela hash com m slots e se o número total de itens a serem inseridos n for igual ao número total de slots m em uma tabela hash, então haverá aproximadamente $m/2$ colisões totais, o que não é um número pequeno! Então, a melhor maneira de reduzir colisões é tornar a tabela hash maior (pelo menos 1,3 vezes o número de itens a serem inseridos).

Sendo assim, precisamos pensar em formas mais eficientes de garantir de não haja colisões. Uma delas é o hashing perfeito, que será apresentado a seguir.

Perfect hashing (Espalhamento perfeito)

Vimos que as Tabelas de Espalhamento funcionam muito bem no caso médio. Porém, as técnicas anteriores não são as melhores quando analisamos o pior caso.

Há uma situação em que é possível atingir complexidade $O(1)$ para busca em Tabelas de Espalhamento mesmo no pior caso: quando o conjunto de chaves é estático (as chaves são fixas). Por exemplo, algumas aplicações em que o conjunto de chaves é estático (não muda) são: no desenvolvimento de um compilador, as palavras reservadas de uma linguagem de programação são estáticas, no processamento de dados, o conjunto de nomes de arquivos armazenados em um CD-ROM também não se altera. Na prática isso significa que, uma vez criada a Tabela de Espalhamento, não serão necessárias inserções nem remoção de chaves. O esquema em questão chama-se *Perfect Hashing*.

A ideia do *Perfect Hashing* é criar uma estratégia de hashing universal em dois níveis:

1. O primeiro nível é essencialmente um esquema de hashing com encadeamento, em que temos que espalhar n chaves em m slots usando uma função de hash h .
2. Ao invés de criar uma lista encadeada das chaves que são mapeadas para um mesmo slot j , a ideia consiste usar uma segunda, porém menor, Tabela de Espalhamento S_j , com uma outra função hash associada. Através de uma escolha adequada das funções hash, pode-se garantir que não haverá colisões no segundo nível. A única restrição é que o tamanho da Tabela de Espalhamento S_j , denotado por m_j , deve ser igual ao quadrado do número de elementos armazenados em S_j .

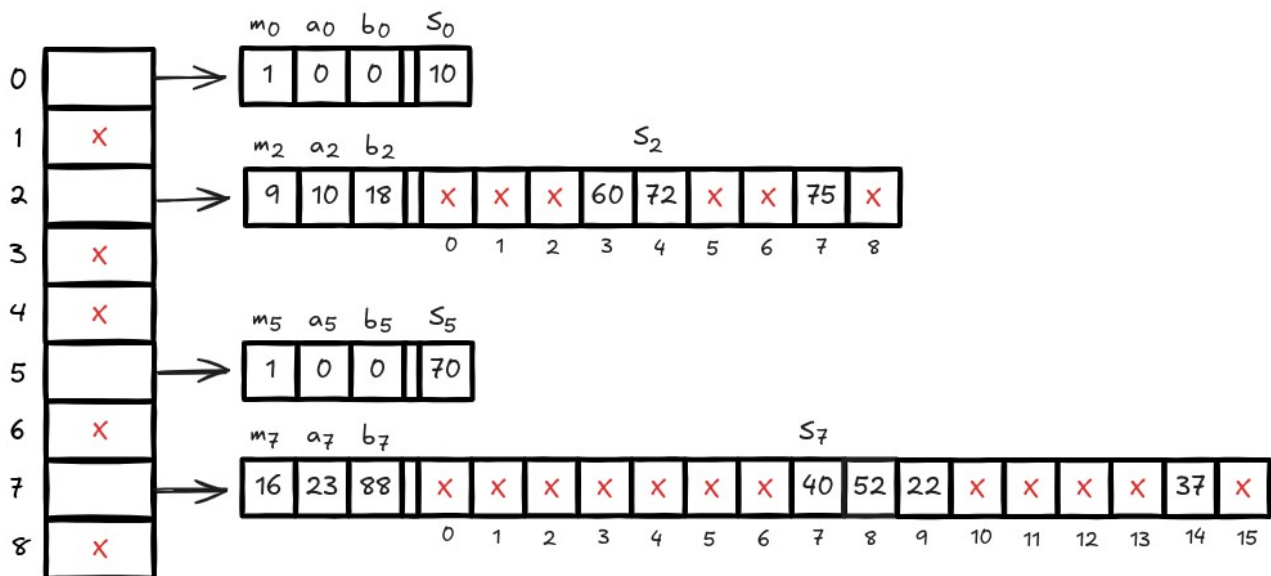
Considere o seguinte exemplo: desejamos utilizar o perfect hashing para armazenar o conjunto estático de chaves a seguir: $K = \{10, 22, 37, 40, 52, 60, 70, 72, 75\}$. A função de hash externa (global) é dada por:

$$h(k) = ((ak + b) \bmod p) \bmod m$$

com $a=3$, $b=42$, $p=101$ e $m=9$. Como $h(75) = 2$, a chave 75 deve ser armazenada no slot 2 da tabela T. Uma Tabela de Espalhamento secundária S_j , armazena as chaves mapeadas para o slot j . Sendo assim, 75 é armazenado em S_2 . Mas onde? Aqui entra a função hash do nível 2. Cada tabela S_j deve ter tamanho $m_j = n_j^2$, além de sua própria função hash:

$$h_j(k) = ((a_j k + b_j) \bmod p_j) \bmod m_j$$

em que no caso de S_2 , os valores são $a_2=10$, $b_2=18$ e $m_2=9$. Como $h_2(75) = 7$, a chave 75 vai ser armazenada na posição 7 de S_2 . Com essa construção, é garantido que não ocorrem colisões no segundo nível de hashing, de forma que a busca tem complexidade $O(1)$ mesmo no pior caso! A Tabela Hash resultante após o Perfect Hashing é ilustrada na figura a seguir.



Um observação é que o *Perfect Hashing* possui uma limitação: antes de iniciar o processo de inserção dos elementos, primeiramente devemos saber quantos elementos cada Tabela de Espalhamento do segundo nível (S_j) terá, ou seja quais são os valores de m_j . Então, é necessária uma etapa de setup da Tabela Hash antes de populá-la com os dados.

Algoritmo básico para *Perfect Hashing*

Para a Tabela de Espalhamento principal, escolha $m=n$ (número de chaves), escolha um número primo p maior que a máxima das chaves. Defina os valores de a e b aleatoriamente para construir a primeira função hash: $h(k) = ((ak + b) \bmod p) \bmod m$.

Teste a função hash $h(k)$ como segue:

1. Para cada chave k , encontre seu slot $h(k)$. Mantenha um contador n_j de quantas chaves foram mapeadas para o slot j .

2. Verifique se o espaço necessário para o armazenamento é muito grande: a soma de todos os n_j^2 é maior que $2n$?

$$\sum_{j=0}^{m-1} n_j^2 > 2n$$

Se a resposta for SIM, então a função de hash $h(k)$ não é boa o suficiente. Repita o processo com outros valores de a e b para construir a função de hash primária.

Se a função de hash primária é boa o suficiente, faça:

a. Para cada slot S_j , defina a função de hash secundária $h_j(k)$

$$h_j(k) = ((a_j k + b_j) \bmod p_j) \bmod m_j$$

setando $p_j = p$, $m_j = n_j^2$ e escolhendo a_j e b_j aleatoriamente.

b. Verifique que as funções hash secundárias $h_j(k)$ não resultam em colisões nas Tabelas de Espalhamento secundárias S_j , para todo j .

Consideremos o exemplo didático a seguir.

$$\begin{aligned} K &= \{8, 22, 36, 75, 61, 13, 84, 58\} & h(k) &= ((ak + b) \bmod p) \bmod m \\ p &= 87, a = 64, b = 5 & h_j(k) &= ((a_j k + b_j) \bmod p) \bmod m \end{aligned}$$

Passo 1:

Selecione p como um número primo maior que a maior das chaves e gere a e b aleatoriamente

0	
1	
2	
3	
4	
5	
6	
7	

$K = \{8, 22, 36, 75, 61, 13, 84, 58\}$
 $p = 87, a = 64, b = 5$

$$h(k) = ((ak + b) \bmod p) \bmod m$$

$$h_j(k) = ((a_j k + b_j) \bmod p) \bmod m$$

Passo 2:

Para cada chave k , calcule $h(k)$ e conte quantas chaves caem em cada slot

	contagem (n_j)
0	0
1	1
2	2
3	0
4	1
5	1
6	1
7	2

$K = \{8, 22, 36, 75, 61, 13, 84, 58\}$
 $p = 87, a = 64, b = 5$

$$h(k) = ((ak + b) \bmod p) \bmod m$$

$$h_j(k) = ((a_j k + b_j) \bmod p) \bmod m$$

Passo 3:

Verifique se $\sum_{j=0}^{m-1} n_j < 2n$

	contagem (n_j)
0	0
1	1
2	2
3	0
4	1
5	1
6	1
7	2

$$\sum_{j=0}^{m-1} n_j = 1 + 4 + 1 + 1 + 1 + 4 = 12 < 16$$

OK

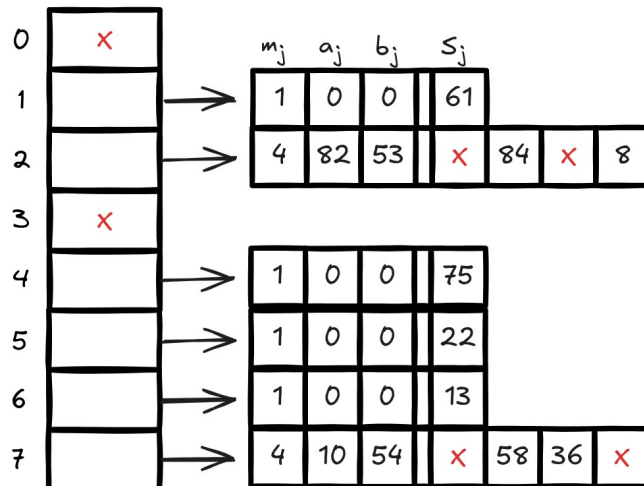
$K = \{8, 22, 36, 75, 61, 13, 84, 58\}$
 $p = 87, a = 64, b = 5$

$$h(k) = ((ak + b) \bmod p) \bmod m$$

$$h_j(k) = ((a_j k + b_j) \bmod p) \bmod m$$

Passo 4:

Distribua as chaves nas tabelas hash do segundo nível



"An arrow can only be shot by pulling it backward. So, when life is dragging you back with difficulties, it means that it's going to launch you into something great. Be patient."
 -- Author Unknown

Grafos: Fundamentos Básicos

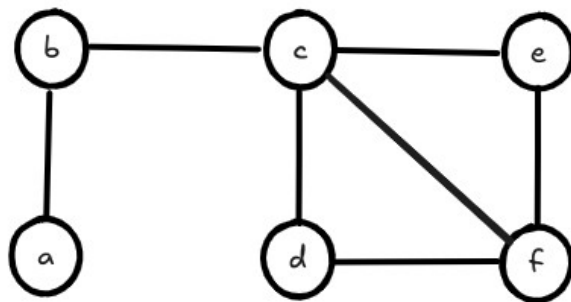
Grafos são objetos matemáticos que representam relações binárias entre elementos de um conjunto finito. Eles são utilizados na computação para representar estruturas complexas em que cada nó pode ser conectar a um número variável de vizinhos, como por exemplo, a internet e as redes sociais. Em termos gerais, um grafo consiste em um conjunto de vértices que podem estar ligados dois a dois por arestas. Se dois vértices são unidos por uma aresta, então eles são vizinhos. É uma estrutura fundamental para a computação, uma vez que diversos problemas do mundo real podem ser modelados com grafos, como encontrar caminhos mínimos entre dois pontos, alocação de recursos e modelagem de redes complexas.

Def: $G = (V, E)$ é um grafo se:

i) V é um conjunto não vazio de **vértices**

ii) $E \subseteq V \times V$ é uma relação binária qualquer no conjunto de vértices: conjunto de **arestas**

A figura a seguir ilustra um grafo $G = (V, E)$ arbitrário.



Denotamos por $N(v)$ o conjunto vizinhança do vértice v . Por exemplo, $N(b) = \{a, c\}$.

Def: Grau de um vértice v : $d(v)$

É o número de vezes que um vértice v é extremidade de uma aresta. Num grafo básico simples, é o mesmo que o número de vizinhos de v . Ex: $d(a) = 1$, $d(b) = 2$, $d(c) = 4$, , ...

Def: A lista de graus de $G = (V, E)$ é a lista que armazena os graus dos vértices em ordem crescente.

$L_G = (1, 2, 2, 2, 3, 4)$

Handshaking Lema: A soma dos graus dos vértices de G é igual a duas vezes o número de arestas.

$$\sum_{i=1}^n d(v_i) = 2m \quad (\text{condição de existência para grafos})$$

onde $n = |V|$ e $m = |E|$ denotam respectivamente o número de vértices e arestas.



Cada aresta contribui com +1 para o grau de cada vértice

Prova: (por indução)

Seja $P(n)$ definido como:

$$P(n): \sum_{i=1}^n d(v_i) = 2m$$

=> BASE: Note que nesse caso, $n = 1$, o que implica dizer que temos um único vértice. Então, para todo $m \geq 0$ (número de arestas), a soma dos graus será sempre um número par pois ambas as extremidades das arestas incidem sobre o único vértice de G .

=> PASSO DE INDUÇÃO: Para k arbitrário, mostrar que $P(k) \rightarrow P(k+1)$

Note que para k vértices, temos:

$$P(k): \sum_{i=1}^k d(v_i) = 2m$$

Ao adicionarmos exatamente um vértice a mais, temos $k+1$ vértices:

$$P(k+1): \sum_{i=1}^{k+1} d(v_i) = 2m'$$

Ao passar de k para $k+1$ vértices, temos duas opções:

- a) o grau do novo vértice é zero;
- b) o grau do novo vértice é maior que zero.

Caso a): Nessa situação, temos o número de arestas permanece inalterado, ou seja, $m = m'$. Pela hipótese de indução e sabendo que o grau no novo vértice é zero, podemos escrever:

$$\sum_{i=1}^{k+1} d(v_i) = \sum_{i=1}^k d(v_i) + d(v_{k+1}) = 2m + 0 = 2m'$$

ou seja, $P(k+1)$ é válida.

Caso b): Nessa situação, temos que o número de arestas $m' > m$. Seja $m' = m + a$, onde a denota o número de arestas adicionadas ao inserir o novo vértice $k+1$. Então, pela hipótese de indução e sabendo que o grau do novo vértice será a , podemos escrever:

$$\sum_{i=1}^{k+1} d(v_i) = \sum_{i=1}^k d(v_i) + d(v_{k+1}) + 1 + 1 + 1 + \dots + 1$$

a vezes 1

pois para cada extremidade das arestas no novo vértice, haverá outra extremidade em algum outro vértice. Isso implica em:

$$\sum_{i=1}^{k+1} d(v_i) = 2m + a + a = 2m + 2a = 2(m + a) = 2m'$$

ou seja, $P(k+1)$ é válida. Note que mesmo que alguma das arestas inseridas possuam ambas as extremidades no novo vértice $k+1$, a soma dos graus também será igual a $2m+2a$, o que continuará validando $P(k+1)$. Portanto, a prova está concluída.

Teorema: Em um grafo $G = (V, E)$ o número de vértices com grau ímpar é sempre par.

Podemos particionar V em 2 conjuntos: P (grau par) e I (grau ímpar). Assim,

$$\sum_{i=1}^n d(v_i) = \sum_{v \in P} d(v) + \sum_{u \in I} d(u) = 2m$$

Isso implica em

$$\sum_{u \in I} d(u) = 2m - \sum_{v \in P} d(v)$$

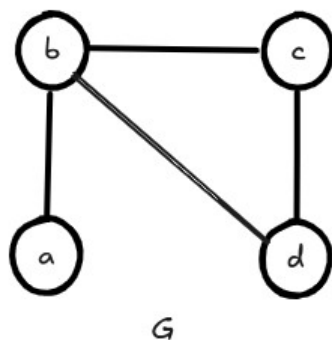
Como $2m$ é par e a soma de números pares é sempre par, resulta que a soma dos números ímpares também é par. Para que isso ocorra temos que ter $|I|$ par (número de elementos do conjunto I é par)

Representações computacionais de grafos

1. Matriz de adjacências A : matriz quadrada $n \times n$ definida como:

a) Grafos básicos simples

$$A_{i,j} = \begin{cases} 1, & j \in N(i) \\ 0, & j \notin N(i) \end{cases}$$



$$A = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{bmatrix} \end{matrix}$$

Propriedades básicas

i) $\text{diag}(A) = 0$

ii) matriz binária

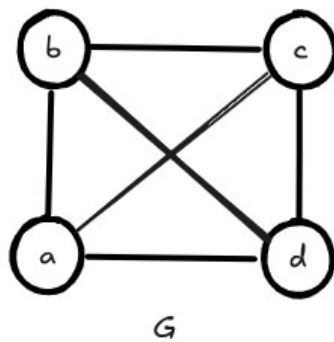
iii) $A = A^T$ (com exceção de grafos direcionados)

iv) $\sum_j A_{i,j} = d(v_i)$

v) Esparsa

vi) $O(n^2)$ em espaço – requer $\frac{n^2}{8}$ bytes para armazenamento (contíguos na memória)

2. Matriz de Incidência M: matriz $n \times m$ em que as linhas referem-se aos vértices e as colunas referem-se as arestas:

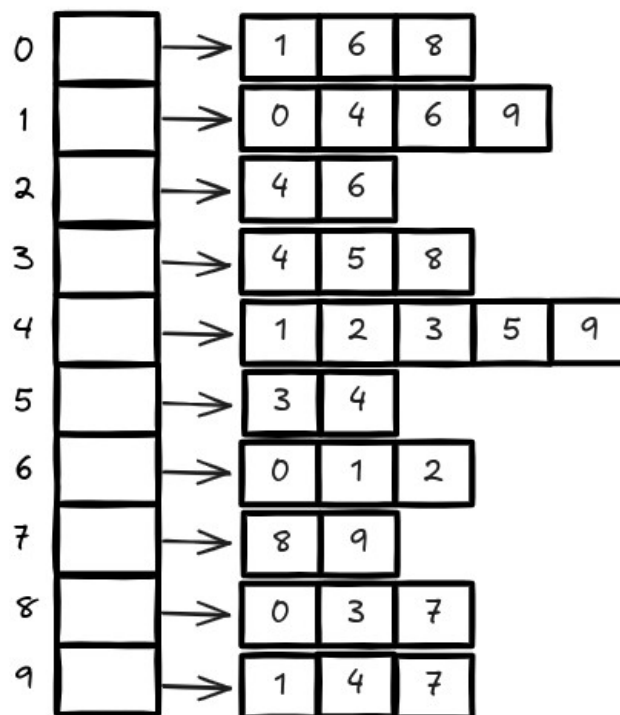


$$M = \begin{matrix} & \begin{matrix} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \end{matrix} \\ \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix} & \begin{matrix} a \\ b \\ c \\ d \end{matrix} \end{matrix}$$

$\Rightarrow O(nm)$ em espaço – requer $\frac{nm}{8}$ bytes para armazenamento (contíguos na memória) - RUIM

3. Lista de adjacências: estrutura dinâmica em que cada nó possui uma referência para uma lista encadeada com seus vizinhos.

Lista de adjacências



Note que ao contarmos a soma dos graus dos vértices (somatório dos tamanhos de cada lista encadeada), estamos de fato contando cada aresta duas vezes. Assim, de acordo com o *Handshaking Lema*, devemos dividir esse número total por 2 para obter o número exato de arestas.

Como temos n referências e cada referência é uma lista encadeada possui $d(v_i)$ nós, temos que o número total de nós é igual a $\sum_{i=1}^n d(v_i) = 2m$, o que resulta em $O(m)$.

Pergunta: Quantos bytes são necessários para armazenar um grafo G em memória com uma lista de adjacências?

Supondo que cada inteiro ocupa 32 bits, temos que cada nó ocupa $2m\left(\frac{32}{8}\right)$ bytes, o que equivale a 8m bytes (não precisa ser contíguo na memória).

Sabendo que a densidade de um grafo é dada por $d = \frac{m}{n^2}$, quando é preferível usar uma lista de adjacências em detrimento a uma matriz de adjacências?

A lista de adjacências deve ser escolhida se:

$$8m < \frac{n^2}{8} \rightarrow \frac{m}{n^2} < \frac{1}{64} \rightarrow d < \frac{1}{64} \rightarrow d < 0.015625 \text{ (grafo esparso)}$$

Ex: Se $n = 100$ e $m = 150$, temos $d = 150/10000 = 0.015$, o que é menor que 0.01562

Lembrando que o máximo valor de densidade é dado por:

$$d = \frac{\frac{n(n-1)}{2}}{n^2} = \frac{n^2 - n}{2n^2} = \frac{1}{2} - \frac{1}{2n} = \frac{1}{2} \left(1 - \frac{1}{n}\right)$$

o que tende a 0.5 quando n cresce arbitrariamente.

"Lembre-se sempre: as últimas partes a crescer em uma árvore são os frutos."
-- Autor Anônimo

Grafos: Busca em Grafos (BFS e DFS)

Importância: como navegar em grafos de maneira determinística de modo a percorrer um conjunto de dados não estruturado, visitando todos os nós exatamente uma vez.

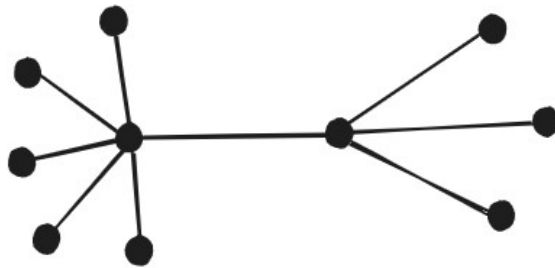
Objetivo: acessar/recuperar todos os elementos do conjunto V

Questões: De quantas maneiras podemos fazer isso? Como? Qual a melhor maneira?

Árvores e suas propriedades

Árvores são grafos especiais com diversas propriedades únicas. Devido a essas propriedades são extremamente importantes na resolução de vários tipos de problemas práticos. Veremos ao longo do curso que vários problemas que estudaremos se resumem a: dado um grafo G , extrair uma árvore T a partir de G , de modo que T satisfaça uma certa propriedade (como por exemplo, mínima profundidade, máxima profundidade, mínimo peso, mínimos caminhos, etc).

Def: Um grafo $G = (V, E)$ é uma árvore se G é acíclico e conexo.



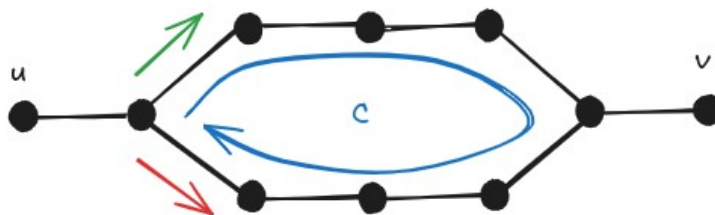
Teorema: G é uma árvore $\Leftrightarrow \exists$ um único caminho entre quaisquer 2 vértices $u, v \in V$

1. (ida) $p \rightarrow q = \neg q \rightarrow \neg p$

$\nexists$ um único caminho entre quaisquer $u, v \rightarrow G$ não é uma árvore

a) Pode existir um par u, v tal que $\nexists$ caminho (zero caminhos). Isso implica em G desconexo, o que implica que G não é uma árvore

b) Pode existir um par u, v tal que $\exists$ mais de um caminho.

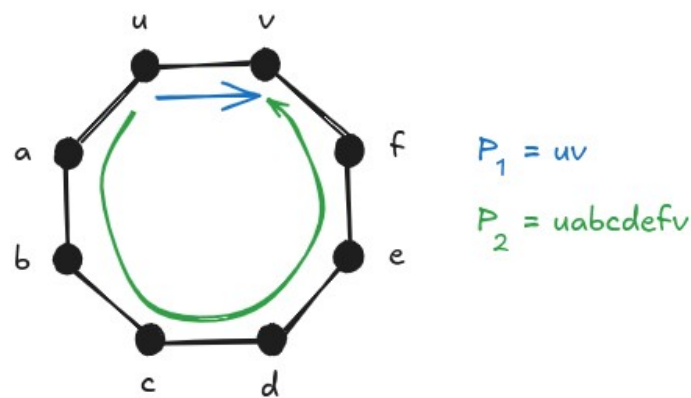


Porém neste caso temos a formação de um ciclo e portanto G não pode ser árvore.

2. (volta) $q \rightarrow p = \neg p \rightarrow \neg q$

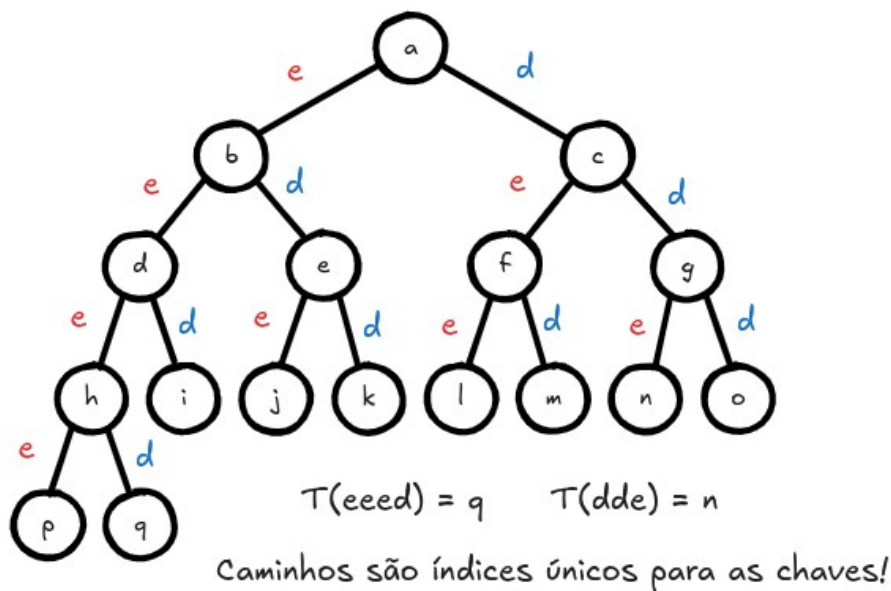
G não é árvore $\rightarrow \nexists$ único caminho entre quaisquer u, v

Para G não ser árvore, G deve ser desconexo ou conter um ciclo. Note que no primeiro caso existe um par u, v tal que não há caminho entre eles. Note que no segundo caso existem 2 caminhos entre u e v , conforme ilustra a figura

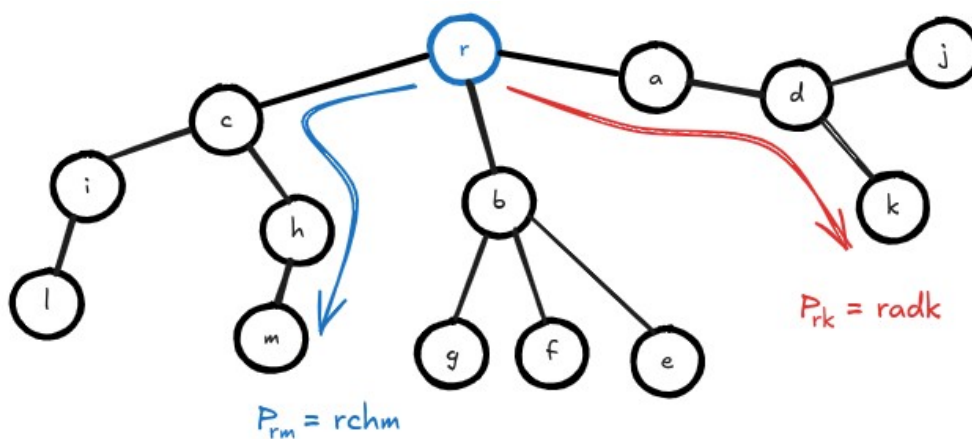


Relação entre busca em grafos e árvores

Buscar elementos num grafo G é basicamente o processo de extrair uma árvore T a partir de G . Mas porque? Como busca se relaciona com uma árvore? Isso vem de uma das propriedades das árvores. Numa árvore existe um único caminho entre 2 vértices u, v (caminho é um índice único).



Pode-se criar um esquema de indexamento baseado nos nós a esquerda e a direita. Cada elemento do conjunto possui um índice único que o recupera. No caso de árvores genéricas, o caminho faz o papel do índice único



Portanto, dado um grafo G , extrair uma árvore T com raiz r a partir dele, significa indexar unicamente cada elemento do conjunto (princípio por trás da busca).

Busca em Largura (Breadth-First Search - BFS)

Ideia geral: a cada novo nível descoberto, todos os vértices daquele nível devem ser visitados antes de prosseguir para o próximo nível

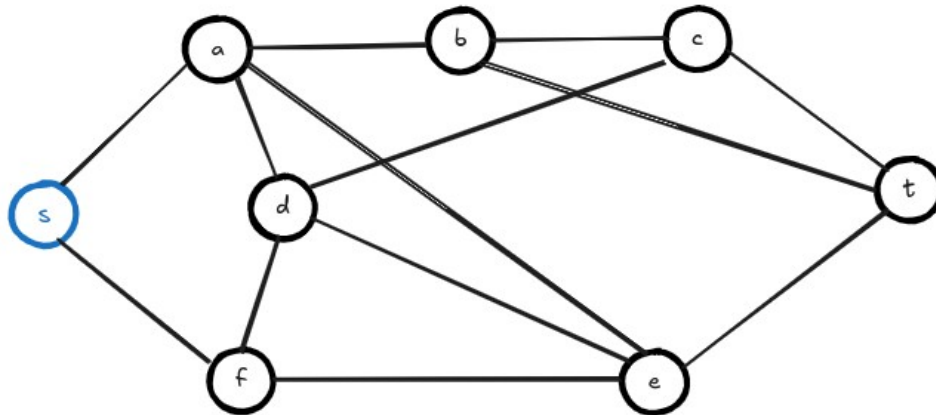
Definição das variáveis usadas no algoritmo

- i) v . color : status do vértice v (existem 3 possíveis valores)
 - a) WHITE: vértice v ainda não descoberto (significa que v ainda não entrou na fila Q)
 - b) GRAY: vértice já descoberto (significa que v está na fila Q)
 - c) BLACK: vértice finalizado (significa que v já saiu da fila Q)
- ii) $\lambda(v)$: armazena a menor distância de v até a raiz
- iii) $\pi(v)$: predecessor de v (onde estava quando descobri v)
- iv) Q : Fila (FIFO)
 - 2 primitivas
 - a) pop: remove elemento do início da fila
 - b) push: adiciona um elemento no final da fila

ALGORITMO

```
BFS( $G, s$ ) {  
  for each  $v \in V - \{s\}$  {           # inicializa as variáveis  
     $v$ .color = WHITE  
     $\lambda(v) = \infty$   
     $\pi(v) = \text{NIL}$   
  }  
   $s$ .color = GRAY  
   $\lambda(s) = 0$   
   $Q = \emptyset$   
  push( $Q, s$ )                        # insere raiz na fila  
  while  $Q \neq \emptyset$  {           # enquanto fila não for vazia  
     $u = \text{pop}(Q)$   
    for each  $v \in N(u)$  {           # para todo vizinho de  $u$   
      if  $v$ .color == WHITE {        # se ainda não passei aqui  
         $\lambda(v) = \lambda(u) + 1$     #  $v$  é descendente de  $u$   
         $\pi(v) = u$   
         $v$ .color = GRAY  
        push( $Q, v$ )  
      }  
    }  
     $u$ .color = BLACK # após processar todo vizinho, finaliza  
  }  
}
```

O algoritmo BFS recebe um grafo não ponderado G e retorna uma árvore T , conhecida como BFS-tree. Essa árvore possui uma propriedade muito especial: ela armazena os menores caminhos da raiz s a todos os demais vértices de T (menor caminho de s a v , $\forall v \in V$). O exemplo a seguir ilustra o trace completo do algoritmo BFS partindo do vértice s .



Trace do algoritmo BFS

i	$u = \text{pop}(Q)$	$V' = \{v \in N(u) \mid v.\text{color} = \text{WHITE}\}$	$\lambda(v)$	$\pi(v)$
0	s	{a, f}	$\lambda(a) = \lambda(s) + 1 = 1$ $\lambda(f) = \lambda(s) + 1 = 1$	$\pi(a) = s$ $\pi(f) = s$
1	a	{b, d, e}	$\lambda(b) = \lambda(a) + 1 = 2$ $\lambda(d) = \lambda(a) + 1 = 2$ $\lambda(e) = \lambda(a) + 1 = 2$	$\pi(b) = a$ $\pi(d) = a$ $\pi(e) = a$
2	f	$\emptyset$	---	---
3	b	{c, t}	$\lambda(c) = \lambda(b) + 1 = 3$ $\lambda(t) = \lambda(b) + 1 = 3$	$\pi(c) = b$ $\pi(t) = b$
4	d	$\emptyset$	---	---
5	e	$\emptyset$	---	---
6	c	$\emptyset$	---	---
7	t	$\emptyset$	---	---

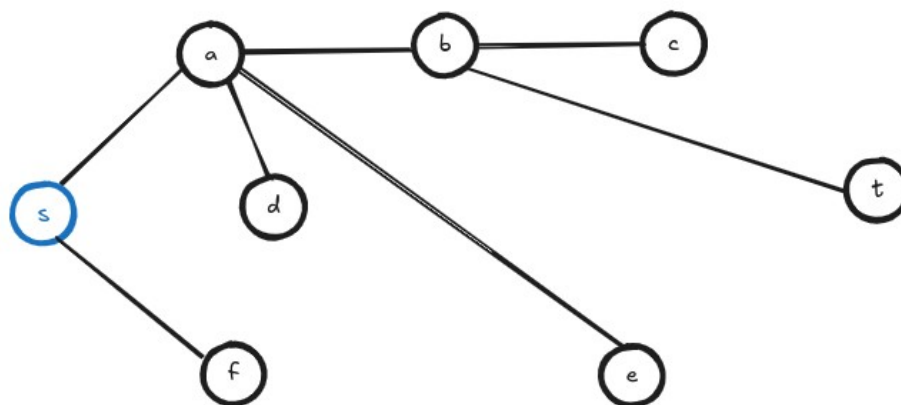
FILA

$Q^{(0)} = [s]$
 $Q^{(1)} = [a, f]$
 $Q^{(2)} = [f, b, d, e]$
 $Q^{(3)} = [b, d, e]$
 $Q^{(4)} = [d, e, c, t]$
 $Q^{(5)} = [e, c, t]$
 $Q^{(6)} = [c, t]$
 $Q^{(7)} = [t]$
 $Q^{(8)} = \emptyset$

A complexidade da busca em largura é $O(n + m)$, onde n é o número de vértices e m é o número de arestas. É fácil perceber que cada vértice e aresta serão acessados exatamente uma vez.

Árvore BFS

v	s	a	b	c	d	e	f	t
$\pi(v)$	---	s	a	b	a	a	s	b
$\lambda(v)$	0	1	2	3	2	2	1	3

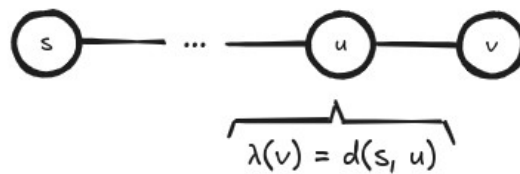


Note que a árvore nada mais é que a união dos caminhos mínimos de s (origem) a qualquer um dos vértices do grafo (destinos). A BFS-tree geralmente não é única, porém todas possuem a mesma profundidade (mínima distância da raiz ao mais distante)

Teorema: A BFS sempre termina com $\lambda(v) = d(s, v)$ para $\forall v \in V$, onde $d(s, v)$ é a distância geodésica (menor distância entre s e v).

Prova por contradição:

1. Sabemos que na BFS $\lambda(v) \geq d(s, v)$
2. Suponha que $\exists v \in V$ tal que $\lambda(v) > d(s, v)$, onde v é o primeiro vértice que isso ocorre ao sair da fila Q
3. Então, existe caminho P_{sv} pois senão $\lambda(v) = d(s, v) = \infty$ (contradiz 2)
4. Se existe P_{sv} então existe um caminho mínimo P_{sv}^*
5. Considere $u \in V$ como predecessor de v em P_{sv}^*
6. Então, $d(s, v) = d(s, u) + 1$ (pois u é predecessor de v)
7. Assim, temos



pois v foi o 1º a sair de Q com $\lambda(v) \neq d(s, v)$

$$\begin{array}{ccc} \lambda(v) > d(s, v) & = d(s, u) + 1 & = \lambda(u) + 1 \\ (2) & (6) & (5) \end{array}$$

e portanto $\lambda(v) > \lambda(u) + 1$ (*), o que é uma contradição pois só existem 3 possibilidades quando u sai da fila Q , ou seja, $u = \text{pop}(Q)$

i) v é WHITE: $\lambda(v) = \lambda(u) + 1$ (contradição)

ii) v é BLACK: se isso ocorre significa que v sai da fila Q antes de u , ou seja, $\lambda(v) < \lambda(u)$ (contradição)

iii) v é GRAY: então v foi descoberto por um w removido de Q antes de u , ou seja, $\lambda(w) \leq \lambda(u)$. Além disso, $\lambda(v) = \lambda(w) + 1$. Assim, temos $\lambda(w) + 1 \leq \lambda(u) + 1$, o que finalmente implica em $\lambda(v) \leq \lambda(u) + 1$ (contradição)

Portanto, $\nexists v \in V$ tal que $\lambda(v) > d(s, v)$.

Busca em Profundidade (Depth-First Search - DFS)

Ideia geral: a cada vértice descoberto, explorar um de seus vizinhos não visitados (sempre que possível). Imita a exploração de labirinto, aprofundando sempre que possível.

Definição das variáveis

i) $v.d$: discovery time (tempo de entrada em v)
 $v.f$: finishing time (tempo de saída de v)

ii) v . color : status do vértice v (existem 3 possíveis valores)

- a)** WHITE: vértice v ainda não descoberto
- b)** GRAY: vértice já descoberto
- c)** BLACK: vértice finalizado

iii) $\pi(v)$: predecessor de v (onde estava quando descobri v)

iv) Q : Pilha (LIFO)

Porém, para simular a pilha de execução, pode-se utilizar um recurso computacional: recursão!

Assim, não é necessário implementar de fato essa estrutura de dados (vantagem)

Porém, em casos extremos (tamanho muito grande), recursão pode gerar problemas (overflow).

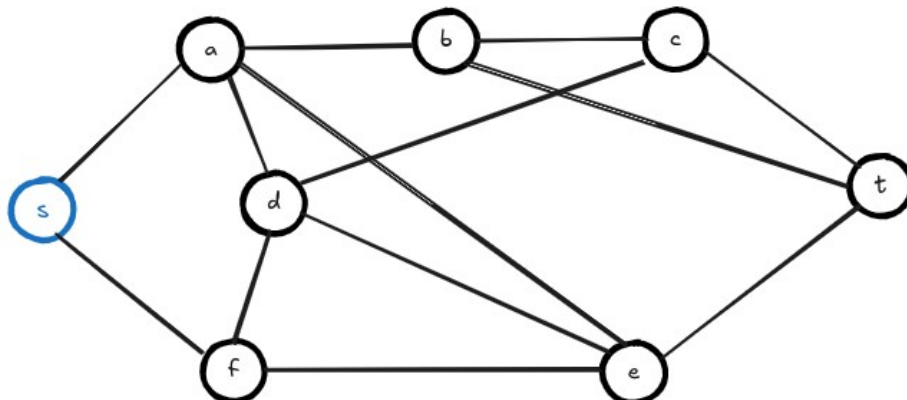
ALGORITMO (versão recursiva)

```
DFS(G, s) {  
  for each u ∈ V {    # inicializa as variáveis  
    u.color = WHITE  
    π(u) = NIL  
  }  
  time = 0            # variável global para contar o tempo  
  for each u ∈ V {  
    if u.color == WHITE  
      DFS_visit(G, u)  
  }  
}  
  
# Função recursiva chamada sempre que um vértice é descoberto  
DFS_visit(G, u) {  
  time = time + 1  
  u.d = time  
  u.color = GRAY  
  for each v ∈ N(u) {  
    if v.color == WHITE {  
      π(v) = u  
      DFS_visit(G, v)    # chamada recursiva  
    }  
  }  
  time = time + 1  
  u.f = time  
  u.color = BLACK  
}
```

Note que para implementar a versão iterativa (não recursiva) da Busca em Profundidade (DFS), basta usar o mesmo algoritmo da Busca em Largura (BFS) trocando a estrutura de dados fila por uma pilha.

Da mesma forma que o algoritmo BFS, esse método recebe um grafo G não ponderado e retorna uma árvore, a DFS_tree .

O exemplo a seguir ilustra o trace completo do algoritmo DFS.



u	u.color	u.d	$V' = \{v \in N(u) / v.color = WHITE\}$	$\pi(v)$	u.f
s	G	1	{a, f}	--	16
a	G	2	{b, d, e}	s	15
b	G	3	{c, t}	a	14
c	G	4	{d, t}	b	13
d	G	5	{e, f}	c	12
e	G	6	{f, t}	d	11
f	G	7	$\emptyset$	e	8
t	G	9	$\emptyset$	e	10

Diferenças entre BFS e DFS

BFS

- possui aspecto espacial
- encontrar caminhos mínimos
- estrutura de dados fila

x

DFS

- possui aspecto temporal
- vértices de corte, ordenação topológica
- estrutura de dados pilha

Propriedades da árvore da busca em profundidade (DFS_tree)

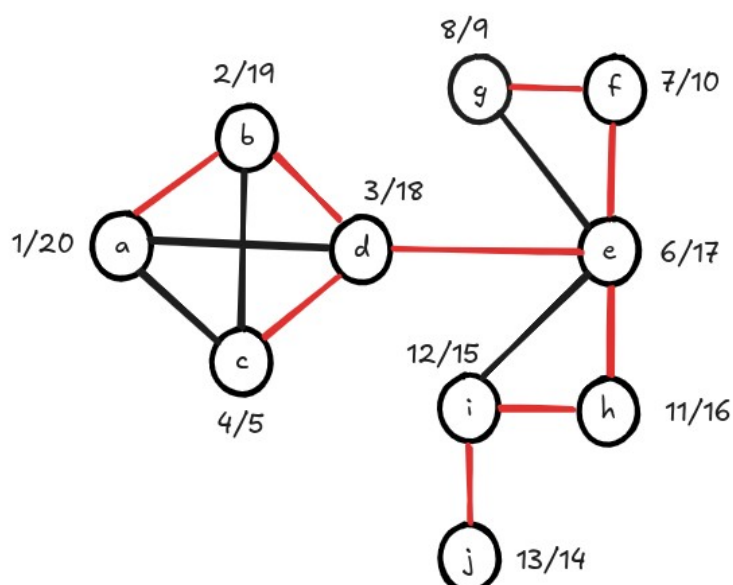
1. A rotulação tem o seguinte significado:

- $[u.d, u.f] \subset [v.d, v.f]$: u é descendente de v
- $[v.d, v.f] \subset [u.d, u.f]$: v é descendente de u
- $[v.d, v.f]$ e $[u.d, u.f]$ são disjuntos: estão em ramos distintos da árvore

2. Após a DFS, podemos classificar as arestas de G como:

- t_edges: $e \in T$
- b_edges (backward edges): $e \notin T$ (permitem voltar a um ancestral)

Def: Um vértice v é um vértice de corte em G, se e somente se v possui um filho s tal que $\nexists$ b_edge ligando s ou qualquer descendente de s a um ancestral de v.



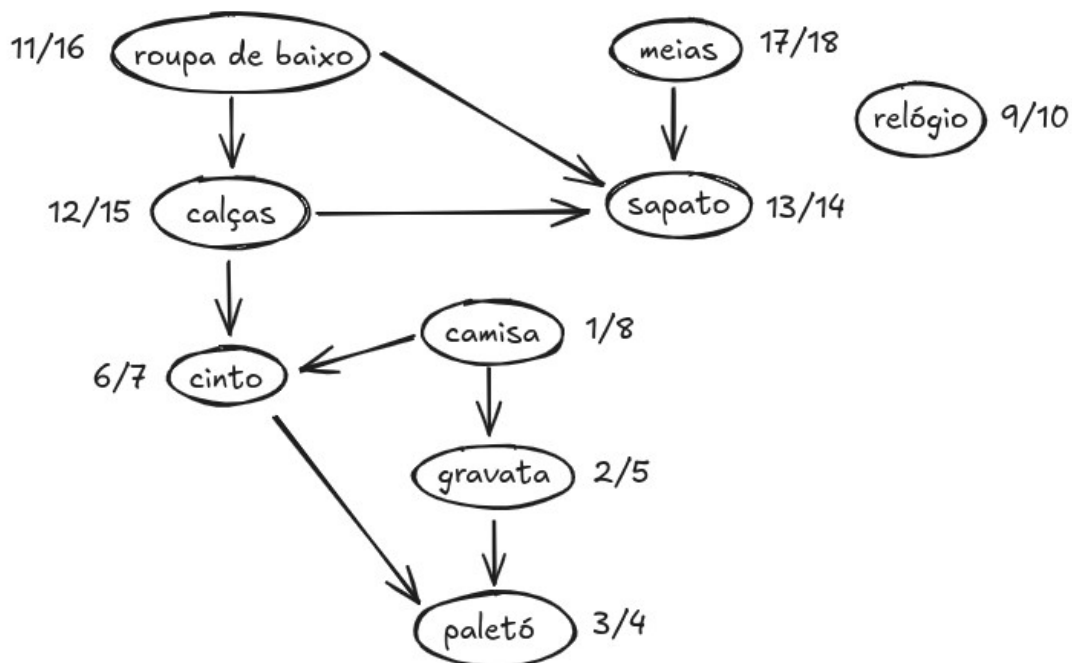
Perguntas:

- a) b é vértice de corte? Não pois b_edge (a, d) liga um sucessor a um antecessor
- b) d é vértice de corte? Sim, pois não há b_edge entre sucessor e antecessor
- c) e é vértice de corte? Sim, pois não há b_edge

Ordenação topológica

Uma ordenação topológica de um DAG (Directed Acyclic Graph) $G = (V, E)$ é uma ordenação linear de todos os seus vértices de modo que se G contém uma aresta (u, v) , então u aparece antes de v na ordenação. Podemos pensar na ordenação topológica de G como uma ordenação de seus vértices ao longo de uma reta horizontal de modo que todas as arestas apontam da esquerda para a direita.

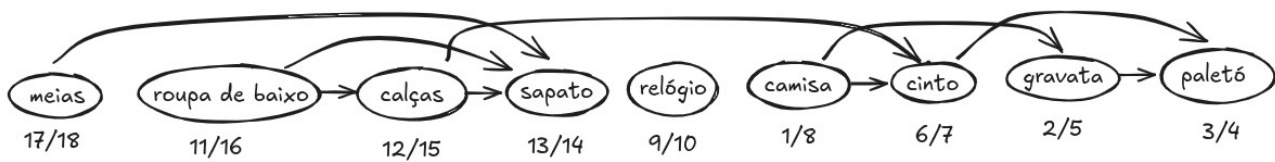
Uma aplicação da ordenação topológica seria a seguinte: imagine que você queira “ensinar” um robô humanoide a se vestir. Claramente, existe uma ordem natural que precisa ser seguida durante o processo. Por exemplo, não podemos calçar o sapato e depois colocar as meias. Suponha que o grafo acíclico direcionado que representa as dependências seja dado conforme a figura a seguir.



A rotulação obtida pela Busca em Profundidade mostra os tempos de entrada e saída para cada um dos vértices. O algoritmo Topological_Sort mostra a sequência lógica de passos necessárias para ordenar um DAG arbitrário.

```
Topological_Sort(G) {
    Execute DFS(G) para calcular v.f para todo  $v \in V$ 
    Conforme cada vértice é finalizado, insira v.f no início de
    uma lista encadeada L
    return L
}
```

A complexidade da ordenação topológica é a mesma da Busca em Profundidade, ou seja, $O(n + m)$, onde n é o número de vértices e m é o número de arestas. O resultado da ordenação topológica do DAG anterior é ilustrada na figura a seguir.



Lema: Um grafo direcionado $G = (V, E)$ é acíclico se e somente se uma busca em profundidade em G não gera b_edges.

(ida) $p \rightarrow q \equiv \neg q \rightarrow \neg p$

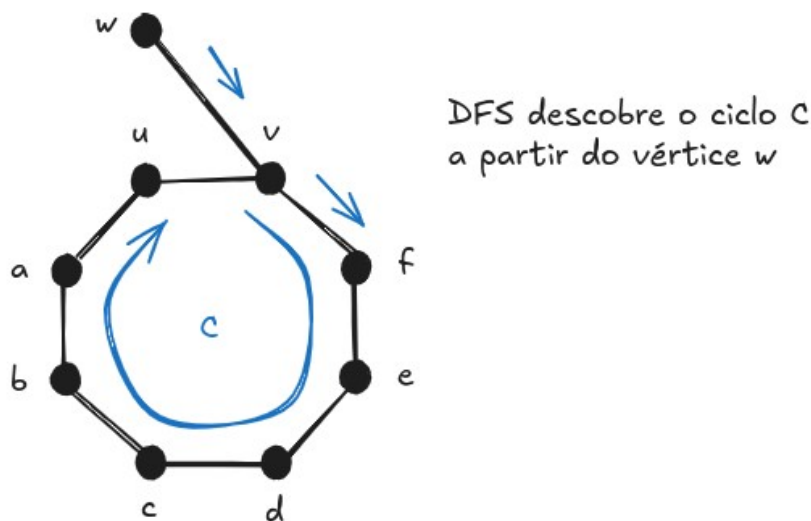
G acíclico $\rightarrow$ DFS não gera b_edge $\equiv$ DFS gera b_edge $\rightarrow G$ possui ciclo

1. Suponha que o algoritmo DFS gere uma b_edge (u, v) .
2. Então, por definição, ela liga u a um ancestral v .
3. Mas, na DFS_tree deve existir um caminho de v até u (pois é uma árvore), o que implica na geração de um ciclo.

(volta) $q \rightarrow p \equiv \neg p \rightarrow \neg q$

G contém ciclo $\rightarrow$ DFS gera b_edge

1. Suponha que G contenha um ciclo C .
2. Seja v o primeiro vértice a ser descoberto em C e seja (u, v) a aresta que precede v em C .



3. Então, no tempo v.d, os vértices do ciclo C formam um caminho de vértices WHITE de v até u .
4. Portanto, pelas propriedades do algoritmo DFS, u será um descendente de v na DFS-tree, o que implica que (u, v) é uma b_edge (pois não irá pertencer a árvore T).

A seguir veremos um resultado que garante a corretude do algoritmo Topological_Sort.

Teorema: O algoritmo Topological_Sort realiza a ordenação topológica de um grafo direcionado acíclico G .

1. Suponha que o algoritmo DFS seja executado no DAG G para determinar $v.f$, $\forall v \in V$.

2. Basta mostrar que para qualquer par de vértices $u, v \in V$, se existe uma aresta de u para v , então $v.f < u.f$
3. Considere uma aresta arbitrária (u, v) explorada pelo algoritmo DFS.
4. Note que quando essa aresta é explorada, v não pode ser GRAY, senão v seria um ancestral de u e a aresta (u, v) seria uma `b_edge`, gerando uma contradição ao lema anterior. Portanto, v deve ser WHITE ou BLACK.
5. Se v é WHITE, ele se torna um descendente de u e portanto temos $v.f < u.f$.
6. Se v é BLACK, ele já foi finalizado, de modo que o valor de $v.f$ já foi computado. Como a busca ainda está explorando u , o valor de $u.f$ ainda não foi calculado e será definido em um momento posterior. Logo, $v.f < u.f$, e a prova está concluída.

OBS: Uma página web muito interessante sobre a visualização de algoritmos em grafos pode ser acessada em <https://workshape.github.io/visual-graph-algorithms/>

The real voyage of discovery consists not in seeking new landscapes, but in having new eyes."
-- Marcel Proust

Grafos: Caminhos Mínimos e o Algoritmo de Dijkstra

Encontrar caminhos mínimos em grafos é um dos mais importantes problemas da computação, em grande parte por ser utilizado em aplicações nas mais diversas áreas da ciência. Vimos que a busca em largura é capaz de encontrar caminhos mínimos em grafos não ponderados. Veremos aqui como resolver o problema no caso de grafos ponderados

Def: Caminho ótimo

Seja $G = (V, E, w)$ com $w: E \rightarrow R^+$ uma função de custo para as arestas. Um caminho P^* de v_0 a v_n é ótimo se seu peso é o menor possível:

$$w(P^*) = \sum_{i=0}^{n-1} w(v_i, v_{i+1}) = w(v_0, v_1) + w(v_1, v_2) + \dots + w(v_{n-1}, v_n) \quad (\text{soma dos pesos das arestas})$$

Antes de introduzirmos os algoritmos, iremos apresentar uma primitiva comum a todos eles. Trata-se da função relax, que aplica a operação conhecida como relaxamento a uma aresta de um grafo ponderado.

Primitiva relax

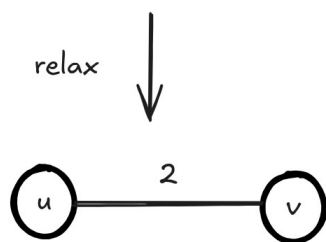
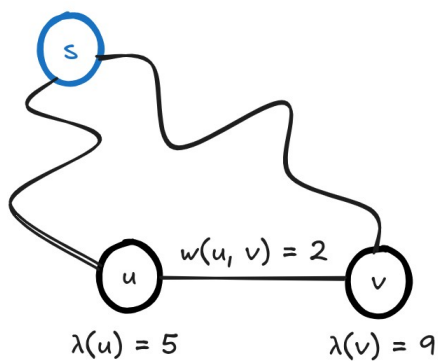
$\text{relax}(u, v, w)$: relaxar a aresta (u, v) de peso w

Para entender o que significa relaxar a aresta (u, v) de peso w , precisamos definir $\lambda(u)$ e $\lambda(v)$

Quem é $\lambda(u)$? É o custo atual de sair da origem s e chegar até u

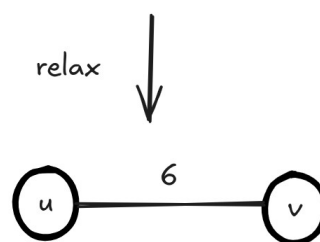
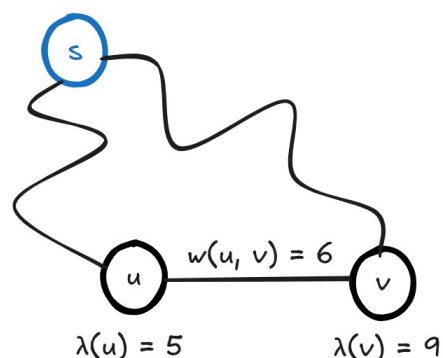
Quem é $\lambda(v)$? É o custo atual de sair da origem s e chegar até v

Ideia geral: é uma boa ideia passar por u para chegar em v sabendo que o custo de ir de u até v é w ?



$$\lambda(v) = \min\{\lambda(v), \lambda(u) + w(u, v)\} = 7$$

vale a pena passar por u para chegar em v



$$\lambda(v) = \min\{\lambda(v), \lambda(u) + w(u, v)\} = 9$$

não vale a pena passar por u para chegar em v

Note que a operação $\text{relax}(u, v, w)$ nunca aumenta o valor de $\lambda(v)$, apenas diminui!

ALGORITMO

<pre>relax(u, v, w) { if $\lambda(v) > \lambda(u) + w(u, v)$ { $\lambda(v) = \lambda(u) + w(u, v)$ $\pi(v) = u$ } }</pre>	<pre>relax(u, v, w) { $\lambda(v) = \min\{\lambda(v), \lambda(u) + w(u, v)\}$ if $\lambda(v)$ was updated $\pi(v) = u$ }</pre>
---	---

O que varia nos diversos algoritmos para encontrar caminhos mínimos são os seguintes aspectos:

- i) Quantas e quais arestas devemos relaxar?
- ii) Quantas vezes devemos relaxar as arestas?
- iii) Em que ordem devemos relaxar as arestas?

A seguir veremos um algoritmo muito mais eficiente para resolver o problema: o algoritmo de Dijkstra. Basicamente, esse algoritmo faz uso de uma política de gerenciamento de vértices baseada em aspectos de programação dinâmica. O que o método faz é basicamente criar uma fila de prioridades para organizar os vértices de modo que quanto menor o custo $\lambda(v)$ maior a prioridade do vértice em questão.

Assim, a ideia é expandir primeiramente os menores ramos da árvore de caminhos mínimos, na expectativa de que os caminhos mínimos mais longos usarão como base os subcaminhos obtidos anteriormente. Trata-se de um mecanismo de reaproveitar soluções de subproblemas para a solução do problema como um todo.

Definição das variáveis

$\lambda(v)$: menor custo até o momento para o caminho s-v

$\pi(v)$: predecessor de v na árvore de caminhos mínimos

Q : fila de prioridades dos vértices (maior prioridade = menor $\lambda(v)$)

A fila de prioridades Q possui 3 primitivas básicas:

a) Insert(Q, v): insere um vértice v no fim da fila Q.

b) ExtractMin(Q): remove da fila o vértice de maior prioridade.

c) DecreaseKey(Q, v, $\lambda(v)$): modifica a prioridade do vértice v da fila Q, atualizando o seu valor de $\lambda(v)$ (note que sempre irá diminuir esse valor que é o tamanho do caminho mínimo)

S: conjunto dos vértices já finalizados (vértices para os quais o caminho mínimo já foi obtido).

```
Dijkstra(G, w, s) {
    for each  $v \in V$  {
         $\lambda(v) = \infty$ 
         $\pi(v) = \text{nil}$ 
    }
     $\lambda(s) = 0$ 
     $S = \emptyset$ 
```

```

Q = ∅
for each v ∈ V
    Insert(Q, v)
while Q ≠ ∅ {
    u = ExtractMin(Q)
    S = S ∪ {u}
    for each v in N(u) {
        λ(v) = min{λ(v), λ(u) + w(u,v)}
        if λ(v) was updated {
            π(v) = u
            Decrease_Key(Q, v, λ(v))
        }
    }
}

```

Algoritmos em grafos e suas estruturas de dados

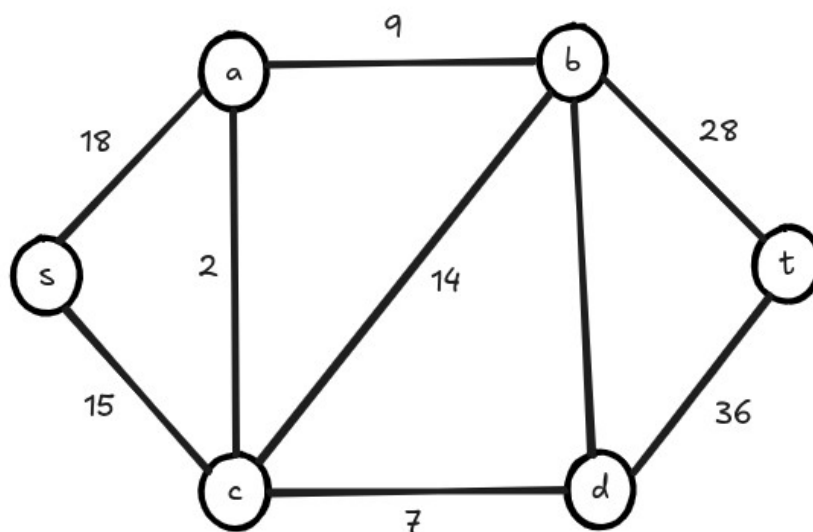
1. Busca em Largura (BFS): Fila **Q**
2. Busca em Profundidade (DFS): Pilha **S**
3. Algoritmo de Dijkstra: Fila de prioridades **Q**

Neste contexto, note que o algoritmo de Dijkstra é uma generalização da busca em largura para o caso em que os pesos das arestas não são todos iguais. Ambos crescem primeiro os menores ramos da árvore.

No algoritmo BFS toda aresta tem mesmo peso, já no algoritmo de Dijkstra esse peso é variável.

Considere o seguinte grafo ponderado. Suponha que deseja-se encontrar os menores caminhos do vértice *s* até todos os demais vértices. Para isso, basta aplicarmos o algoritmo de Dijkstra com raiz no vértice *s*.

A seguir apresentamos um trace completo (passo a passo) desse algoritmo.



Fila de prioridades

	s	a	b	c	d	t
$\lambda^{(0)}(v)$	0	∞	∞	∞	∞	∞
$\lambda^{(1)}(v)$		18	∞	15	∞	∞
$\lambda^{(2)}(v)$		17	29		22	∞
$\lambda^{(3)}(v)$			26		22	∞
$\lambda^{(4)}(v)$			26			58
$\lambda^{(5)}(v)$						54

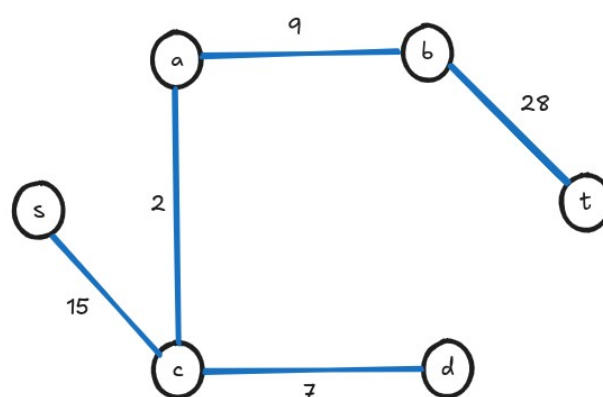
Ordem de acesso aos vértices

u	$V' = \{v \in N(u) \wedge v \in Q\}$	$\lambda(v), \forall v \in V'$	$\pi(v)$
s	{a, c}	$\lambda(a) = \min\{\lambda(a), \lambda(s) + w(s, a)\} = \min\{\infty, 18\} = 18$ $\lambda(c) = \min\{\lambda(c), \lambda(s) + w(s, c)\} = \min\{\infty, 15\} = 15$	$\pi(a) = s$ $\pi(c) = s$
c	{a, b, d}	$\lambda(a) = \min\{\lambda(a), \lambda(c) + w(c, a)\} = \min\{18, 17\} = 17$ $\lambda(b) = \min\{\lambda(b), \lambda(c) + w(c, b)\} = \min\{\infty, 29\} = 29$ $\lambda(d) = \min\{\lambda(d), \lambda(c) + w(c, d)\} = \min\{\infty, 22\} = 22$	$\pi(a) = c$ $\pi(b) = c$ $\pi(d) = c$
a	{b}	$\lambda(b) = \min\{\lambda(b), \lambda(a) + w(a, b)\} = \min\{29, 26\} = 26$	$\pi(b) = a$
d	{b, t}	$\lambda(b) = \min\{\lambda(b), \lambda(d) + w(d, b)\} = \min\{26, 32\} = 26$ $\lambda(t) = \min\{\lambda(t), \lambda(d) + w(d, t)\} = \min\{\infty, 58\} = 58$	--- $\pi(t) = d$
b	{t}	$\lambda(t) = \min\{\lambda(t), \lambda(b) + w(b, t)\} = \min\{58, 54\} = 54$	$\pi(t) = b$
t	$\emptyset$	---	---

Mapa de predecessores (árvore final)

v	s	a	b	c	d	t
$\pi(v)$	---	s	a	b	a	b
$\lambda(v)$	0	17	26	15	22	54

Árvore de caminhos mínimos (armazena os menores caminhos de s a todos os demais vértices)

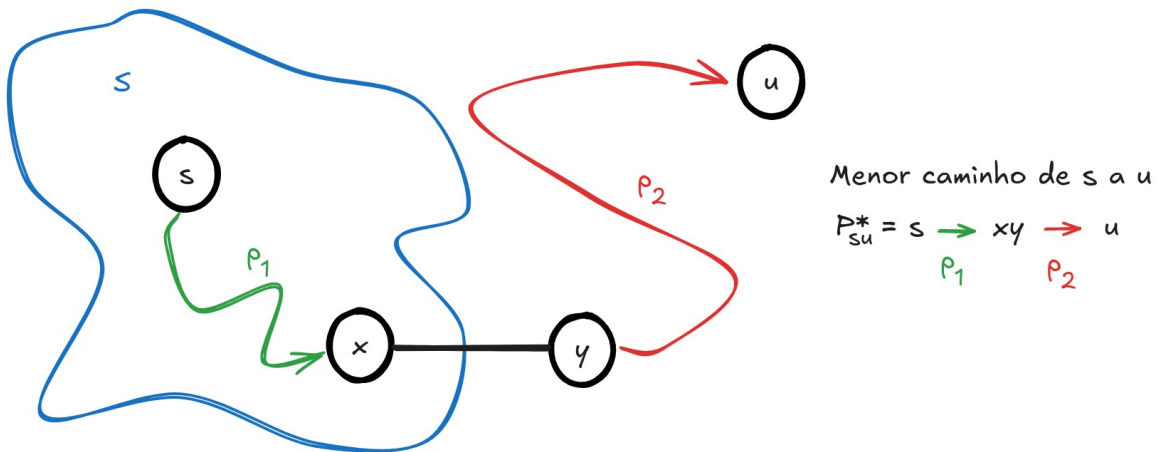


A seguir iremos demonstrar a otimalidade do algoritmo de Dijkstra.

Teorema: O algoritmo de Dijkstra termina com $\lambda(v) = d(s, v), \forall v \in V$

Note que sempre $\lambda(v) \geq d(s, v)$ (*)

1. Suponha que u seja o 1º vértice para o qual $\lambda(u) \neq d(s, u)$ quando u entra em S .
2. Então, $u \neq s$ pois senão $\lambda(s) = d(s, s) = 0$
3. Assim, existe um caminho P_{su} pois senão $\lambda(u) = d(s, u) = \infty$. Portanto, existe um caminho mínimo P_{su}^*
4. Antes de adicionar u a S , P_{su}^* possui $s \in S$ e $u \in V - S$
5. Seja y o 1º vértice em P_{su}^* tal que $y \in V - S$ e seja x seu predecessor ($x \in S$)



Obs: Note que tanto p_1 quanto p_2 não precisam necessariamente ter arestas

6. Como $x \in S$, $\lambda(x) = d(s, x)$ e no momento em que ele foi inserido a S , a aresta (x, y) foi relaxada, ou seja:

$$\lambda(y) = \lambda(x) + w(x, y) = d(s, x) + w(x, y) = d(s, y)$$

7. Mas y antecede a u no caminho e como $w: E \rightarrow \mathbb{R}^+$ (pesos positivos), temos:

$$d(s, y) \leq d(s, u)$$

e portanto

$$\lambda(y) = d(s, y) \leq d(s, u) \leq \lambda(u)$$

(6) (7) (*)

8. Mas como ambos y e u pertencem a $V - S$, quando u é escolhido para entrar em S temos $\lambda(u) \leq \lambda(y)$

9. Como $\lambda(y) \leq \lambda(u)$ e $\lambda(u) \leq \lambda(y)$ então temos que $\lambda(u) = \lambda(y)$, o que implica em:

$$\lambda(y) = d(s, y) = d(s, u) = \lambda(u)$$

o que gera uma contradição. Portanto $\nexists u \in V$ tal que $\lambda(u) \neq d(s, u)$ quando u entra em S .

Análise da complexidade

Há duas formas de analisar a complexidade do algoritmo de Dijkstra dependendo das estruturas de dados utilizadas.

Caso 1: Fila de prioridades Q representada por um array simples de n elementos.

- a) Inicialização dos $\lambda(v)$ é $O(n)$
- b) Inserção dos vértices na fila Q é $O(n)$ (é $O(1)$ para cada vértice)
- c) Loop WHILE é executado n vezes (1 vez para cada $v \in Q$)
- d) $u = \text{ExtractMin}(Q)$ é $O(n)$ (equivale a encontrar menor elemento do array)
- e) Atualização do valor de $\lambda(v)$ é $O(1)$, mas executa $k = d(u)$ vezes (loop FOR)
- f) Decrease_Key é $O(1)$ (acesso direto)

Sendo assim a função $T(n)$ que mede a complexidade do algoritmo é:

$$T(n) = O(n) + O(n) + O(n) * O(n) + (O(1) + O(1)) * (d(v_1) + d(v_2) + \dots + d(v_n))$$

Sabendo que a multiplicação de dois termos lineares resulta em quadrático e que de acordo com o Handshaking Lema, a soma dos graus de um grafo é igual a duas vezes o número de arestas, temos:

$$T(n) = O(n) + O(n^2) + O(1) * O(m)$$

Como em todo grafo básico simples $m < n^2$, temos finalmente que o algoritmo é $O(n^2)$.

Caso 2: Fila de prioridades Q representada por um heap binário (min-heap)

- a) Inicialização dos $\lambda(v)$ é $O(n)$
- b) Inserção dos vértices na fila Q é $O(n)$, pois todos os pesos são iguais inicialmente (infinitos).
- c) Loop WHILE é executado n vezes (1 vez para cada $v \in Q$)
- d) $u = \text{ExtractMin}(Q)$ é $O(\log n)$ (dequeue), mas dentro de loop (n vezes)
- e) Atualização do valor de $\lambda(v)$ é $O(1)$, mas executa $k = d(u)$ vezes (loop FOR)
- f) Decrease_Key é $O(\log n)$ (pode ter que subir no min-heap até a raiz – altura da árvore)

Sendo assim a função $T(n)$ que mede a complexidade do algoritmo é:

$$T(n) = O(n) + O(n \log n) + (O(1) + O(\log n)) * (d(v_1) + d(v_2) + \dots + d(v_n))$$

De modo similar ao caso anterior, podemos escrever:

$$T(n) = O(n) + O(n \log n) + O(m) + O(m \log n)$$

Como o termo com logaritmo domina os demais: $T(n) = O((n+m) \log n)$

Mas como em grafos conexos $m > n - 1$, chega-se que $T(n)$ é $O(m \log n)$.

Qual das duas implementações é mais eficiente? Depende! Devemos preferir a primeira opção se $m \log n > n^2$, o que equivale a:

$$\frac{m}{n^2} > \frac{1}{\log n} \rightarrow d > \frac{1}{\log n}$$

Em grafos mais densos, o caso 1 é mais eficiente.

Em grafos menos densos, o caso 2 é mais eficiente.

Por exemplo, se $n = 1024$, temos a seguinte regra:

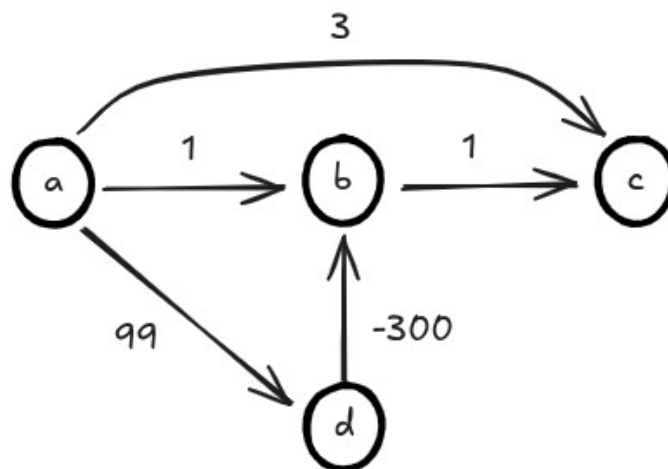
- se $d > 0.1$, opção 1 é melhor
- se $d < 0.1$, opção 2 é melhor

Para que d seja igual a 0.1, um grafo com $n = 1024$ vértices deve ter 104857 arestas.

Lembrando que o algoritmo de Dijkstra tem uma limitação: em grafos com pesos negativos, sua convergência não é garantida! Ou seja, ele pode não funcionar corretamente.

Situações em que o algoritmo de Dijkstra falha

O algoritmo de Dijkstra pode não funcionar corretamente quando o grafo admite pesos negativos em suas arestas. Para entender o porque isso ocorre, iremos apresentar um exemplo ilustrativo. Considere o seguinte grafo ponderado. Desejamos encontrar a árvore de caminhos mínimos com raiz em A.



A seguir encontra-se a execução passo a passo do algoritmo de Dijkstra.

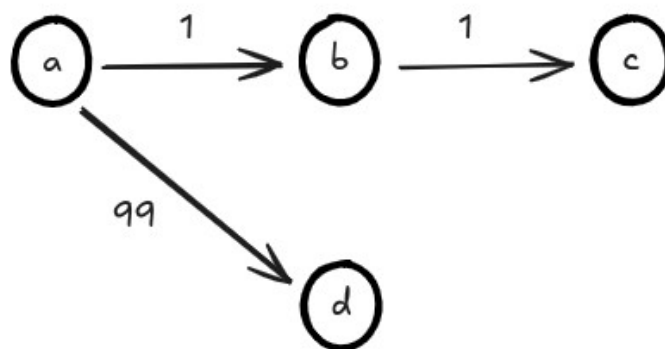
Fila de prioridades

	a	b	c	d
$\lambda^{(0)}(v)$	0	∞	∞	∞
$\lambda^{(1)}(v)$		1	3	99
$\lambda^{(2)}(v)$			1	99
$\lambda^{(3)}(v)$				99

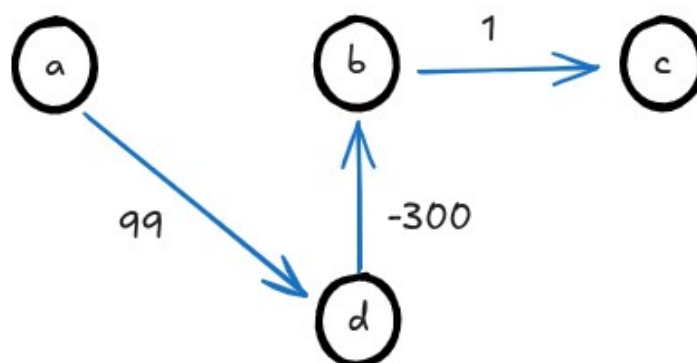
Ordem de acesso aos vértices

u	$V' = \{v \in N(u) \wedge v \in Q\}$	$\lambda(v), \forall v \in V'$	$\pi(v)$
a	{b, c, d}	$\lambda(b) = \min\{\infty, 1\} = 1$	$\pi(b) = a$
		$\lambda(c) = \min\{\infty, 3\} = 3$	$\pi(c) = a$
		$\lambda(d) = \min\{\infty, 99\} = 99$	$\pi(d) = a$
b	{c}	$\lambda(c) = \min\{3, 1+1\} = 2$	$\pi(c) = b$
c	$\emptyset$	---	---
d	$\emptyset$	---	---

Note que ao visitar o vértice d, o vértice b já não está mais na fila. Portanto, a árvore de caminhos mínimos retornada pelo Dijkstra é a seguinte:



o que não está correto, pois deveria ser:



A pergunta natural é: como evitar que isso ocorra?

Para isso, devemos utilizar o algoritmo de Bellman-Ford, que apesar de menos eficiente do que o algoritmo de Dijkstra, é o único capaz de encontrar caminhos mínimos em grafos em que as arestas possuem custo negativo.

Algoritmo de Bellman-Ford

Ideia: a cada passo relaxar $\forall e \in E$ em ordem arbitrária, repetindo o processo $|V| - 1$ vezes

```

Bellman_Ford(G, w, s) {
    for each v ∈ V {
        λ(v) = ∞
        π(v) = NIL
    }
    λ(s) = 0
    for i = 1 to |V|-1 {
        for each e = (u, v) in E
            relax(u, v, w)
    }
}

```

Para aplicar o algoritmo Bellman-Ford, o primeiro passo é definir uma ordem para que as arestas sejam relaxadas. Note que essa ordem pode ser arbitrária! Vamos considerar a seguinte ordem:

(a, b); (a, c); (a, d); (b, c); (d, b)

Como temos 4 vértices, o loop principal terá 3 iterações.

1ª iteração: (a, b): $\lambda(b) = \min\{\infty, 1\} = 1$

(a, c): $\lambda(c) = \min\{\infty, 3\} = 3$

(a, d): $\lambda(d) = \min\{\infty, 99\} = 99$

(b, c): $\lambda(c) = \min\{3, 1+1\} = 2$

(d, b): $\lambda(b) = \min\{1, 99-300\} = -201$

2ª iteração: (a, b): $\lambda(b) = \min\{1, 1\} = 1$

(a, c): $\lambda(c) = \min\{2, -201+1\} = -200$

(a, d): $\lambda(d) = \min\{99, 99\} = 99$

(b, c): $\lambda(c) = \min\{-200, -201+1\} = -200$

(d, b): $\lambda(b) = \min\{-201, 99-300\} = -201$

3ª iteração: (a, b): $\lambda(b) = \min\{1, 1\} = 1$

(a, c): $\lambda(c) = \min\{2, -201+1\} = -200$

(a, d): $\lambda(d) = \min\{99, 99\} = 99$

(b, c): $\lambda(c) = \min\{-200, -201+1\} = -200$

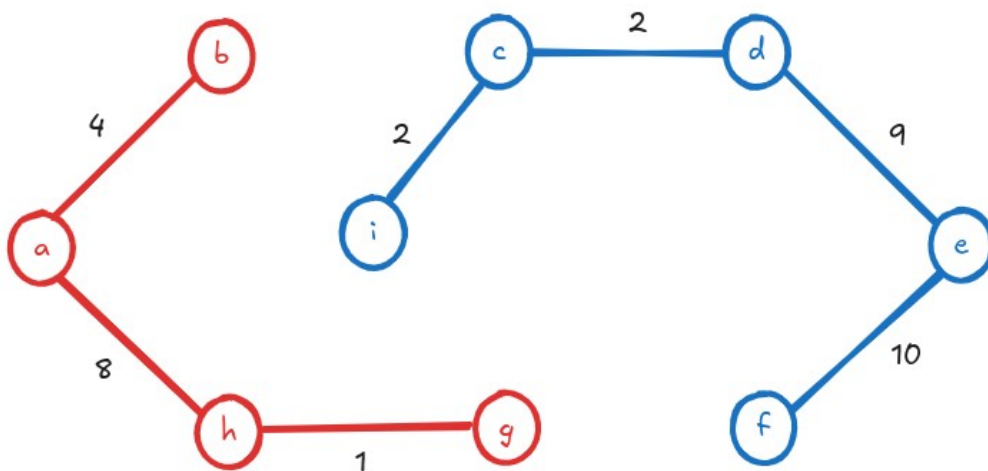
(d, b): $\lambda(b) = \min\{-201, 99-300\} = -201$

o que resulta na árvore desejada.

Ordem de acesso aos vértices

u	$V' = \{v \in N(u) \wedge v \in Q\}$	$\lambda(v), \forall v \in V'$	$\pi(v)$
a	{b, h}	$\lambda(b) = \min\{\infty, 4\} = 4$ $\lambda(h) = \min\{\infty, 8\} = 8$	$\pi(b) = a$ $\pi(h) = a$
e	{d, f}	$\lambda(d) = \min\{\infty, 9\} = 9$ $\lambda(f) = \min\{\infty, 10\} = 10$	$\pi(d) = e$ $\pi(f) = e$
b	{c, h}	$\lambda(c) = \min\{\infty, 4+8\} = 12$ $\lambda(h) = \min\{8, 4+11\} = 8$	$\pi(c) = b$ -----
h	{i, g}	$\lambda(i) = \min\{\infty, 8+7\} = 15$ $\lambda(g) = \min\{\infty, 8+1\} = 9$	$\pi(i) = h$ $\pi(g) = h$
d	{c, f}	$\lambda(c) = \min\{12, 9+2\} = 11$ $\lambda(f) = \min\{10, 9+14\} = 10$	$\pi(c) = d$ -----
g	{f, i}	$\lambda(f) = \min\{10, 9+14\} = 10$ $\lambda(i) = \min\{15, 9+5\} = 14$	$\pi(i) = g$ -----
f	{c}	$\lambda(c) = \min\{11, 10+4\} = 11$	-----
c	{i}	$\lambda(i) = \min\{14, 11+2\} = 13$	$\pi(i) = c$ -----
i	$\emptyset$	-----	-----

Floresta de caminhos ótimos



A heurística A*

É uma técnica aplicada para acelerar a busca por caminhos mínimos em certos tipos de grafos. Pode ser considerado uma generalização do algoritmo de Dijkstra. Um dos problemas com o algoritmo de Dijkstra é não levar em consideração nenhuma informação sobre o destino. Em grafos densamente conectados esse problema é amplificado devido ao alto número de arestas e aos muitos caminhos a serem explorados. Em suma, o algoritmo A* propõe uma heurística para dizer o quanto estamos chegando próximos do destino através da modificação da prioridades dos vértices na fila Q. É um algoritmo muito utilizado na IA de jogos eletrônicos.

Ideia: modificar a função que define a prioridade dos vértices

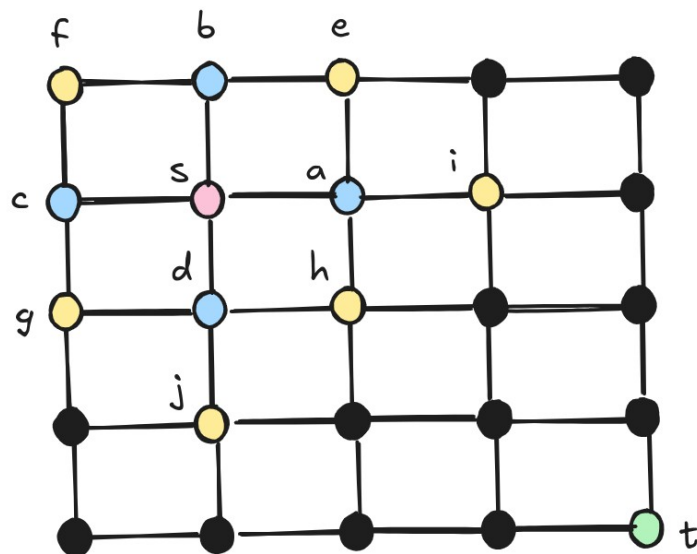
$$\alpha(v) = \lambda(v) + \gamma(v) \quad \text{onde}$$

$\lambda(v)$: custo atual de ir da origem s até v

$\gamma(v)$: custo estimado de v até o destino t (alvo)

Desafio: como calcular $\gamma(v)$?

- É viável apenas alguns casos específicos



Considere o grafo acima. O vértice s é a origem e o vértice t é o destino. Neste caso temos:

$$\lambda(a) = \lambda(b) = \lambda(c) = \lambda(d) = 1$$

o que significa que no Dijkstra, todos eles teriam a mesma prioridade. Note porém que, utilizando a distância Euclidiana para obter uma estimativa de distância até a origem, temos:

$$\begin{aligned} \gamma(a) &= \gamma(d) = \sqrt{4+9} = \sqrt{13} \\ \gamma(b) &= \gamma(c) = \sqrt{9+16} = \sqrt{25} = 5 \end{aligned}$$

Ou seja, no A^* , devemos priorizar a e d em detrimento de b e c uma vez que

$$1 + \sqrt{13} < 1 + 5$$

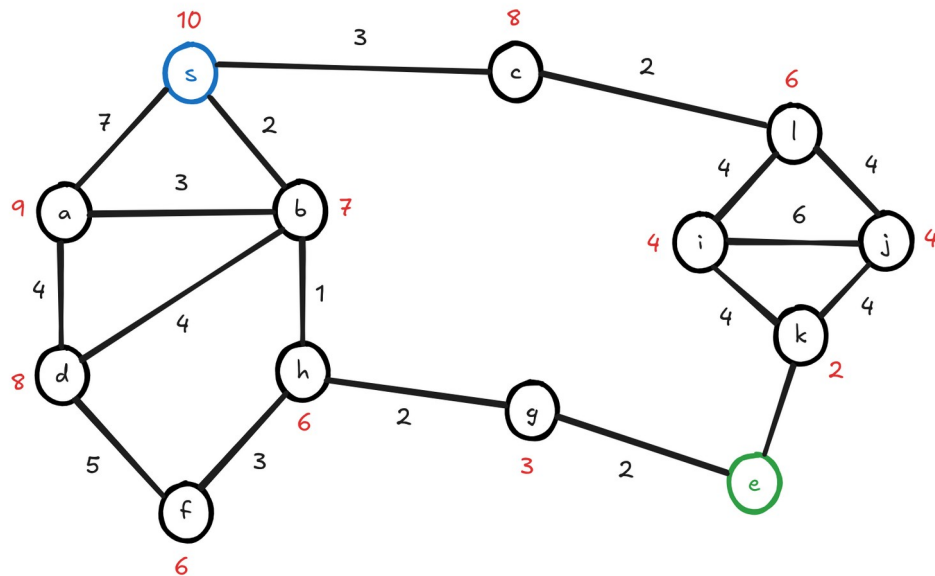
e portanto a e d saem da fila de prioridades antes. Isso ocorre pois no A^* eles são considerados mais importantes. O mesmo ocorre nos demais níveis

$$\lambda(e) = \lambda(f) = \lambda(g) = \lambda(h) = \lambda(i) = \lambda(j) = 2$$

$$\gamma(e) = \sqrt{18} \qquad \gamma(j) = \sqrt{10} \qquad \gamma(h) = \sqrt{8}$$

A ideia é que o alvo t atraia o caminho. Se t se move, a busca por caminhos mínimos usando A^* costuma ser bem mais eficiente que o Dijkstra em casos como esse.

Considere o seguinte exemplo: o grafo ponderado a seguir ilustra um conjunto de cidades e os pesos das arestas são as distâncias entre elas. Estamos situados na cidade s e deseja-se encontrar um caminho mínimo até a cidade e . O números em vermelho indicam o valor de $\gamma(v)$, ou seja, são uma estimativa para a distância de v até o destino e . Execute o algoritmo A* para obter o caminho mínimo de s a e .



Fila

	s	a	b	c	d	e	f	g	h	i	j	k	l
$\gamma^{(0)}(v)$	0	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞	∞
$\gamma^{(1)}(v)$		7+9	2+7	3+8	∞	∞	∞	∞	∞	∞	∞	∞	∞
$\gamma^{(2)}(v)$		5+9		3+8	6+8	∞	∞	∞	3+6	∞	∞	∞	∞
$\gamma^{(3)}(v)$		5+9		3+8	6+8	∞	6+6	5+3					
$\gamma^{(4)}(v)$		5+9		3+8	6+8	7	6+6						

Ordem de acesso aos vértices

u	$V' = \{v \in N(u) \wedge v \in Q\}$	$\lambda(v), \forall v \in V'$	$\pi(v)$
s	{a, b, c}	$\lambda(a) = \min\{\infty, 7\} = 7$ $\lambda(b) = \min\{\infty, 2\} = 2$ $\lambda(c) = \min\{\infty, 3\} = 3$	$\pi(a) = s$ $\pi(b) = s$ $\pi(c) = s$
b	{a, d, h}	$\lambda(a) = \min\{7, 2+3\} = 5$ $\lambda(d) = \min\{\infty, 2+4\} = 6$ $\lambda(h) = \min\{\infty, 2+1\} = 3$	$\pi(a) = b$ $\pi(d) = b$ $\pi(h) = b$
h	{f, g}	$\lambda(f) = \min\{\infty, 3+3\} = 6$ $\lambda(g) = \min\{\infty, 3+2\} = 5$	$\pi(f) = h$ $\pi(g) = h$
g	{e}	$\lambda(e) = \min\{\infty, 5+2\} = 7$	$\pi(e) = g$

Note como a ordem de retirada dos vértices da fila é orientada ao destino. Há um algoritmo que resolve exclusivamente o problema de calcular as distâncias geodésicas entre todos os pares de vértices do grafo: o algoritmo de Floyd-Warshall, cuja complexidade computacional é $O(n^3)$.

"Solutions are not found by pointing fingers; they are reached by extending hands."
-- Aysha Taryam

Bibliografia

CORMEN, T. H., Leiserson, C. E., Rivest, R. L., Stein, C. Introduction to Algorithms. 4ª edição, The MIT Press, 2022.

KLEINBERG, J.; TARDOS, E. Algorithmic Design, Addison Wesley, 2005.

T. ROUGHGARDEN. Algorithms Illuminated: The Omnibus edition. Soundlikeyourself Publishing, LLC, 2022.

R. SEDGEWICK. Algorithms in C, Parts 1-4: fundamentals, data structures, sorting, searching. 3rd. ed., Boston: Addison - Wesley, 1998.

R. SEDGEWICK. Algorithms in C, Part 5: graph algorithms., Boston: Addison - Wesley, 2002.

R. SEDGEWICK, K. WAYNE. Algorithms, 4th. ed., Addison-Wesley, 2011.

R. THAREJA. Data Structures Using C, 2a edição, Oxford University Press, 2018.

Y. LANGSAM, M. J. AUGENSTEIN, A. M. TENENBAUM. Data structures using C and C++. 2. ed. Upper Sadle River: Prentice Hall, 1996.

N. ZIVIANI. Projetos de algoritmos: com implementações em Pascal e C. 3. ed. rev. e ampl. São Paulo: Cengage Learning, 2012.

LEVITIN, A. Introduction to the Design and Analysis of Algorithms, 3ª edição, Pearson, 2012.

SKIENA, S. S. The Algorithm Design Manual, 2ª edição, Springer, 2008.

S. DASGUPTA, C.H. PAPADIMITRIOU, U.V. VAZIRANI. Algorithms, McGraw-Hill, 2007.

FELICE, M. C. S. Material e aulas das disciplinas Algoritmos e Estruturas de Dados I e II. Disponíveis em: <http://www.aloc.ufscar.br/felice/#teaching>

PUGH, W. Skip Lists: a probabilistic alternative to balanced trees, Communications of the ACM, vol. 33, n. 6, pp. 668-676, 1990.

SZWARCFITER, J. L. Teoria Computacional de Grafos: Os algoritmos, Elsevier, 2018.

ZIVIANI N. Projeto de algoritmos: com implementações em Java e C++. 2. ed. São Paulo: Cengage Learning, 2011.

NICOLETTI, M. C.; HRUSCHKA, E. R. Fundamentos da Teoria dos Grafos para Computação, 2ª ed., Série Apontamentos, EdUFSCar, 2009.

CLARK, J., DEREK, A. H. A First Look at Graph Theory, World Scientific, 1998.

Geeks for Geeks – A computer science portal for geeks. <https://www.geeksforgeeks.org/>

Sobre o autor

Alexandre L. M. Levada concluiu o curso de graduação em Bacharelado em Ciência da Computação no Departamento de Estatística, Matemática e Computação (DEMAC) da Universidade Estadual Paulista "Júlio de Mesquita Filho" (UNESP), em Rio Claro, no ano de 2002. Concluiu o mestrado em Ciência da Computação, na Universidade Federal de São Carlos (UFSCar), em fevereiro de 2006. Concluiu o doutorado em Física Computacional no Instituto de Física de São Carlos (IFSC) da USP em 2010 sob a orientação do Prof. Dr. Nelson Delfino d'Ávila Mascarenhas e co-orientação do Prof. Dr. Alberto Tannús. Atualmente é Professor do Departamento de Computação na Universidade Federal de São Carlos. Atua nas áreas de processamento de imagens e sinais e reconhecimento de padrões. Possui experiência em ministrar diversas disciplinas para cursos de Ciência e Engenharia de Computação, como Lógica Matemática, Teoria dos Grafos, Matemática Discreta, Algoritmos e Estruturas de Dados 1 e 2, Projeto e Análise de Algoritmos, Otimização Matemática, Processamento Digital de Imagens, Reconhecimento de Padrões e Programação Científica.

"Being happy doesn't mean everything is perfect. It means you've decided to look beyond the imperfections."
~ Author Unknown